

Department of Computer Science & Engineering
University of Ioannina

Token- and Resource-Aware Long-Term Memory for Small On-Device Agents

Ioannis Bouzas
Supervisor: Konstantinos Skianis

Undergraduate Thesis
Ioannina, October 2026

Περίληψη

Τα μικρά γλωσσικά μοντέλα μπορούν να εκτελούνται σε κινητά τηλέφωνα, αλλά η διατήρηση μακρών αλληλεπιδράσεων επιβαρύνει το περιορισμένο παράθυρο συμφραζομένων, την μπαταρία, τη μνήμη και τα θερμικά τους όρια. Οι μελέτες σε φορητές συσκευές που εξετάσαμε δεν συγκρίνουν αναπαραστάσεις μνήμης για ένα ιστορικό συνομιλίας ως προς την ποιότητα των απαντήσεων και την ενέργεια στην ίδια συσκευή. Η παρούσα διπλωματική εργασία εξετάζει πώς οι στρατηγικές μνήμης επηρεάζουν την ποιότητα των απαντήσεων, ποιους συμβιβασμούς πόρων δημιουργούν σε ένα κινητό τηλέφωνο και πού η μνήμη με μορφή γράφου γνώσης επιτυγχάνει ή αποτυγχάνει.

Μια κοινή υλοποίηση συγκρίνει το πλήρες ιστορικό με περικοπή, τη διανυσματική ανάκτηση και γράφους που εξάγονται από μικρά γλωσσικά μοντέλα. Η κύρια μελέτη εκτελείται σε ένα Samsung Galaxy S23 Ultra και μετρά την ακρίβεια, την ενέργεια, τον χρόνο απόκρισης, τη χρήση μνήμης και τη θερμοκρασία, αναφέροντας την ακρίβεια και την ενέργεια χωριστά και ως σωστές απαντήσεις ανά κιλοτζάουλ (kJ). Ένας σταθμός εργασίας εκτελεί τις ίδιες μονάδες με περισσότερες ερωτήσεις και μοντέλα Gemma και Qwen στα LoCoMo και LongMemEval, και εξετάζει το βάθος ανάκτησης, τις διορθώσεις των γράφων και την επαναφορά του αρχικού κειμένου.

Στο τηλέφωνο, η διανυσματική ανάκτηση απάντησε σωστά σε περισσότερες ερωτήσεις από το πλήρες ιστορικό με περικοπή και κατανάλωσε λιγότερη ενέργεια ανά σωστή απάντηση. Στον σταθμό εργασίας, το πλήρες ιστορικό με μεγάλο παράθυρο συμφραζομένων είχε υψηλότερες σημειακές εκτιμήσεις ακρίβειας από τη διανυσματική ανάκτηση στο LoCoMo και χαμηλότερες στο LongMemEval-S, με πολλαπλάσιο μήκος εισόδου. Σε μία πιλοτική κατασκευή στο τηλέφωνο, η δημιουργία γράφου κόστισε σημαντικά περισσότερη ενέργεια από τη δημιουργία του διανυσματικού ευρετηρίου. Η ακρίβεια της μνήμης γράφου μετρήθηκε στον σταθμό εργασίας. Εκεί, η μνήμη γράφου περιορίστηκε από αποτυχίες εξαγωγής και από τον τρόπο με τον οποίο τα στοιχεία έφταναν στο μοντέλο, ενώ η επαναφορά του αρχικού κειμένου πίσω από τα ανακτημένα γεγονότα βελτίωσε τις απαντήσεις των γράφων για τα περισσότερα μοντέλα, χωρίς να φτάσει τη διανυσματική ανάκτηση. Η μέθοδος Επιλεκτικής Επέκτασης Πηγών (Selective Provenance Expansion, SPE), που αναπτύχθηκε για να επιλέγει ανάμεσα σε συμπαγή γεγονότα και αρχικό κείμενο υπό όριο λεκτικών μονάδων (tokens), δεν ανέκτησε με συνέπεια αυτό το όφελος.

Τα ευρήματα συνηγορούν στην αξιολόγηση της μνήμης τόσο ως προς τα στοιχεία που παρέχει όσο και ως προς τους πόρους που απαιτεί η κατασκευή και η χρήση της. Περιορίζονται από τη μία συσκευή, το μικρό δείγμα στο τηλέφωνο και την αυτόματη βαθμολόγηση, και αφορούν ερωταποκρίσεις και όχι ολοκληρωμένες ροές εργασίας κινητών πρακτόρων.

Λέξεις-κλειδιά: μικρά γλωσσικά μοντέλα, μακροπρόθεσμη μνήμη, παραγωγή επαυξημένη με ανάκτηση, γράφοι γνώσης, τοπική εκτέλεση μοντέλων, ενεργειακή αποδοτικότητα.

Abstract

Small language models can run on smartphones, but remembering long interactions strains their limited context, battery, memory and thermal headroom. The on-device studies reviewed here do not compare memory representations for a conversation history by both answer quality and energy on one handset. This thesis examines how memory strategies affect answer quality, what resource trade-offs they create on a phone, and where knowledge-graph memory succeeds or fails.

A shared pipeline compares truncated full context, vector retrieval and graphs extracted by small language models. The main study runs on a Samsung Galaxy S23 Ultra and measures accuracy, energy, latency, memory use and temperature, reporting accuracy and energy separately and as correct answers per kilojoule. A workstation runs the same modules with more questions and Gemma and Qwen readers on LoCoMo and LongMemEval, and tests retrieval depth, graph repairs and source-text restoration.

On the phone, vector retrieval answered more questions correctly than truncated full context and used less energy per correct answer. On the workstation, full context with a long window had higher accuracy point estimates than vector retrieval on LoCoMo and lower ones on LongMemEval-S, while using many times more prompt tokens. In a single construction pilot on the phone, building a graph cost substantially more energy than building the vector index. The accuracy of graph memory was measured on the workstation. There, graph memory was limited by extraction failures and by how its evidence reached the reader, and restoring the source text behind retrieved facts improved graph answers for most readers without reaching vector retrieval. Selective Provenance Expansion, developed to choose between compact facts and source text under a token budget, did not reliably recover that benefit.

The results suggest evaluating memory by both the evidence it delivers and the resources needed to build and use it. They are limited by one handset, a small phone sample and automatic judging, and cover question answering rather than complete mobile-agent workflows.

Keywords: small language models; long-term memory; retrieval-augmented generation; knowledge graphs; on-device inference; energy efficiency.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem statement . . . 1
- 1.3 Research questions . . . 2
- 1.4 Approach and scope . . . 3
- 1.5 Contributions . . . 3
- 1.6 Thesis organization . . . 4

2 Background and related work **5**
- 2.1 Small language models and local inference . . . 5
  - 2.1.1 The model within an agent . . . 5
  - 2.1.2 Context windows and persistent memory . . . 6
  - 2.1.3 Prefill, decoding and quantization . . . 6
- 2.2 Representations of long-term conversational memory . . . 7
  - 2.2.1 The memory life cycle . . . 7
  - 2.2.2 Dense retrieval over dialogue . . . 8
  - 2.2.3 Compression, hierarchy and memory management . . . 8
- 2.3 Graph retrieval and preservation of evidence . . . 9
  - 2.3.1 What the graph represents . . . 9
  - 2.3.2 Construction, retrieval and reader limitations . . . 9
  - 2.3.3 Provenance and source-text expansion . . . 10
- 2.4 Efficient construction and budget-aware memory use . . . 10
  - 2.4.1 When to construct memory . . . 10
  - 2.4.2 Routing and selective processing . . . 10
- 2.5 On-device memory and physical resource costs . . . 11
  - 2.5.1 The closest mobile studies . . . 11
  - 2.5.2 Tokens, time, power and energy . . . 12
- 2.6 Evaluating conversational memory . . . 13
  - 2.6.1 LoCoMo and LongMemEval . . . 13
  - 2.6.2 Automatic judging and comparison boundaries . . . 13
- 2.7 Synthesis and position of the thesis . . . 14

3 Methodology **16**
- 3.1 Research design: one pipeline, two measurement settings . . . 16
- 3.2 Language models and runtime environment . . . 17
- 3.3 Memory strategies . . . 19
  - 3.3.1 Vector retrieval . . . 20
  - 3.3.2 Graph construction and retrieval . . . 21
- 3.4 Benchmarks and task configuration . . . 22
  - 3.4.1 Fixed configuration and the one-variable rule . . . 24

3.5 Retrieval configuration . . . 24
3.6 Answer scoring and judge characterization . . . 25
3.7 On-device resource measurement . . . 26
3.8 Controls, validity, and statistical plan . . . 28
3.9 Software and reproducibility . . . 29

4 Memory Strategy Evaluation 30
4.1 Evaluation scope and statistical interpretation . . . 30
4.2 LoCoMo strategy comparison . . . 31
4.2.1 Vector retrieval versus truncated full context . . . 31
4.2.2 Vector retrieval versus the triple-only graph . . . 33
4.2.3 What source restoration changes . . . 33
4.2.4 Where the differences occur . . . 34
4.3 Model scaling within each memory strategy . . . 35
4.4 Retrieval-depth analysis . . . 36
4.5 Transfer from LoCoMo to LongMemEval-S . . . 39
4.6 Full context with a long window . . . 41
4.7 Reasoning experiments . . . 45
4.8 Robustness checks . . . 47
4.9 Experimental limitations . . . 48
4.10 Summary . . . 49

5 Resource-aware long-term memory on a smartphone 50
5.1 A resource-aware objective . . . 50
5.2 Experimental boundary . . . 51
5.3 The two statistical results used in this chapter . . . 51
5.3.1 Workstation accuracy cannot substitute for phone accuracy . . . 51
5.3.2 Question family changes the observed difficulty . . . 52
5.4 Measured phone comparison . . . 52
5.5 Correct answers per charge . . . 53
5.6 Latency, RAM and thermal headroom . . . 54
5.6.1 Prefill dominance and KV-cache size . . . 56
5.7 The measured graph-construction boundary . . . 58
5.8 Position relative to mobile systems literature . . . 61
5.9 Implications for resource-aware long-term memory . . . 61
5.10 Experimental limitations . . . 61
5.11 Chapter conclusion . . . 62

6 Repairing the graph memory pipeline: what improved and what remained 63
6.1 What the graph baseline represents . . . 63
6.2 The repair sequence . . . 64
6.2.1 GraphRAG v1: entity-match retrieval . . . 64
6.2.2 GraphRAG v2: turn-first retrieval . . . 65
6.2.3 GraphRAG v3: larger extraction budget . . . 66
6.2.4 GraphRAG v4: source turns restored . . . 66
6.3 How much the repairs recovered . . . 66
6.4 What reached the answer model . . . 68
6.5 Would more triples be enough? . . . 69
6.6 Extraction quality and model choice . . . 69
6.6.1 Structure of the repaired Gemma graphs . . . 70
6.6.2 Empty responses and discarded values . . . 71

6.6.3 Separate the model that builds the graph from the model that answers . 71
6.7 What the study establishes and where it stops . . . 71
6.8 From full source restoration to selective expansion . . . 72

7 Selective Provenance Expansion 73
7.1 Why source text may matter . . . 73
7.2 SPE design . . . 74
7.2.1 SPE-1: a simple heuristic score . . . 75
7.2.2 SPE-2: score the two representations separately . . . 75
7.3 Evaluation against the pre-registered criteria . . . 75
7.3.1 The selector repair and why the repaired rule is the one evaluated . . . 76
7.3.2 Registered E2B results . . . 76
7.3.3 E4B addendum and reader–extractor factorization . . . 77
7.3.4 Why SPE retained only triples . . . 77
7.4 Discussion . . . 79
7.5 Limitations . . . 80
7.6 Summary . . . 80

8 Discussion 81
8.1 RQ1: how memory strategy affects answer quality . . . 81
8.2 RQ2: what memory strategy costs on a phone . . . 84
8.3 RQ3: what graph memory can and cannot do here . . . 84
8.4 What it took to make these measurements trustworthy . . . 85
8.5 Threats to validity and what the results do not show . . . 85

9 Conclusion 87
9.1 Findings . . . 87
9.2 Implications for on-device memory . . . 88
9.3 Future work . . . 88

A Supplementary analyses for on-device memory 90
A.1 Cross-device agreement . . . 90
A.2 Question-family randomization test . . . 91
A.3 Phone energy replication . . . 91
A.4 Run-length observation . . . 92
A.5 Thermal peak detail . . . 93
A.6 Long-context and routing sensitivities . . . 93
A.7 Energy and charge conventions . . . 95

B Reproducibility record 97
B.1 Prompts . . . 97
B.2 Question sets . . . 99
B.3 Memory configuration . . . 100
B.4 Worked prompt example . . . 101
B.5 Model artifacts . . . 102
B.6 Runtime configuration . . . 102
B.7 Software versions and phone server commands . . . 103
B.8 Instrumentation . . . 103
B.9 Analysis scripts . . . 104
B.10 Known limitations of this record . . . 104
B.11 Glossary . . . 104

Chapter 1

Introduction

1.1 Motivation

Small language models make it possible to run language-based applications on a smartphone, close to the user and without a remote inference service [Xu et al., 2024]. Running on the device is only part of an assistant's usefulness, however. An assistant that supports repeated interactions must also retain what it learned earlier, such as a user's preferences, earlier requests, observations of the device and the outcomes of previous actions, and recover it when it becomes relevant again. Acting on a previously stated preference is therefore a memory problem as well as an inference problem.

The resources for this process are limited. A smartphone shares its memory and processing capacity with the operating system and other applications, and battery capacity and heat limit how much computation can be sustained; studies of small-model deployment therefore evaluate prompt processing, generation speed and memory alongside model capability [Lu et al., 2025]. Keeping the whole interaction history in the model's context window is the simplest way to make past information available, but the history keeps growing, and longer inputs require more prompt processing and more storage for attention states, depending on the architecture and runtime [Xu et al., 2024]. Even when a long context fits in memory, processing it may take too much time or energy for repeated use on a phone.

External memory separates the amount of information retained from the amount presented to the model for a particular request. A system can store a long history and retrieve only a small part of it, following the retrieval-augmented generation approach of supplying external evidence at inference time [Lewis et al., 2020]. This creates a design choice: how should the history be represented, which parts should be retrieved, and what should the model receive within its available context budget? This thesis studies that choice for small language models, with particular attention to the physical cost of memory use on a smartphone.

1.2 Problem statement

A useful memory strategy must make relevant past information available at a cost the application can afford. Storage alone does not meet this requirement: the stored information must be retrievable, the retrieved representation must preserve enough detail for the model to use it correctly, and construction, retrieval and answer generation all add to the cost.

This thesis compares three approaches. The full-context baseline presents the dialogue history within a configured input limit, truncating it when necessary. Vector retrieval stores dialogue text and selects relevant turns using embedding similarity. The graph approach asks a small language model to extract subject–relation–object triples, stores the resulting relations and retrieves graph evidence for answering. These approaches differ both in the work required

to prepare memory and in the evidence subsequently shown to the answer model, referred to here as the *reader*.

Knowledge graphs are of particular interest because they represent explicit relations between entities. A compact relational representation may allow the reader to receive several relevant facts without processing their full conversational context. Its utility depends, however, on the extraction and retrieval stages. A missing relation cannot be recovered through traversal alone, and a short triple may omit a qualification or temporal detail needed to answer a question. Prior comparisons of retrieval and graph retrieval show that performance depends on the graph design and the evidence supplied to the reader [Han et al., 2026]. This motivates testing specific graph-memory choices rather than assuming that structure is beneficial by itself.

The same distinction applies to efficiency. A short query prompt does not reveal the cost already paid to build its memory: generating a graph requires repeated model calls, while vector indexing embeds the stored text. Whether that work pays off depends on answer quality and on how often the memory is reused, so an on-device comparison must account for measured physical cost as well as prompt length.

Related work already examines parts of this problem. MobileRAG evaluates retrieval for document question answering on a smartphone, including its energy cost [Park et al., 2025]. SmartRAG studies an on-device graph-based system and reports memory and power measurements, while its comparison baselines use a cloud-side simulation [Jiang et al., 2026]. A benchmark of memory architectures for compact mobile agents measures latency on a laptop and lists real-phone latency and device-level cost measurement as future work [Zhang et al., 2026c]. These studies each cover part of the problem, but they do not compare alternative memory representations for a conversation history on one phone while measuring both answer quality and energy. This thesis makes that comparison with small readers, combining a broader workstation evaluation with direct smartphone measurements and a detailed analysis of the tested graph pipeline.

1.3 Research questions

The thesis addresses three research questions.

RQ1. How do different long-term memory strategies affect the question-answering performance of small language models?

RQ2. What trade-offs between answer quality and resource consumption arise when deploying long-term memory strategies on smartphones?

RQ3. What are the capabilities and limitations of knowledge-graph-based memory for small language models under constrained context and resource budgets?

RQ1 establishes the quality comparison, including how it depends on retrieval depth, reader size and the context budget. RQ2 relates answer quality measured on the phone to energy, latency, memory use and thermal behavior; correct answers per kilojoule give a joint measure, while the individual resource measurements explain practical deployment constraints. RQ3 asks how extraction, evidence selection and representation affect what reaches the reader, what repairing retrieval or restoring source text recovers, and whether Selective Provenance Expansion (SPE), a policy developed in this thesis, can choose whether a retrieved fact is omitted, kept as a triple or accompanied by its source text under a context budget.

The phone comparison of answer quality covers vector retrieval and truncated full context; graph memory enters the phone study through a directly measured construction pilot. The graph repairs and SPE are evaluated for quality on the workstation, and their prompt lengths and retrieval times do not substitute for phone-energy measurements.

1.4 Approach and scope

The experiments use question answering to evaluate whether a memory system can recover and supply information from an extended interaction history. LoCoMo provides long conversations and questions that require using information from those conversations [Maharana et al., 2024]. LongMemEval evaluates long-term interactive memory across several abilities, including information extraction, reasoning across sessions, temporal reasoning, knowledge updates and abstention [Wu et al., 2025]. The thesis uses specified subsets and configurations of these benchmarks, described in Chapter 3.

These tasks give the comparison an observable outcome: whether the answer produced from the available memory is correct under a common judging procedure. They also allow the memory representation to change while the question and underlying history remain fixed. Mobile agents motivate the study broadly, but the measured task is memory-dependent question answering. Improvements in action selection, tool-call correctness or complete mobile workflows would require additional evaluation.

One experimental pipeline runs in two settings that complement each other. The main platform is a Samsung Galaxy S23 Ultra, where the memory modules run locally and each run records the phone's answers together with their energy, battery charge, latency, memory use and temperature. A workstation runs the same modules on more questions, five readers, both benchmarks and several graph variants, more than a phone can process in practical time. One automatic judge, run on the workstation outside the measured phone workload, scores both. Device runtimes and model packaging can change the generated answers, so a workstation score is never used as a phone score: every phone efficiency figure uses the accuracy of the answers the phone itself produced.

The findings concern the evaluated models, histories and execution conditions. The phone measurements come from one handset, with a smaller question sample and uneven replication across phone conditions. Graph construction on the phone is represented by an early pilot, not a complete graph-QA comparison. The study also characterizes the repeatability of its automatic judge and checks its verdicts against a blind human audit with a single annotator. These limits define how far the results can be generalized and are developed in the relevant chapters.

1.5 Contributions

The thesis contributes an empirical comparison of long-term memory strategies for small language models and an analysis of the costs and failure mechanisms that shape their use on a phone. Its main contributions are:

1. A controlled comparison of full context, truncated and with a long window, vector retrieval and SLM-extracted graph memory for conversational question answering. The evaluation examines reader size and retrieval depth alongside memory strategy, with a second benchmark used to assess whether the observed ordering persists beyond the primary question set.

2. A smartphone study that relates answer quality to measured energy and other device resources. It reports correct answers per unit of energy alongside latency, memory and thermal observations, and compares the energy of building vector and graph memory on the tested handset.

3. An analysis of graph-memory limitations through successive retrieval repairs, source-text restoration and controls that distinguish extractor choice from reader choice. The analysis

identifies several stages at which useful evidence can be lost or fail to reach the reader, rather than attributing all errors to a single property of knowledge graphs.

4. The design and evaluation of Selective Provenance Expansion against criteria recorded before its evaluation runs. The result is negative: the tested policies did not reliably recover the benefit of source-text restoration under the evaluated budgets, and the repaired selector chose no source turns. The measured costs are consistent with the per-token scoring rule and the budget favouring triples, which bounds this per-fact policy without ruling out query-level selective expansion.

Each of Chapters 3 to 7 opens by stating what it adds to the work reviewed in Chapter 2, and Tables 8.1 and 8.2 list the main claims with their sample sizes, tests and status.

On the phone, vector retrieval had a higher accuracy point estimate, lower energy per answer and shorter query time than truncated full context in every reader pair, with 5.8–26.9× more correct answers per kilojoule where the ratio is defined; on the workstation, with full context truncated to a 4,096-token window, it had the highest accuracy point estimate for every reader. Long-window full context, which held the whole LoCoMo history, matched or exceeded vector retrieval on LoCoMo and trailed it, without separating, on the much longer LongMemEval-S histories, at about 39 and 48 times the prompt tokens. Building memory in the phone graph pilot cost about 26 times more energy per dialogue turn than vector ingestion. Restoring source text raised Gemma 4 E2B's graph accuracy from 21.5% to 33.0% and narrowed the gap to vector retrieval for some readers without closing it for any, and the repaired SPE selector chose no source turns.

1.6 Thesis organization

Chapter 2 introduces small language models, external memory, graph retrieval and on-device execution, and positions the thesis against related work. Chapter 3 describes the shared pipeline, benchmarks, judging procedure and resource measurements.

Chapter 4 addresses RQ1 through the memory-strategy comparison and its sensitivity to retrieval depth and reader choice. Chapter 5 addresses RQ2 through the smartphone measurements. RQ3 spans Chapter 6, which analyzes the graph-repair sequence, and Chapter 7, which presents and evaluates SPE.

Chapter 8 brings the findings together as answers to the three research questions and examines their limitations. Chapter 9 sets out the implications for on-device memory and the remaining research directions. Supporting statistical analyses appear in Appendix A, and Appendix B records the prompts, question sets, model artifacts and runtime settings needed to interpret and repeat the experiments.

Chapter 2

Background and related work

A conversational memory system determines which parts of an interaction history remain available to a language model and in what form. Keeping the original dialogue, retrieving selected passages and constructing a graph make different demands on the model and on the device running it. This chapter separates the information an application stores from the resources needed to use it, introduces the memory mechanisms evaluated in the thesis, and reviews the work most relevant to their accuracy and resource costs on small models and smartphones.

Much of the closest work is recent and was available only as an arXiv preprint when this chapter was written. This includes the two on-device systems SmartRAG and MobileRAG, the source-expansion system LeanMem, and the memory systems LazyMem, MemFlow, MemoryCPT, RippleMem and MOSAIC [Jiang et al., 2026, Park et al., 2025, Liao et al., 2026, Yu et al., 2026b, Chen et al., 2026, Lei et al., 2026, Ji et al., 2026, Zhao et al., 2026]. Preprints have not been peer reviewed. They are cited here for the designs and measurements their authors report. They give context for the thesis's results but are not evidence for them. Every reference was checked against its arXiv or publisher record. Where a preprint has since appeared at a conference or in a journal, the bibliography cites that version.

2.1 Small language models and local inference

2.1.1 The model within an agent

A language model predicts tokens conditioned on the sequence supplied to it. An agent places that model inside a system that can receive observations, call tools, maintain state and act on its environment. Memory is one component of this system: it supplies information from earlier interactions that is relevant to a current decision. The survey by Zhang et al. [2025] distinguishes the model's capabilities from the mechanisms that store, retrieve and process its interaction history.

The argument for using small language models (SLMs) in agents rests partly on this division of work. An application may repeatedly need a restricted operation, such as classifying a request or answering from supplied evidence. Belcak et al. [2025] argue that compact models can serve such operations economically and that agent systems can combine models of different sizes. This is a design argument for selecting models by their assigned task; it does not establish that a small model can perform every memory operation reliably. Extracting relations, resolving conflicting statements and answering from retrieved text remain distinct capabilities that require evaluation.

There is no single parameter threshold that determines whether a model is practical on a phone. Architecture, weight precision, context length and runtime support also affect its resource requirements. The on-device review by Xu et al. [2024] therefore treats model design,

compression and hardware execution together. In an empirical study of compact models, Lu et al. [2025] likewise find that model size alone does not explain capability or execution speed. This thesis uses SLM to describe its evaluated compact readers; Chapter 3 specifies their identities and parameter scales. As in Chapter 1, the reader is the model that receives the selected evidence and generates the answer. When another model prepares the memory, its role and cost must be specified separately.

2.1.2 Context windows and persistent memory

Three meanings of memory need to be distinguished. *Parametric memory* refers to information encoded in model weights through training. The *context window* contains the tokens available to a particular inference call. *External memory* is information stored by the surrounding application and made available to later calls. Retrieval-augmented generation connects an external store to a language model without requiring each new fact to be incorporated into its weights [Lewis et al., 2020]. MemGPT makes the distinction between limited in-context working space and longer-term external storage explicit through a hierarchy of memory tiers [Packer et al., 2023].

Appending an interaction to a prompt does not, by itself, update the model's weights. If that interaction is omitted from a later call, the application needs another mechanism to recover it. A large context window postpones the point at which history must be selected or compressed, but it does not specify how a growing history should be managed. Moreover, input capacity and effective use of evidence are different properties. Liu et al. [2024] show that answer quality can depend on where relevant information appears within a long input, including for models designed to accept long contexts. Their result motivates testing context use rather than treating the advertised context length as a guarantee of recall.

In a bounded full-context baseline, the reader receives as much of the history as the configured input limit permits. Once that limit is exceeded, a truncation policy determines which material remains visible. Such a baseline is a useful reference for the cost of reading dialogue directly, but its name must not imply that every historical turn is present. Retrieval addresses a different selection problem: it chooses evidence according to the current query instead of relying only on the portion of the history that fits.

2.1.3 Prefill, decoding and quantization

Autoregressive inference has two stages with different resource demands. During *prefill*, the model processes the input tokens and prepares the internal state used for generation. During *decoding*, it generates subsequent tokens sequentially. For attention layers, cached key and value tensors allow the model to reuse representations of earlier tokens. This *KV cache* occupies runtime memory in addition to the model weights. Its size depends on the retained sequence length and the attention architecture, so a model whose weights fit in RAM can still become impractical with a large context [Xu et al., 2024, Lu et al., 2025].

Weight quantization reduces the precision used to represent model parameters. It can reduce storage and memory-bandwidth demand, but its benefits depend on the implementation and hardware. Weight precision also does not determine the precision or size of every runtime buffer. In their edge measurements, Lu et al. [2025] distinguish the predominantly compute-bound prefill stage from the predominantly memory-bandwidth-bound decode stage and show that quantization affects them differently. The relevant bottleneck can therefore change when a memory strategy replaces a long input with additional generated summaries or extraction outputs.

This distinction matters for cost accounting. Reader input tokens, reader output tokens and

tokens generated while constructing memory perform different jobs. Counting them is useful, but adding them into one total conceals the operations responsible for that cost. A resource study must record both the work performed and the device resources consumed.

2.2 Representations of long-term conversational memory

2.2.1 The memory life cycle

External memory involves decisions at several points in an interaction. At ingestion, the system chooses what to retain and may transform raw observations into summaries, facts or structured records. During maintenance, it may merge related entries, revise outdated information or remove material. At query time, retrieval selects candidate evidence and context assembly decides how to present it to the reader. The agent-memory and graph-memory surveys organize these operations as connected parts of a memory life cycle [Zhang et al., 2025, Yang et al., 2026].

Conversational information also differs in how it changes. A stable preference may be useful across many sessions, while an appointment has a date and a decision may later be reversed. Preserving only the latest statement can lose the evidence needed to answer a historical question. THEANINE addresses this issue by linking memories through temporal and causal relations and retrieving timelines, retaining earlier information that helps explain a change [Ong et al., 2025]. Its design illustrates why forgetting and updating policies must be chosen in relation to the task.

Table 2.1 summarizes four recurring representation choices. These are overlapping design families. A graph can retain pointers to raw turns, and a vector index can store summaries instead of verbatim text. The three experimental arms in this thesis select particular configurations from this broader space; Figure 3.2 in Chapter 3 shows how each of the three looks up evidence.

Representation	Information supplied to the reader	Main trade-off
Full context	Original history within an input limit	Preserves included wording; input grows with history and truncation can remove evidence.
Vector retrieval	Text records selected by embedding similarity	Limits reader input; retrieval may miss supporting records or return redundant text.
Compressed memory	Summaries or selected factual records	Reduces repeated text; construction costs work and may discard future query details.
Graph memory	Retrieved entities, relations, events or linked source passages	Supports associative access; usefulness depends on construction, traversal and representation.

Table 2.1: Memory representation choices, synthesized from the reviewed literature [Zhang et al., 2025, Lewis et al., 2020, Chen et al., 2024, Yang et al., 2026]. The trade-offs describe design considerations, not a measured ranking.

2.2.2 Dense retrieval over dialogue

Dense retrieval represents a query and stored text as numerical vectors. DPR establishes a dual-encoder approach in which questions and passages are mapped into a shared space and ranked using a similarity score [Karpukhin et al., 2020]. Contriever demonstrates that contrastive training can produce useful dense retrieval without supervised relevance labels for each target dataset [Izacard et al., 2022]. These methods provide foundations for indexing dialogue with an existing embedding model, although performance on conversational histories still depends on the encoder and the way records are formed.

For a query q and stored record d_i, a common score is cosine similarity:

$$s(q, d_i) = \frac{f(q)^\top g(d_i)}{\|f(q)\| \, \|g(d_i)\|}, \tag{2.1}$$

where f and g denote the query and record encodings. The top-k records are candidates for the reader's context. This notation describes the retrieval operation used in the thesis; dense retrievers can use other scoring functions, including the unnormalized inner product used by DPR.

The retrieval unit affects what a successful match contains. A short turn can isolate one fact but leave a pronoun unresolved. A longer passage can retain its surrounding exchange while also introducing irrelevant content. Speaker identifiers and timestamps can be necessary for interpreting the retrieved words. These are representation choices before the reader sees the evidence, not properties of the reader itself.

Increasing k can bring more supporting material into the candidate set, but it also increases the amount of text that must be selected, ordered and read. Chain-of-Memory examines the resulting separation between retrieving relevant evidence and answering correctly from it [Xu et al., 2026]. Consequently, retrieval coverage and answer accuracy are complementary evaluation targets. High coverage identifies available evidence; it does not establish that the context assembled from that evidence is useful to a particular reader.

2.2.3 Compression, hierarchy and memory management

These systems share one decision: how much interpretation to perform before the future question is known. Some compress what is stored. COMEDY maintains a compressive memory in place of a retrieval database [Chen et al., 2024], RAPTOR recursively clusters and summarizes text into a tree of abstraction levels [Sarthi et al., 2024], and MemoRAG uses a global memory of the input to generate retrieval clues [Qian et al., 2025]. Others manage records as they arrive. MemGPT lets the model move information between in-context and external storage [Packer et al., 2023], Mem0 extracts and updates salient facts, with a graph variant for relations [Chhikara et al., 2025], and A-MEM links notes and lets new memories revise existing ones [Xu et al., 2025]. Hierarchical designs add structure on top: Yu et al. [2026a] pair a temporal and semantic tree with a knowledge graph of the entities it mentions, and the H-MEM system of Sun et al. [2026] routes retrieval between levels of semantic abstraction, with LoCoMo scores that belong to its own reader and protocol.

Early compression can reduce later reader work, but it commits to a view of what is worth preserving. Retaining raw dialogue keeps more information available and moves selection work to query time, and systems that keep source pointers beside compact records partly defer the decision. This thesis approaches the same question from the other side: what a triple loses when the text behind it is not shown.

2.3 Graph retrieval and preservation of evidence

2.3.1 What the graph represents

A knowledge graph represents entities and relations between them. A subject–relation–object triple, such as (person, works at, organization), is one possible unit. Conversational systems may attach timestamps, confidence values and source references to that unit. Other graph memories connect complete events or text passages instead of individual entities. The graph-memory taxonomy of Yang et al. [2026] covers these different representations and the operations used to construct and retrieve them.

GraphRAG is therefore a family label as well as the name of a specific system. The approach of Edge et al. [2024] builds an entity graph, forms communities and generates community summaries for query-focused summarization over a corpus. HippoRAG combines knowledge-graph construction with Personalized PageRank to retrieve information through associative connections [Jiménez Gutiérrez et al., 2024]. LightRAG uses graph and vector representations with retrieval at both detailed entity and broader thematic levels [Guo et al., 2025]. Different choices of graph nodes, edges and retrieval procedures lead to different evidence being shown to the reader.

The graph arm of this thesis uses SLM-extracted triples from dialogue and retrieves evidence from that store. It is a particular graph-memory pipeline, described in Chapter 3. Its behavior cannot stand for every system bearing the GraphRAG name. In particular, a community-summary system and a graph linking intact episodes preserve and expose information differently from a triple-only prompt.

2.3.2 Construction, retrieval and reader limitations

Graph construction can introduce errors before retrieval begins. The extractor must identify the relevant entities and relations, preserve their meaning and produce an output that the storage pipeline can interpret. Valid syntax alone does not establish that a fact was extracted correctly. A missing relation and a relation that has lost a qualification are different errors, and both can affect later answering. The loss can be large when a small model performs the extraction. In a multi-hop kinship-reasoning task, Kelesis et al. [2026] report that an RGCN model reaches 60.69% accuracy on ground-truth facts but 24.88% on facts extracted by a 4B Gemma model.

The controlled comparison by Han et al. [2026] shows that graph results depend on the construction model, the retrieval design and the form of evidence supplied to the reader, including triples alone and triples with source text. Giving plain retrieval the same token budget as community-based graph retrieval on MultiHop-RAG removes most of the graph's advantage: with a 70B reader, token-matched retrieval reaches 71.01% against 71.17% for the graph, compared with 65.77% under the default budget, although the graph stays ahead on temporal and comparison questions. Representation and context volume are therefore necessary controls in a graph comparison. Preparation cost also differs: the same study reports 135 seconds to build its RAG index against 5,560 to 7,702 seconds for its graph variants.

Conversational systems with richer graphs show that graph memory can work well. RippleMem connects event-centred memory units through participants, locations and time, retrieving anchors and expanding through their associations [Ji et al., 2026]. MOSAIC stores entity-typed nodes, checks new items for contradictions at save time, and replaces an LLM classification step with locality-sensitive hashing at negligible accuracy cost [Zhao et al., 2026]. Both build richer memory units than triples emitted by a local small model, and their reported LoCoMo results reflect those designs. A graph result is therefore a property of the whole construction and retrieval design, and the expensive part of that design need not be an additional language-model

call.

2.3.3 Provenance and source-text expansion

Provenance records where a memory item came from. A pointer from a triple or summary to the original dialogue can support inspection and allow the reader to recover details that the compact representation omitted. For example, a statement about a former employer and a statement about a current employer can collapse into the same undated relation if temporal wording is discarded. A source pointer makes the original distinction available without requiring it to be reproduced in every compact record.

SEEM combines an episodic event layer with a graph layer and connects both to their source passages. Its Reverse Provenance Expansion retrieves original passages through the provenance links of recalled event frames [Lu et al., 2026]. LeanMem distinguishes stable profile memory, evolving event memory and detail-preserving record memory. Its records retain pointers to original dialogue spans, which can be expanded when a query requires detailed evidence [Liao et al., 2026]. Both systems explicitly address the loss of information when interaction history is condensed.

SEEM and LeanMem are therefore the closest related work for the Selective Provenance Expansion (SPE) study in Chapter 7. The specific question examined there is how to choose among omitting a retrieved fact, retaining its triple and supplying its source turn under a fixed retrieval-context budget. The study evaluates a loss-aware selection rule with compact readers; its scope is the behavior of that rule and its implementation.

2.4 Efficient construction and budget-aware memory use

2.4.1 When to construct memory

An efficient memory design must account for work performed before a question is asked. The AI Hippocampus finds on LongMemEval-S that a simple vector store can be competitive with more elaborate memory systems while needing much less processing time [Jia et al., 2025], so the benefit of construction must be measured against a plain retrieval baseline. Other systems change when and how processing happens. Chain-of-Memory keeps preparation light and organizes retrieved fragments into memory chains at query time [Xu et al., 2026]. LazyMem and DeferMem retrieve a broad candidate pool and use a trained lightweight model to compress or rewrite it into query-conditioned evidence [Yu et al., 2026b, Yin et al., 2026]. MemoryCPT instead distils the construction pipeline into one compact model and scores systems by quality per unit of inference cost, measured as a dollar price over language-model tokens [Lei et al., 2026]; a distilled construction model is one way to reduce the phone construction energy measured in Chapter 5.

Deferral changes the cost distribution. It reduces eager, query-independent construction but adds processing to every query, while storing or indexing observations still costs work. An amortization calculation for a graph built once and reused across many questions therefore does not describe every memory architecture; a query-time construction system needs its own measurement boundary, including any training or offline preparation relevant to the deployment.

2.4.2 Routing and selective processing

Budget-aware memory systems decide how much work a query warrants. BudgetMem offers memory-processing modules at low, medium and high budget tiers, varying implementation complexity, reasoning effort or model capacity, and trains a compact router among them [Zhang

et al., 2026b]; Chapter 4 places the retrieval-depth sweep relative to these axes. MemFlow addresses compact readers directly: intent routing chooses among profile lookup, targeted retrieval and deeper reasoning, evidence is assembled under a tiered token budget, and a validator can escalate to a heavier path [Chen et al., 2026]. MemSkill learns which memory-processing skills to apply and revises them from difficult cases [Zhang et al., 2026a]. Outside memory, Selective Context removes less informative input before inference [Li et al., 2023], and TALE conditions reasoning on a token budget [Han et al., 2025].

These systems measure cost in tokens, calls, latency or price, and separate execution cost from the offline work used to obtain a policy. The reviewed evaluations do not convert their decisions into measured phone energy, so their savings cannot be read as energy saved on a particular handset.

2.5 On-device memory and physical resource costs

2.5.1 The closest mobile studies

On-device execution brings memory design into contact with shared RAM, battery capacity, processor scheduling and heat dissipation. Lu et al. [2025] evaluate compact models on edge hardware and smartphones and observe differences in prompt processing, decoding and thermal behavior. Their study supports treating runtime and hardware as part of the experimental condition. The on-device survey by Xu et al. [2024] also describes hardware acceleration and model compression as complementary approaches to deployment.

Zhang et al. [2026c] compare raw buffers, tiered memory, vector retrieval and structured graphs on short- and long-term mobile-agent applications. Their findings show that the preferred architecture depends on the application and model capacity, with structured memory remaining competitive in their tested settings. This is a close conceptual comparison because it varies memory architecture with compact models. Its latency measurements use an M2 Pro laptop, and its graph construction uses deterministic rules. These choices differ from phone execution with an SLM extractor. Its MemGUI-Bench and FingerTip tasks also differ from the conversational-QA benchmarks used here. The authors list real-phone latency and device-level cost measurement as future work; Chapter 5 reports both for conversational memory.

MemLoRA targets the memory operations themselves. It distills specialized adapters for extraction, updating and response generation into compact models and evaluates their use in a conversational memory system [Bini et al., 2025]. This provides an alternative to relying on an unadapted small extractor: memory-specific training can change the quality of the intermediate representation. The paper's throughput measurements, however, do not provide a physical smartphone energy comparison. It addresses memory capability and computational efficiency under its reported execution setup.

Two further systems treat memory as part of a phone-native agent runtime. MOBIMEM reports CPU-only experiments on a Snapdragon 8 Elite platform and finds that action reuse can reduce latency by factors of 1.6× to 9×, without reporting handset energy [Liu et al., 2025]. ClawMobile treats token budgets, inference placement and persistent memory as smartphone-runtime concerns, but its preliminary agent uses remote model inference [Du et al., 2026].

MobileRAG provides direct device-energy evidence for document retrieval. It combines EcoVector, a vector search design that partially loads index data, with Selective Content Reduction to reduce the text passed to the reader. Its evaluation on a Galaxy S24 includes energy per query for document-QA workloads [Park et al., 2025]. Its focus is document indexing and query processing rather than representations of long-term conversation history.

SmartRAG is the closest reviewed on-device graph system. It combines a distilled, continually adaptable entity recognizer and relation extraction with a provenance-preserving graph,

hybrid retrieval and a compact answer model. It runs on two OnePlus handsets with 1.7B and 4B backbones and reports memory use and average power during extraction and QA [Jiang et al., 2026]. These measurements show that a graph-based retrieval system can be deployed on a phone when its components are designed for that setting.

Two boundaries distinguish its evidence from the comparison needed here. SmartRAG's QA benchmarks use document corpora, not long-term dialogue histories. In addition, Appendix C states that its competing retrieval systems run in a cloud-side simulation; they are not profiled as complete real-phone deployments. Its average-power measurements therefore characterize the proposed system rather than establish a matched per-answer energy comparison across memory strategies [Jiang et al., 2026].

2.5.2 Tokens, time, power and energy

Cost measures answer different questions. Token counts describe the volume of model input and output. Latency describes the elapsed time for an operation. Power, measured in watts, is the rate of energy use; energy, measured in joules, accumulates over the operation's duration. For a measurement interval of length T,

$$E = \int_0^T P(t)\,\mathrm{d}t. \tag{2.2}$$

An average-power value can be converted to energy only with the duration for the same measurement interval. A lower power draw can coexist with higher task energy when an operation takes sufficiently longer.

The difference between MobileRAG's per-query energy and SmartRAG's average power illustrates why units and boundaries must remain explicit [Park et al., 2025, Jiang et al., 2026]. Similarly, tokens generated by an extractor and tokens read by an answer model cannot be assigned a universal energy coefficient. Prefill and decoding have different bottlenecks, and model architecture and hardware affect their execution [Lu et al., 2025]. Token counts can be informative predictors within a measured configuration, but the relationship must be established for that configuration.

Memory construction also changes the time horizon of an efficiency claim. For a fixed store used by N queries, one accounting model is

$$\overline{E}(N) = \frac{E_{\text{build}} + E_{\text{maint}}}{N} + \frac{1}{N}\sum_{j=1}^{N} E_{\text{query},j}, \tag{2.3}$$

where E_{build} is construction energy, E_{maint} is maintenance energy over the same period, and each query term includes the operations inside the chosen service boundary. This is an accounting identity for the stated workload, not a claim that costs stay constant as a store grows. Query-time construction, as in LazyMem or DeferMem, belongs in the query term [Yu et al., 2026b, Yin et al., 2026].

Accuracy and energy per query describe answer quality and device cost separately; a combined measure asks how much correct output the energy buys. Saad-Falcon et al. [2025] compare local inference configurations by accuracy per watt and accuracy per joule. Because power omits duration (Equation 2.2), only the per-joule form separates a strategy that answers in one minute from one that needs several minutes at a similar draw. Correct answers per kilojoule, used in this thesis, is the per-workload form of accuracy per joule. It describes a device workload only when its correctness labels come from the answers that workload produced, and it complements rather than replaces accuracy, energy, latency, RAM and thermal measurements. Chapter 3 gives its operational definition.

2.6 Evaluating conversational memory

2.6.1 LoCoMo and LongMemEval

LoCoMo evaluates memory over long, multi-session conversations. The published benchmark includes question answering, event summarization and multimodal dialogue generation, with conversations averaging approximately 600 turns and 16,000 tokens and spanning up to 32 sessions [Maharana et al., 2024]. Its QA component includes questions requiring individual facts, evidence across turns, temporal interpretation and open-domain knowledge, together with an adversarial category. This thesis evaluates the QA component using the specified first-four-category convention. Its results do not assess the benchmark's summarization or multimodal-generation tasks.

LongMemEval tests information extraction, multi-session reasoning, temporal reasoning, knowledge updates and abstention through 500 curated questions embedded in interaction histories [Wu et al., 2025]. These abilities separate recalling an isolated fact from combining sessions, identifying the relevant point in time or recognizing that the history does not support an answer. Its inclusion provides a second conversational-memory setting for the thesis. The actual subset and evaluation configuration are defined in Chapter 3.

The benchmarks are related but are not interchangeable. In particular, abstention is an explicit LongMemEval ability, while the thesis's primary LoCoMo analysis excludes its adversarial category under the stated convention. Coverage of one question type cannot be inferred from success on another. Both benchmarks also assess memory through specified tasks; performance on their QA questions is evidence about memory-dependent answering, not a direct measure of tool use or complete mobile-agent workflows.

2.6.2 Automatic judging and comparison boundaries

Short-answer metrics based on lexical overlap can miss semantically correct paraphrases or respond to differences in answer wording. An LLM judge can instead assess a candidate answer against the question and reference answer using a fixed rubric. This makes the judge part of the measurement procedure. MT-Bench and Chatbot Arena study model-based judging and document biases, including sensitivity to answer position and verbosity, alongside agreement with human preferences in their setting [Zheng et al., 2023]. Agreement reported for those tasks cannot calibrate a different judge on conversational-memory QA.

Repeatability and validity must consequently be evaluated separately. Repeated agreement by one judge indicates stability under the tested procedure. Human review addresses whether its labels are appropriate for the task. A reference-guided, single-answer protocol also differs from pairwise preference judging, so biases established for one format should not be assumed to transfer unchanged. LongMemEval already uses the reference-guided format: it scores answers with an expert-written correctness prompt and reports more than 97% agreement between its GPT-4o judge and human experts [Wu et al., 2025]. That figure validates the prompt with that judge model; it does not transfer automatically to a different model applying the same rubric, which is the arrangement used in this thesis. The methodology chapter specifies the judging procedure and the limits of the available validation.

The same care is needed when comparing published systems. Two studies using LoCoMo may use different conversations, exclude different question categories, train on part of the corpus, allow different answer lengths, or use different readers and judges. DeferMem, for example, separately reports LoCoMo evaluation excluding the conversation used to train its distiller [Yin et al., 2026]. That result belongs to a different evaluation set from a study that retains all conversations. The benchmark name alone is insufficient to establish a matched comparison.

For this reason, the related work is compared by method, task and measurement boundary. Published accuracy values establish what a system achieved under its own protocol. The thesis's controlled comparisons use paired questions and a common scoring procedure within the experimental conditions described in Chapter 3.

2.7 Synthesis and position of the thesis

The reviewed work establishes several viable approaches to long-term memory. Dense retrieval preserves access to original text, while compression and graph construction change how information is organized and exposed. Recent systems improve the use of retrieved evidence, defer construction until the query is known, or restore source text through provenance. Their results make memory design a question of representation, processing and evaluation conditions rather than a fixed ordering of architectures.

Table 2.2 summarizes the closest connections. It records the type of evidence supplied by each group of studies, keeping system feasibility, proxy efficiency and physical energy measurements distinct.

Related work	Relevant contribution	Relation to this thesis
MobileRAG [Park et al., 2025]	Smartphone document RAG with per-query energy	Direct energy precedent; different stored content and comparison scope.
SmartRAG [Jiang et al., 2026]	On-phone graph retrieval with power and memory profiles	Graph feasibility precedent; competing systems lack matched phone profiles.
Mobile memory benchmark [Zhang et al., 2026c]	Architecture comparison for compact mobile agents	Different tasks; laptop latency measurements and rule-based graph construction.
MemLoRA [Bini et al., 2025]	Distilled adapters for small-model memory operations	Alternative construction design; no matched handset-energy study.
RippleMem; Chain-of-Memory [Ji et al., 2026, Xu et al., 2026]	Associative retrieval and deliberate evidence assembly	Richer memory units performing well; limits how far a triple-pipeline result generalizes.
LazyMem; DeferMem [Yu et al., 2026b, Yin et al., 2026]	Query-conditioned construction after broad retrieval	Alternative distribution of preparation and query costs.
BudgetMem; MemFlow [Zhang et al., 2026b, Chen et al., 2026]	Query-aware choice of memory processing and budgets	Established routing approaches; phone energy remains a separate measure.
LeanMem; SEEM [Liao et al., 2026, Lu et al., 2026]	Compact memory connected to original source text	Direct related work for the budgeted provenance-selection study.

Table 2.2: Closest related work and its measurement boundaries. Entries summarize the versions reviewed for this chapter and do not form a cross-paper leaderboard.

Read across its rows, the table separates three kinds of evidence: feasibility on a phone, efficiency measured by proxies such as tokens, latency or price, and physical energy. Among the studies reviewed here, per-query phone energy is reported for document retrieval [Park et al., 2025], and average phone power for one graph system whose baselines ran off the phone [Jiang et al., 2026]. The closest comparison of memory architectures for compact mobile agents reports

latency on a laptop and leaves real-phone cost measurement to future work [Zhang et al., 2026c]. These studies do not compare memory representations for conversational histories by answer quality and energy measured on the same handset. This thesis makes that comparison for the tested memory choices and analyses where its graph pipeline loses evidence; the following chapter defines the experimental controls needed to interpret it.

Chapter 3

Methodology

This chapter describes how the experiments were run. One shared pipeline evaluates small language models across memory strategies and model sizes on a smartphone and on a workstation. The evaluation harness, question sets, configuration baselines and quantization tier are the same in both settings; the two platforms differ in their hardware, runtime software and weight packaging.

Contribution of this chapter. The individual components are established: dense turn retrieval [Karpukhin et al., 2020], a reference-guided automatic judge using the LongMemEval rubric [Wu et al., 2025], Coulomb counting of battery charge, and accuracy per joule as an efficiency measure [Saad-Falcon et al., 2025]. The contribution is the protocol that combines them for conversational memory on a phone. Two of its elements distinguish it from the on-device studies reviewed in Section 2.5. First, every compared memory strategy runs on the phone, and its accuracy and energy come from the same phone workload: the answers it produced there are the ones judged, and the energy is measured over the span that produced them. In the reviewed studies, either the competing systems did not run on the phone [Jiang et al., 2026] or the workload was document retrieval rather than a conversation history [Park et al., 2025]. Second, a rule fixed before the comparison decides whether workstation accuracy may stand in for phone accuracy. A third element serves the graph analysis of Chapter 6: the graph pipeline is changed one stage at a time (Table 3.2), so that losses in retrieval, extraction and representation can be told apart.

3.1 Research design: one pipeline, two measurement settings

The study compares three ways of giving a small language model access to a long conversation, at several model sizes, on two long-context benchmarks. Running every condition on the phone is not practical: the full grid spans hundreds of paired questions across five readers, two benchmarks and a sweep of retrieval depths, while one 20-question full-context sitting on the phone took up to about four hours. Energy, battery drain, thermal throttling and memory pressure, however, exist only on the real device. The two settings therefore have complementary jobs:

- The phone (a Samsung Galaxy S23 Ultra, running models locally through the llama.cpp runtime [Gerganov and ggml-org contributors, 2026]) is the main measurement platform. It measures whole-device energy, battery charge, latency, memory use and heat, and it records the answers it generates, which the same judge then scores under the same rubric. Phone accuracy is therefore measured on the phone, not assumed from the workstation.

- The workstation (referred to as "the box") runs the same memory modules through the Ollama runtime [Ollama, 2026] on more questions: up to 200 LoCoMo and 60 LongMemEval-S questions per condition, five readers, and the graph and retrieval-depth variants. It also hosts the 70B judge. Its larger samples allow paired statistical tests of the main strategy comparisons.

Phone accuracy and energy per query are reported separately, and correct answers per unit of energy combines the answers and energy of the same phone workload. Because the runtimes and weight packaging differ, platform equivalence was tested rather than assumed: a predefined parity rule (Section 3.8) decided whether workstation accuracy could substitute for phone accuracy, and every on-device claim uses phone-measured accuracy. The one exploratory projection that combines the two platforms is labelled as such in Appendix A.6.

The comparison is *paired* throughout: for a given benchmark and model, all memory strategies see the same questions, dialogue histories and reference answers, which enables paired tests and guards against data-handling mistakes. Figure 3.1 gives an overview of the pipeline.

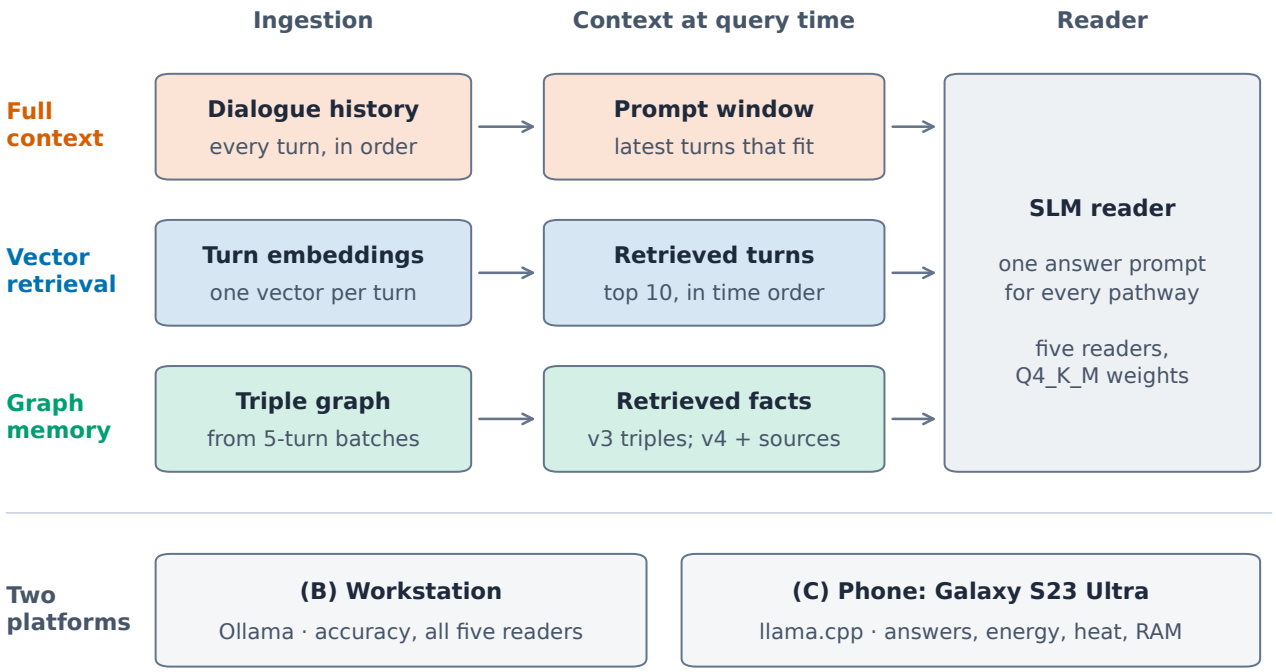


Figure 3.1: Shared memory and evaluation pipeline. Upper rows: truncated full context, vector retrieval and SLM-extracted graph memory construct evidence for the same reader and answer prompt; GraphRAG v4 adds source turns to v3 triples. (B) The workstation evaluates all five readers through Ollama. (C) The Galaxy S23 Ultra runs selected configurations through llama.cpp and records device resources. A workstation-hosted 70B judge scores both platforms' answers; their accuracies remain separate.

3.2 Language models and runtime environment

The model that reads the supplied conversational context and produces the answer is called the *reader*. Five readers are evaluated, drawn from two open-weight families and spanning from 0.8 to 4 billion parameters (Table 3.1). Using two families shows whether the comparison between memory strategies holds beyond one model family.

Family	Reader	Parameter scale	Attention design
Gemma 4	E2B	2.3B effective (5B total)	Hybrid local/global attention
Gemma 4	E4B	4.5B effective (8B total)	Hybrid local/global attention
Qwen3.5	0.8B	0.8B	Hybrid linear/full attention
Qwen3.5	2B	2B	Hybrid linear/full attention
Qwen3.5	4B	4B	Hybrid linear/full attention

Table 3.1: The five reader models evaluated across the memory strategies. [Qwen Team, 2026], [Gemma Team, Google DeepMind, 2026].

Model architectures. The two families differ in how they handle long inputs. The Gemma 4 readers use a hybrid attention scheme that interleaves local sliding-window attention with full global attention, with the final layer always global; the global layers apply proportional rotary position encoding, and many layers share the key–value cache of earlier layers, both of which reduce the memory cost of long contexts [Gemma Team, Google DeepMind, 2026]. The "E" in E2B and E4B stands for *effective* parameters: the smaller Gemma models use per-layer embeddings, giving each decoder layer its own small lookup table per token, so the effective parameter count is much lower than the raw total. These design choices matter here because they bear directly on the on-device memory analysis presented later. The Qwen3.5 readers are dense models (no mixture of experts) with a different hybrid design: in every block of four layers, three use Gated DeltaNet linear attention and one uses gated full attention [Qwen Team, 2026]. Gemma 4 is the primary reader family of this thesis; the Qwen3.5 readers were added as a second family to test whether the findings hold beyond Gemma.

Quantization and packaging. Running local inference on mobile hardware requires weight compression. All five readers are evaluated at one quantization tier, the standard `Q4_K_M` format, which stores weights at roughly four bits each with a fixed per-layer layout. The phone runs the Unsloth-distributed GGUF files directly through llama.cpp [Unsloth AI, 2026, Gerganov and ggml-org contributors, 2026], while the workstation serves the same `Q4_K_M` tier through Ollama [Ollama, 2026]. These are the standard static `Q4_K_M` builds, not the adaptive-width Dynamic variants. One on-device cell was repeated on an adaptive 4-bit build (`UD-Q4_K_XL`) to check whether the choice of 4-bit build changes the results. It returned the same verdict on all twenty questions as the standard build, so one build is used throughout. Higher-precision weights were not tested.

On-device hardware and runtime. On-device measurements were performed on a Samsung Galaxy S23 Ultra. Inference ran on the CPU of its Snapdragon 8 Gen 2 (one Cortex-X3 prime core at 3.36 GHz, four performance cores at 2.80 GHz and three efficiency cores at 2.00 GHz), with 12 GB of RAM and a 5,000 mAh battery; temperature was read from the phone's thermal zones. Inference ran through a local inference server inside a Termux environment, with execution pinned to the five high-performance cores, five worker threads, and FlashAttention enabled. The phone server ran with an 8,192-token context. The llama.cpp server rejects a prompt longer than its context, and full-context prompts built to the 4,096-token budget are longer than 4,096 tokens (Section 3.3), so the server was given this room to keep them from failing. The workstation was a dedicated Linux host running Ollama on GPU hardware. Ollama does not reject an over-long prompt; it trims it to its 4,096-token window.

Decoding. Every reader generates its answer by greedy decoding: taking the most likely token at each step, with temperature set to zero ($T = 0.0$). Greedy decoding removes sampling

randomness. On the phone, repeated sittings returned identical answers on every question they shared, so the variation between repeated sittings reflects physical energy and runtime differences, not differences in what the model said. Model cards typically document a non-zero sampling temperature (for example $T = 1.0$), so greedy decoding is an intentional deviation, chosen because deterministic decoding is standard for benchmarking and makes repeated runs directly comparable; the paired tests themselves rest on every condition answering the same questions. A control experiment on LoCoMo found no detectable difference between greedy decoding and the vendor sampling setting (36.0% against 35.0%, $p = 0.69$, McNemar's exact test).

Step-by-step reasoning. Some readers can write out intermediate reasoning before the final answer. This is toggled through the vendor-documented request parameter. In the baseline configuration reasoning is off, and the models return concise answers of about a dozen tokens. In the dedicated reasoning experiments reasoning is on and the generation budget is raised from 64 to 768 tokens.

3.3 Memory strategies

Three ways of giving the reader access to past dialogue turns are compared. The reader, the decoding settings, and the answer format are held constant; only the way the context is built differs. All three strategies work from the same rendered turns. When a LoCoMo turn shares a photo, the caption that the dataset provides for that photo is appended to the turn's text.

Full context (concatenation). The whole conversation is placed in the prompt in chronological order, each turn rendered with its timestamp, speaker, and text. The oldest turns are dropped until the history fits a 4,096-token budget, less 192 tokens reserved for the template and the answer. The harness counts tokens as characters divided by four, which undercounts LoCoMo dialogue, so the resulting prompts measured 4,670 to 5,254 tokens. The phone server held them in full. Ollama on the workstation trimmed the oldest part of each prompt to exactly 4,096 tokens, so workstation full context read somewhat less history than phone full context. On the twenty questions that both platforms answered, the wording of the answers differed, but the judged verdicts agreed on 18 or 19 of 20 for every reader. An exploratory workstation rerun for three readers raised the window to 131,072 tokens (262,144 for one cell), so that the history fit whole in every prompt except four LongMemEval-S prompts per Gemma reader. Those four lost their oldest 2 to 4 turns (Section 4.6).

Vector retrieval (dense turn retrieval). Each turn is embedded once at ingestion time. At query time the question is embedded, its cosine similarity to every stored turn is computed, and the most similar turns (by default the top ten) are retrieved, sorted back into chronological order, and placed in the prompt.

Graph memory (SLM-extracted graph). The reader model itself acts as the extractor while the conversation is ingested. Reading the dialogue in batches of five turns, it writes out factual relationships as (subject, relation, object) triples in structured form. The facts are stored in a directed multigraph, from which the facts relevant to a question are retrieved at query time. Because the graph is built by the small model itself rather than by an external rule-based parser or a larger model, it is called an *SLM-extracted graph* throughout.

Figure 3.2 sets the three lookups side by side: what each strategy stores when the conversation is ingested, what it does when a question arrives, and what text the reader finally receives.

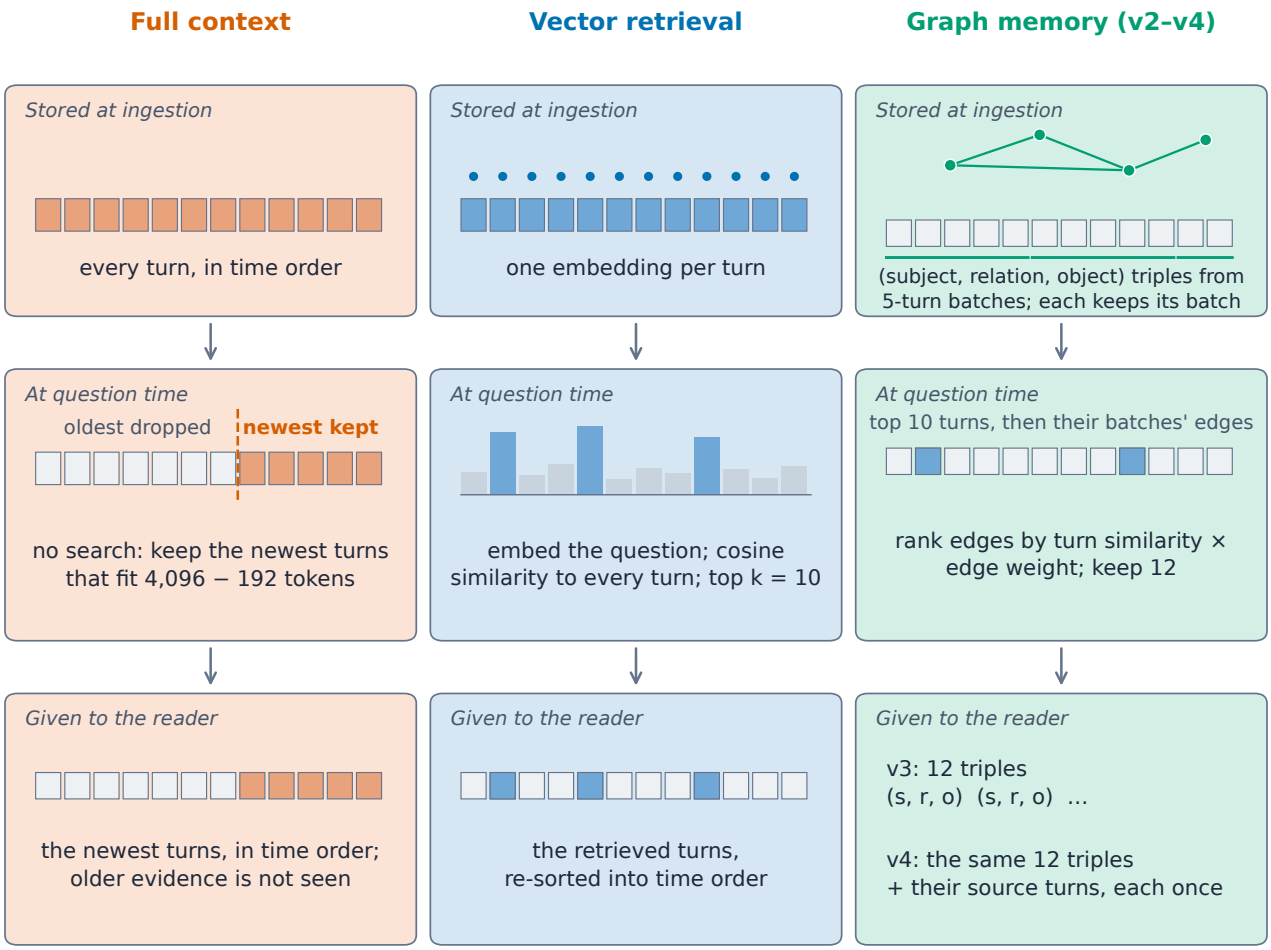


Figure 3.2: How each memory strategy looks up evidence (schematic; the turns are illustrative). Full context performs no search: it keeps the newest turns that fit the 4,096-token budget, less 192 tokens, and drops the oldest. Vector retrieval scores every stored turn embedding against the question embedding and passes the ten most similar turns to the reader in time order. Graph memory (GraphRAG v2–v4) uses the same top ten turns to find edges extracted from their five-turn batches, ranks those edges by the similarity of their best turn multiplied by the edge weight, and keeps twelve; v3 shows them as triples and v4 adds each source turn once. GraphRAG v1 instead matched question terms to node names and took the one-hop neighbourhood (Table 3.2).

3.3.1 Vector retrieval

Vector retrieval is a standard retrieval-augmented generation arrangement [Lewis et al., 2020] built on a dual-encoder dense retriever [Karpukhin et al., 2020]. There is no reranking stage, no query rewriting or expansion, and no summarization or compression of the retrieved text. The design choices made here are the unit of retrieval and the decision to add nothing on top of the retriever.

Unit and ingestion. The retrieval unit is a single dialogue turn, rendered with its timestamp, speaker, and text exactly as the full-context condition renders it. Each turn is embedded once when it is ingested, and turns longer than roughly 1500 tokens are truncated before embedding. Nothing is re-embedded at query time and nothing is embedded twice, so one ingestion pass serves every question asked of that conversation and every retrieval depth swept over it.

Query time. The question is embedded with the query-side prefix, cosine similarity to every stored turn is computed as a dot product over unit-normalized vectors, and the k highest scoring turns are taken. Those turns are then sorted back into chronological order before they enter the reader prompt, so the reader sees dialogue in the order it happened, not in similarity rank. The default depth is $k = 10$; Chapter 4 sweeps it from 2 to 50.

Embedder. Nomic Embed Text v1.5 [Nussbaum et al., 2025] is used off the shelf, with no fine-tuning on either benchmark and no training or tuning for this study. It is a general-purpose embedder trained on large collections of text pairs, and general-purpose dense retrievers are known to work on new collections without task-specific training [Izacard et al., 2022]. The retrieval arm therefore has no advantage from tuning to these benchmarks. The task prefixes this model expects, and the one respect in which the workstation and on-device runs differ in applying them, are described in Section 3.5.

What is stored. The ingestion artifact is a set of per-turn embeddings that depends only on the embedding model and on whether the task prefixes were applied, not on the reader or the retrieval depth; depth is applied at query time by slicing the ranked list. Every reader compared at a given depth therefore sees byte-identical retrieved turns, and every depth in a sweep reads the same stored embeddings, so differences across readers or depths reflect what was shown, not how it was indexed.

3.3.2 Graph construction and retrieval

The base SLM-extracted graph works as follows.

Extraction and normalization. Turns are ingested in non-overlapping, consecutive batches of five. The extractor is prompted to identify factual relationships, to preserve dates, times, and locations, to resolve pronouns to explicit speaker names where the reference is clear, and to return a list of triples. Extraction is limited to 256 tokens of output in the baseline (raised to 512 or 1024 in specific repair runs). The extracted text is stripped of formatting markers and parsed; entity and relation strings are then normalized deterministically by lower-casing, removing trailing punctuation and possessives, and collapsing familial aliases to a canonical form (for example, *mom* to *mother*). A short stop-list of bare pronouns filters out ungrounded nodes.

Graph structure and reinforcement. Facts are stored as a directed multigraph. A new triple adds a weighted edge between its two entities, carrying the dialogue timestamp and the source turn range; a repeat of the same relation between the same pair of entities reinforces the existing edge's weight up to a ceiling and updates its timestamp and sources. The implementation supports edge decay and pruning, but decay is disabled in every reported run (the decay rate is fixed so weights never shrink), so the evaluated graph is strictly append-only with reinforcement. No claim about forgetting is made.

The four evaluated versions, GraphRAG v1 to v4, form a controlled improvement ladder (Table 3.2). Every version stores the same kind of triple edges in a directed multigraph of

Version	Retrieval mechanism	Extraction budget	Graph contents	Context given to reader
v1	Fuzzy entity matching and one-hop ego-subgraph	256 tokens	Original build	Retrieved triples
v2	Dense turn search followed by associated-triple ranking	256 tokens	Rebuilt with source links and standardized names	Retrieved triples
v3	Same vector-first retrieval as v2	512 tokens	Rebuilt; more triples survive extraction	Retrieved triples
v4	Same retrieval as v3	512 tokens	Same as v3	Triples and verbatim source turns

Table 3.2: Configuration of GraphRAG v1 to v4. The triple schema is the same in every version; the columns show which stage each version changes.

the same form. Each version changes one part of the preceding configuration, which separates a retrieval problem from an extraction-budget problem or a loss of evidence during graph compression; the v1-to-v2 step also adds source links and name standardization (Chapter 6).

The v1-to-v2 step asks whether entity matching retrieves the wrong part of the conversation, v2-to-v3 whether structured output cut off at 256 tokens loses facts, and v3-to-v4 whether triples preserve enough evidence or the original wording is still needed. Version 4 reuses the v3 graph and adds the supporting source turns when the answer prompt is assembled.

3.4 Benchmarks and task configuration

Two conversational long-context benchmarks are used.

LoCoMo [Maharana et al., 2024] contains long, multi-session dialogues between recurring speakers, with paired questions. The benchmark defines several tasks; this study evaluates only the *question-answering* task, and every result statement refers to that task. Evaluation uses a fixed set of 200 questions, 50 from each of Categories 1 through 4, taken in turn from each of the ten conversations (round-robin) so that every conversation contributes. The benchmark paper reports a fifth, adversarial category as well [Maharana et al., 2024]; later work that evaluates on LoCoMo commonly reports only the first four, and this thesis follows that convention. Zhang et al. [2026b], for example, state the same exclusion when reporting per-category LoCoMo results. The exclusion keeps the results comparable with that work and is not a judgement about the omitted category.

LongMemEval-S [Wu et al., 2025] provides much longer histories, averaging about 500 dialogue turns, with each question labelled by one of six reasoning types. Because the questions are stored grouped by type, a simple prefix of the file would sample only one type; to avoid that, a stratified subset of 60 questions is used, ten from each of the six types, drawn with a fixed random seed so the subset is reproducible.

Roles of the two benchmarks. LoCoMo is the primary benchmark: it carries the full experimental grid, the graph-repair and selective-expansion studies, the answer-prompt substitution and all on-device measurements. LongMemEval-S tests whether the main strategy

ordering holds at 500-turn scale; it carries the core memory-strategy comparison, a retrieval-depth sweep and a reasoning probe. Where a result exists for only one benchmark, the text says so.

Balanced sampling and population weighting. Both benchmarks are sampled with equal questions per category, which gives every category the same number of questions for fine-grained error analysis, so the plain sample mean is an unweighted category average. Because the real benchmark populations are not evenly split across categories (for example, Category 4 is about 55% of the four evaluated LoCoMo categories), a population-weighted accuracy is also computed as a secondary sensitivity check. Table 3.3 summarizes both evaluated datasets, and Figure 3.3 shows the structure of one LoCoMo history and where the evidence for one question lies.

	LoCoMo	LongMemEval-S
Histories	10 conversations	60 histories, one per question
Turns per history	369–689 (mean 588)	396–561 (mean 491)
Sessions per history	19–32	39–61 (mean 48)
Words per history	8.0k–16.2k (mean 13.4k)	74k–82k (mean 79k)
Questions evaluated	200, 50 from each of Categories 1–4	60, 10 from each of six types
Phone runs	a 20-question slice	not run on the phone
Role	primary benchmark	tests whether the ordering transfers

Table 3.3: The two evaluated datasets. Sizes are computed from the evaluated data files; token counts depend on the reader's tokenizer and are not shown.

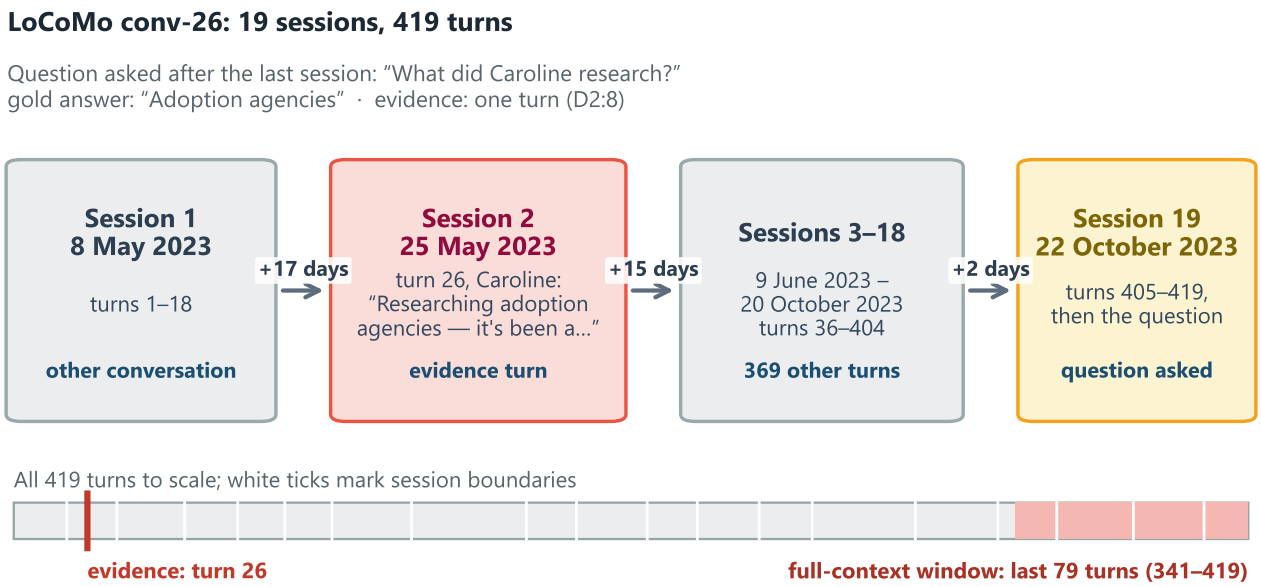


Figure 3.3: One LoCoMo history, `conv-26`: 419 turns across 19 sessions. The labelled evidence for "What did Caroline research?" lies in session 2, outside the 79 newest turns that full context retains. Those 79 turns occupy 4,762 tokens on the phone; the workstation trims the prompt further to 4,096. The strip below the sessions draws all 419 turns to scale.

3.4.1 Fixed configuration and the one-variable rule

Apart from the variable under study, every run uses one fixed configuration. Most comparisons change a single element of this baseline. Three change more than one and are reported as combined changes: GraphRAG v2 (retrieval, source links and name standardization together), the reasoning runs (reasoning mode together with a longer answer limit), and the graph model-size comparison (extractor and reader together).

- **History budget.** 4,096 tokens across all models and conditions, which sets where full context truncates the history; the only exception is the long-window full-context rerun of Section 4.6. Section 3.3 describes how each platform applies it.
- **Decoding.** Greedy ($T = 0.0$), reasoning off.
- **Answer length.** A generation budget of 64 tokens. In practice this ceiling is almost never reached: the reader is asked for a short, direct answer and returns about ten to fifteen tokens. The reasoning runs raise the budget to 768 tokens, because the reasoning has to be written out; that change is confined to those runs and is treated as a controlled variable, not part of the fixed baseline.
- **Reader prompt.** One curated short-answer prompt is given to the reader across all models, strategies, and benchmarks.
- **Retrieval.** Ten retrieved turns by default, ranked by cosine similarity using Nomic Embed Text v1.5 with its task prefixes (Section 3.5). Turns are truncated to about 1500 tokens before embedding.
- **Graph construction.** Extractor batches of five turns, a 256-token extraction budget, up to twelve retrieved facts per question, decay disabled, and the extractor model set to the same model that answers.
- **Scoring.** The same 70B judge and the same judge prompt for every answer, on both benchmarks and both devices (Section 3.6).

The varied elements are the memory strategy, the model size, the number of retrieved turns, whether reasoning is on, the graph's extraction budget and the answer prompt, with the three combined changes named above. Everything else stays as above.

3.5 Retrieval configuration

Dense retrieval and vector-first graph retrieval use Nomic Embed Text v1.5 [Nussbaum et al., 2025, Nomic AI], which expects asymmetric task prefixes: `search_document:` for stored turns and triple strings, and `search_query:` for questions. The prefixes were added partway through the study, and the affected workstation conditions were rerun (Appendix B).

Reporting convention. A completed prefixed rerun replaces the earlier unprefixed result for the same condition, whether its score is higher or lower; historical values appear only where needed to explain development or fixed evaluation criteria, and are identified as such. Three configurations result:

- all primary workstation comparisons use prefixed retrieval;
- the primary 20-question phone comparison keeps its original unprefixed runs, because the replicated campaign was too costly to repeat, and footnotes mark this exception;

- the E2B and E4B 100-question phone runs and the E2B two-turn phone probe use prefixes.

Full context and GraphRAG v1 perform no embedding, so the setting does not apply to them.

Comparability. The prefixed 100-question phone runs contain all 20 questions of the primary phone comparison. Apart from the prefixes, they use the same reader, retrieval depth, history budget, prompt, decoding and judge. On those 20 questions, the prefixes changed Gemma 4 E2B from 10 to 8 correct answers and E4B from 8 to 10. Each reader had two discordant questions, in opposite directions (exact McNemar $p = 0.5$ for each). At this sample size, the phone shows no detectable effect of the prefixes on accuracy. The check covers accuracy only: the 100-question sittings spread their memory build over five times as many questions, so their energy per answer is not comparable. Workstation prefix gains are not applied to phone scores, and every phone efficiency result uses answers and energy from the same measured workload.

3.6 Answer scoring and judge characterization

Answer correctness is decided by an automatic judge: Llama 3.1 70B Instruct at `Q4_K_M` precision [Grattafiori et al., 2024], run on the workstation. The judge is prompted with the official LongMemEval correctness rubric, which supplies the question, the reference answer, and the model's prediction, and asks whether the response contains or is equivalent to the reference answer. This correctness prompt is distinct from the answer prompt given to the reader. LoCoMo answers use its standard form; LongMemEval answers use the benchmark's own variants for temporal, knowledge-update, preference and abstention questions (Appendix B.1). Answers produced on the phone are transferred to the workstation and scored by the same judge under the same rubric. Every accuracy figure in the thesis, on either device, comes from this judge and is stamped with the judge's identity so that its provenance is traceable.

Judge characterization and reliability. The judge is treated as an instrument with measurable noise, not an infallible oracle. Its consistency was checked by re-judging identical prediction sets at $T = 0$. Three back-to-back re-judgings of 500 predictions in one server session changed no verdict. When 100 predictions were judged again in a later session, one verdict changed; GPU-level non-determinism is a plausible cause, and a single comparison of this size does not give a tight bound. Lexical measures (exact match, token overlap, substring containment) are computed as sanity checks, to confirm that the relative ranking of the strategies is not an artifact of judge leniency.

Human validation. Because an automatic judge cannot validate itself, a blind human audit checked its verdicts on 100 judged answers. The answers were drawn from eight runs covering both benchmarks, all three memory strategies and both Gemma readers, about half of them judged correct and half judged incorrect; some come from runs made before the embedding-prefix correction, which does not affect what the judge is asked to decide. Every condition identifier was removed and the order was shuffled. Because the runs answer the same questions, the 100 answers cover 86 distinct questions. The author labelled each answer as correct, incorrect or ambiguous against the reference answer, using the judge's rubric, without seeing the judge's verdicts.

On the 83 answers given a decisive label, the judge agreed with the human label on 82 (98.8%, Cohen's $\kappa = 0.976$); keeping one answer per question gives 73 of 74. The single disagreement was a false positive: the reference was "fitness tracker" and the judge accepted the hedged answer "fitness tracker or smartwatch". The judge marked no correct answer as wrong. The other 17 answers were labelled ambiguous. Thirteen were LoCoMo answers that matched

the reference only partly or loosely, such as a specific date where the reference names a week; four were LongMemEval answers, three of them to abstention questions. Ambiguous answers were not assigned a label and are left out of the agreement denominator, which is therefore 83. The judge had accepted 14 of the 17. Counting every ambiguous answer as a disagreement instead would lower agreement to 82 of 100. The audit has one annotator, so it does not measure agreement between humans, and no second judge model was compared. Because the sample is balanced between judged-correct and judged-incorrect answers, it measures how often the judge's verdicts are right, not how far any particular accuracy figure would move.

Missing data and error handling. Model generation failures and judge infrastructure errors are kept distinct. Predictions that are genuinely empty, contain only unclosed formatting tags, or exhaust the generation budget before producing an answer are kept in the denominator and scored as incorrect. Transient connection failures or timeouts during judging are retried with exponential backoff; verdicts that still cannot be parsed are isolated and logged rather than counted as wrong.

3.7 On-device resource measurement

Physical resource measurements are made on the Samsung Galaxy S23 Ultra.

Measurement environment. Runs are performed with the device unplugged, held under a continuous wake-lock so the processor does not sleep, and with the display off.[1]

Energy method of record. Battery-side energy is estimated from the phone's charge counter and measured voltage. The counter's charge decrease agrees with the reported battery-level decrease to within about one percentage point. A separate cross-check using integrated current and voltage agrees with counter-based energy to within 10% on sittings longer than 50 minutes (Appendix A.7). Resource telemetry is sampled every 0.5 seconds by a background profiler logging directly to file. The physical quantity directly measured on the handset is the battery charge consumed, ΔQ (in mAh), recorded by the battery's hardware Coulomb counter:

$$\Delta Q = \frac{Q(t_0) - Q(t_N)}{1000}. \tag{3.1}$$

Here, $Q(t)$ is the charge counter in μAh and $t_0, \ldots, t_N$ are the usable unplugged samples inside the measured run span. Whole-device battery energy is then calculated by pairing each charge interval with the mean battery voltage measured over that interval:

$$E = 3.6 \times 10^{-3} \sum_{i=0}^{N-1} [Q(t_i) - Q(t_{i+1})] \, \frac{V(t_i) + V(t_{i+1})}{2}, \tag{3.2}$$

where $V(t)$ is measured in volts and E is obtained in joules. The factor 3.6×10^{-3} converts μAh to coulombs. This voltage-weighted Coulomb-counting calculation uses the phone's recorded voltage, not a fixed nominal-voltage assumption. The study reports both mAh and J: the former is the direct charge measurement, while the latter is the corresponding battery-side energy estimate.

[1]The 22 sittings behind the main phone tables started at 86 to 100% charge and a battery temperature of 31.7 to 36.8°C, and each reached 38.6 to 43.3°C within ten minutes. Eight of them, and three further runs, started at the highest possible charge (100%), where the phone's charge counter can pause for the first few minutes, so those runs under-count their energy (Appendix A.7).

On-device efficiency metric. Accuracy a measures the fraction of answers judged correct, while energy per query, E/N, measures the cost of producing an answer, whether correct or wrong. Both are reported separately. Their combined measure asks how many correct answers the workload produces per unit of energy. For a run containing N scored answers with phone-measured accuracy a, correct answers per unit of energy are

$$G_E = \frac{aN}{E/1000}, \tag{3.3}$$

reported per kilojoule. The denominator includes the energy consumed over the recorded workload, including queries answered incorrectly. Answer correctness does not validate the energy instrument; the telemetry cross-checks and instrument limitations are assessed separately. When a full-charge equivalent is reported, it is calculated directly from charge use as $G_Q = aN \times 5000/\Delta Q$, where 5000 mAh is the rated battery capacity. Thus the energy-normalized metric uses the measured voltage trace, whereas the full-charge metric does not require an assumed battery voltage.

Instrument limits. Energy uses the hardware Coulomb counter, which integrates charge continuously at the battery controller, rather than polled instantaneous current, which drifts under rapid load changes; the current reading serves only as a cross-check. The counter moves in 0.1% steps and holds its value at full charge, which affected nine of the eleven sittings that started at 100% (Appendix A.7).

Physical telemetry and resource metrics. Beyond battery charge, the background profiler requests a sample every 0.5 seconds. Each battery query through Termux:API takes a second or more, so the realized interval was 1.2 to 3.2 seconds (median per run). Each sample records:

- **Battery charge and energy.** Directly measured charge drop (ΔQ in mAh) and derived whole-device energy estimate (E in Joules) consumed across the entire smartphone (SoC, memory, display-off circuitry, background services) from initial ingestion to the final emitted token.
- **CPU clock frequencies.** Active clock frequencies (in MHz) sampled across all eight CPU cores (the Cortex-X3 prime core and the Cortex-A7x and A510 clusters) to monitor frequency scaling. A frequency trace shows when cores slowed, but not whether the thermal governor caused it.
- **Temperature.** The hottest reading across all thermal zones that can be read without root (per-core CPU, GPU and other sensors), in degrees Celsius (°C). It is a device-wide maximum, not one SoC sensor, and it is not the battery temperature, which is logged separately. Sampling starts with the harness, after the model servers have loaded, so the first sample is not an idle baseline.
- **Memory footprint (RAM).** System used and available physical memory (in MB) sampled via `/proc/meminfo` to evaluate memory pressure and KV-cache overhead during long-context processing.
- **Latency.** End-to-end wall-clock time per question, with the prompt and answer token counts reported by the server. The phone records only the total generation time, so prefill and decode time are not separated.

Replicates and reporting conventions. Vector retrieval on all five readers and full context on Gemma E2B were each run in three independent sittings, to measure how much the energy varies between runs. Full context on Gemma E4B and on the three Qwen readers used one sitting each. Because a fixed-question run spreads its one-time memory-construction cost over only twenty questions, per-answer energy includes building the memory, and the ratios between strategies are specific to this workload. Graph-construction energy on the phone comes from one directly measured 419-turn build (21.9 kJ, or 52.3 J/turn); any value at other conversation lengths is a projection from a workstation-measured slope and is labelled as such.

3.8 Controls, validity, and statistical plan

The statistical analysis has three specific purposes. It compares memory strategies on the same questions, checks whether the grouping of LoCoMo questions by conversation affects those comparisons, and tests whether workstation accuracy can be used in place of phone accuracy. The three procedures below are kept separate because they answer different questions.

Paired strategy comparisons. For a fixed benchmark and reader, every memory strategy answers the same questions. Each answer is reduced to a binary outcome (correct or incorrect), and the strategies are compared with McNemar's exact test [McNemar, 1947]. The test uses only the discordant pairs: questions answered correctly by one strategy and incorrectly by the other. Results report the accuracy of each strategy, the difference in percentage points, the two discordant counts, and the exact p-value. Statistical claims are limited to the main comparisons between full context, vector retrieval, and the graph versions. Retrieval-depth sweeps, prompt experiments, and other ablations are treated as descriptive analyses rather than separate significance claims.

Conversation-level sensitivity. The 200 LoCoMo questions come from ten multi-session conversations, so questions from the same conversation may be correlated. A conversation-level block bootstrap is used as a sensitivity analysis: complete conversations are resampled with replacement, and the paired accuracy difference is recalculated for each bootstrap sample. With only ten conversation clusters, this analysis checks the stability of the result rather than replacing the question-level McNemar test. LongMemEval-S contains 60 questions drawn from 60 separate sessions and therefore does not use this conversation-level check.

Cross-environment agreement check. The workstation and phone use different runtimes and weight packages, so workstation accuracy is not assumed to transfer to the phone. The same 100 E2B vector-retrieval questions were evaluated in both environments. The comparison reports the percentage of matching correctness labels, Cohen's κ, and an exact McNemar test for directional bias. The decision rule was fixed before the comparison: workstation accuracy could be substituted only if agreement reached at least 90% and McNemar's test gave $p > 0.05$. The outcome is reported in Section 5.3.1.

Answer-prompt robustness. The reader is given one curated short-answer prompt across all conditions and both benchmarks, so that strategy and benchmark comparisons are not confounded with prompt wording. Because a fixed prompt could still interact with the context representation, the LoCoMo grid was re-run with LoCoMo's own official question prompt, an externally defined standard, in place of the curated one, with everything else held fixed (Section 4.8). The substitution was not evaluated on LongMemEval-S, which limits claims about absolute accuracy on that benchmark.

Reporting conventions. Every result in Chapters 4 to 7 carries one of four labels. Tables 8.1 and 8.2 list the main claims with their label, sample size and test.

- *Pre-registered*: a criterion fixed before the run that tests it. Only the parity threshold above and the two SPE criteria of Chapter 7 have this status, and only these are confirmatory tests.
- *Primary*: nine paired comparisons, eight on Gemma and one on Qwen3.5 4B, reported with their exact McNemar p-value and a Holm correction across the nine. This family was designated during the analysis, after the results were available, so it is not pre-registered.
- *Exploratory*: every other paired test, reported with its unadjusted exact p-value.
- *Descriptive*: measurements reported without a test, including all phone energy and efficiency ratios.

Four readings apply throughout and are not repeated in each chapter. A difference that does not reach significance is never read as evidence that two conditions are equivalent. Workstation results carry no phone-energy measurement, and workstation accuracy never stands in for phone accuracy. Graph conclusions concern graphs extracted by the tested small models and retrieved by turn-anchored triple selection, not knowledge graphs in general. Finally, restoring source text adds both content and prompt length, so its effect does not isolate the information lost when dialogue is reduced to triples.

3.9 Software and reproducibility

The pipeline is implemented in Python and coordinates a llama.cpp inference server on the phone (under Android Termux) and Ollama on the Linux workstation. The graph is implemented with NetworkX [Hagberg et al., 2008], and embeddings use Nomic Embed Text v1.5 [Nussbaum et al., 2025].

Every result, table, and statistical test reported in the thesis is produced by scripts directly from the stored per-run logs rather than copied by hand: one script aggregates the master accuracy table, one derives the on-device energy metric and is the sole method of record for that number, and one produces the statistical report. Hand-computed intermediate figures from working sessions are not used in the text.

To keep the exact artifacts identifiable, an artifact manifest collects the runtime build versions, model repository revisions and file hashes, and the exact prompt and template texts used in the measured runs. The model revisions and hashes were reconstructed afterwards from the surviving model cache and server logs, and the build commit of the phone runtime could not be recovered (Section 5.2). The exact identifiers, command flags, and prompt texts are listed in the reproducibility appendix.

Chapter 4

Memory Strategy Evaluation

This chapter compares three ways of presenting conversational memory to a small language model: truncated full context, vector retrieval, and an SLM-extracted graph. A source-restoration ablation also tests whether graph triples lose information that remains available in the original dialogue. The analysis then varies reader scale, benchmark, retrieval depth, and step-by-step reasoning. All headline comparisons use the workstation protocol from Chapter 3. Each memory condition answers the same questions and is scored by the same 70B judge.

With the history cut to a 4,096-token window, vector retrieval has the highest accuracy point estimate in every LoCoMo model cell, and the same ordering appears on the category-balanced LongMemEval-S subset. Long-window full context, which holds the whole LoCoMo history, matches vector retrieval on LoCoMo and trails it on the much longer LongMemEval-S histories, at about 39 and 48 times the prompt tokens. Reader scale improves several conditions, while retrieval depth produces larger but more expensive gains. Step-by-step reasoning is less consistent and often exhausts its generation budget.

Contribution of this chapter. Comparisons of plain retrieval with graph memory exist for larger readers [Han et al., 2026, Jia et al., 2025], and the ordering found here agrees with them. This chapter makes the comparison with five readers of 0.8 to 4 billion parameters from two families, on the same paired questions, with graphs that the reader model extracts itself. It also checks which labelled evidence turns each prompt contains, which shows why truncated full context fails, reruns full context with a long window for three readers, and measures on one pipeline how much accuracy changes with retrieval depth and how much with reader size.

4.1 Evaluation scope and statistical interpretation

The primary LoCoMo analysis [Maharana et al., 2024] uses 200 questions from ten conversations. Each reader answers the same 200 questions under every memory condition, so conditions are compared with McNemar's exact test, and Wilson 95% intervals describe each accuracy. Gemma 4 is the primary reader family; the Qwen3.5 readers form a second, supporting family. Results are labelled primary, exploratory or descriptive as defined in Section 3.8. A conversation-level bootstrap checks sensitivity to clustering within the ten dialogue histories. The sample contains 50 questions from each evaluated category, so its accuracy is an unweighted category mean. It does not estimate performance under LoCoMo's original category frequencies.

Every accuracy in this chapter is a workstation measurement. Phone accuracy is reported in Chapter 5.

The graph results come from one SLM-extracted graph pipeline. Its primary condition gives the reader retrieved triples. A source-restoration ablation adds the original dialogue turns linked

to those triples without building a different graph.

The retrieval-depth and reasoning studies are exploratory. Their interpretation rests on effect sizes, coverage, token counts and failure rates.

4.2 LoCoMo strategy comparison

Table 4.1 reports the current-record LoCoMo grid. Vector retrieval leads all five rows, while full context ranks last despite filling the whole 4096-token window of the workstation server. Graph triples improve on full context for both Gemma readers and slightly improve the point estimate for the two smaller Qwen readers. Restoring source turns closes much of the graph-to-vector gap for the larger readers.

Reader	Full context	Graph triples	Graph + sources	Vector retrieval
Qwen3.5 0.8B	4.5%	6.0%	9.0%	**22.0%**
Qwen3.5 2B	7.0%	9.5%	9.0%	**32.5%**
Qwen3.5 4B	6.5%	23.5%	32.0%	**39.0%**
Gemma 4 E2B	7.5%	21.5%	33.0%	**36.0%**
Gemma 4 E4B	10.5%	27.5%	32.5%	**40.5%**

Table 4.1: LoCoMo question-answering accuracy across five readers and four memory representations ($n = 200$ paired questions per cell). "Graph triples" is the triple-only SLM-extracted graph. "Graph + sources" restores the verbatim dialogue turns attached to the retrieved graph facts. Bold values mark the highest point estimate within each reader. Full context is truncated to the 4,096-token window; Section 4.6 reports long-window runs for three readers.

4.2.1 Vector retrieval versus truncated full context

Table 4.2 collects the primary contrasts between vector retrieval and full context truncated to the 4,096-token window. The advantage ranges from 28.5 to 32.5 percentage points. Both Gemma cluster intervals exclude zero, and all three paired tests remain separated after Holm correction. Section 4.6 repeats the full-context condition with a long window.

Reader	Vector	Full	Difference	Vector/full wins	McNemar p	Holm-adjusted p	Cluster interval
Gemma E2B	36.0%	7.5%	+28.5	62/5	1.42×10^{-13}	9.94×10^{-13}	19.2 to 36.8
Gemma E4B	40.5%	10.5%	+30.0	66/6	7.26×10^{-14}	5.81×10^{-13}	21.2 to 37.6
Qwen 4B	39.0%	6.5%	+32.5	66/1	9.22×10^{-19}	8.29×10^{-18}	not estimated

Table 4.2: Primary LoCoMo comparisons between vector retrieval and truncated full context ($n = 200$ paired questions per row). Differences and cluster intervals are in percentage points. "Vector/full wins" gives discordant questions answered only by vector retrieval and only by full context. McNemar p is the exact two-sided test on those discordant questions; the Holm-adjusted value corrects it across the nine primary comparisons. Cluster intervals resample the ten conversations and are reported for the Gemma comparisons.

Full context stops being complete when the conversation exceeds the input window. The prompt keeps the newest turns and drops the oldest until it fits. Vector retrieval uses fewer tokens but can select turns from anywhere in the stored conversation. Both conditions select evidence. One uses recency and the other uses semantic similarity to the question.

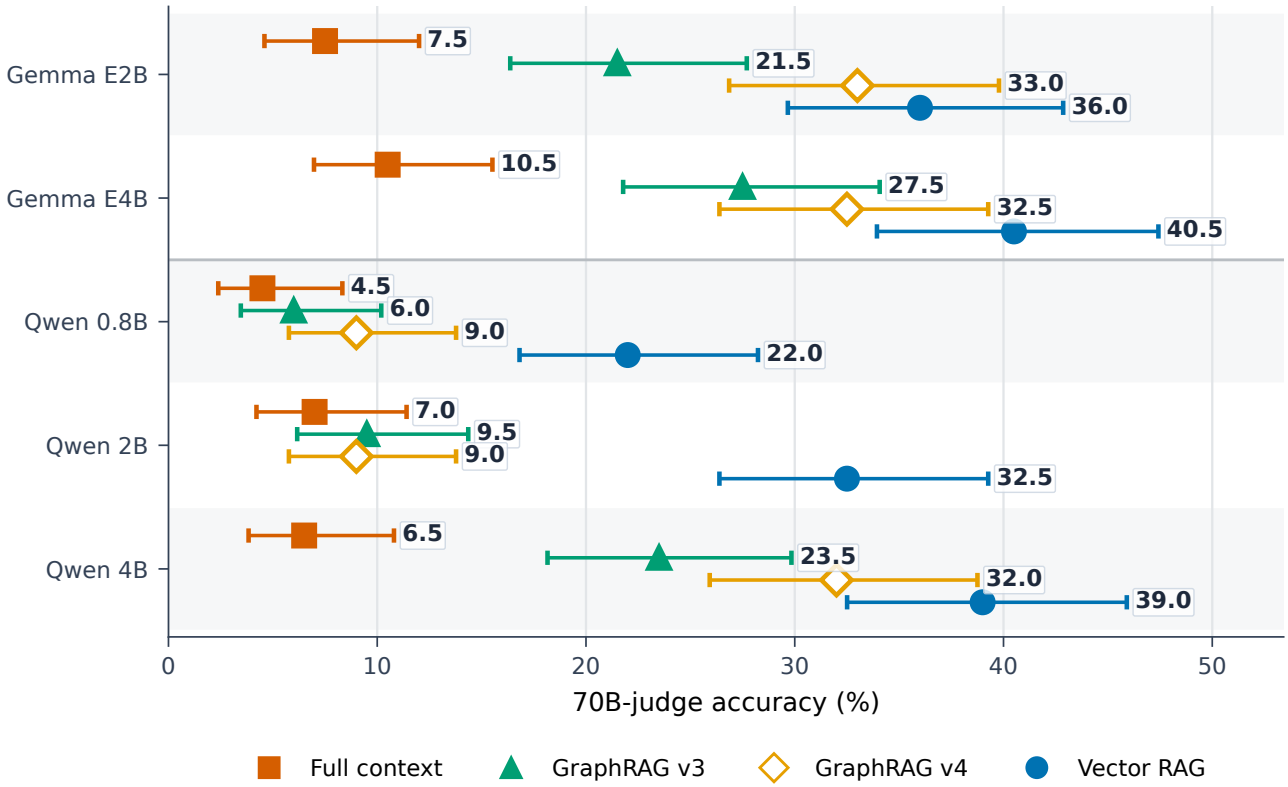


Figure 4.1: Memory-strategy accuracy on LoCoMo. Points are observed proportions and whiskers are Wilson 95% confidence intervals ($n = 200$ per condition). With full context truncated to the 4,096-token window, vector retrieval has the highest point estimate for every reader. Overlapping marginal intervals do not replace the paired McNemar tests reported in the text.

Table 4.3 measures the resulting coverage before the reader generates an answer. Full context contains at least one labelled evidence turn for 4.5% of eligible questions. Vector retrieval raises this to 63.8%. Complete evidence remains harder to retrieve, reaching 44.2% under vector retrieval.

Prompt policy	Any evidence	Complete evidence	Turn-level recall
Recent suffix (full context)	4.5%	0.5% (1/199)	2.4%
Semantic retrieval ($k = 10$)	63.8%	44.2%	42.1%

Table 4.3: Evidence coverage for the main Gemma LoCoMo prompts. Coverage uses the 199 of 200 questions with a usable evidence annotation. All 200 questions remain in the accuracy evaluation. E2B and E4B have identical coverage because prompt assembly is shared.

The retrieval arm is deliberately plain: turn-level embeddings, cosine similarity, no reranking, no query rewriting, and no component trained on either benchmark (Section 3.3.1). Published comparisons suggest that plain retrieval of this kind is a strong baseline. On LongMemEval-S with a GPT-4o-mini reader, a plain vector store scored 0.600 against 0.602 for Mem0, which took about 330 times longer per question (Mem0 was run on a 10% sample of the questions) [Jia et al., 2025]. In the MemoRAG evaluation, dense retrieval averaged 29.7 against 29.4 for GraphRAG global search over community summaries [Qian et al., 2025]. On LoCoMo, flat top-20 retrieval scored 0.67 with an LLM judge, against 0.41 to 0.57 for six write-time memory-construction methods; only two query-time construction methods scored higher, at 0.68 and 0.69 [Yu et al., 2026b].

4.2.2 Vector retrieval versus the triple-only graph

Graph triples use fewer prompt tokens but lose 14.5 points to vector retrieval for Gemma E2B and 13.0 points for E4B. Both differences have Holm-adjusted $p = 0.000327$ (exact McNemar $p = 8.17 \times 10^{-5}$ for E2B and 1.06×10^{-4} for E4B), and both conversation-cluster intervals exclude zero. Qwen 4B shows a similar 15.5-point gap (raw $p = 1.64 \times 10^{-6}$, exploratory). The smaller Qwen graphs reach only 6.0% at 0.8B and 9.5% at 2B. Section 4.3 links these weak cells to extraction failures.

4.2.3 What source restoration changes

For Gemma E2B, adding the source turns raises accuracy from 21.5% to 33.0%, an 11.5-point gain (McNemar $p = 3.40 \times 10^{-5}$, Holm-adjusted $p = 0.000170$). Vector retrieval reaches 36.0%, leaving a three-point difference (raw $p = 0.327$). Access to the underlying text therefore recovers much of the triple-only deficit.

The direction repeats for Gemma E4B, from 27.5% to 32.5%, and Qwen 4B, from 23.5% to 32.0%. These comparisons are exploratory, with raw $p = 0.0129$ and $p = 0.000221$. Source restoration changes little for Qwen 0.8B and 2B because their graphs contain little useful material. The extractor must first identify a relevant fact and preserve its provenance.

Source restoration raises any-evidence coverage to 48.7% for E2B and 48.2% for E4B, against 63.8% for vector retrieval. Both methods start from the same ten retrieved turns; the graph then restricts which turns can supply text through its selected triples. Chapter 6 examines this selection loss and the information recovered by source restoration. A larger-model comparison reports the same ordering of triples, triples with source text, and flat retrieval under a different protocol [Han et al., 2026].

4.2.4 Where the differences occur

The balanced sample has 50 questions in each evaluated LoCoMo category: multi-hop, temporal reasoning, open-domain knowledge, and single-hop. The fifth category is excluded throughout under the common four-category convention described in Chapter 3.[1] Table 4.4 gives the decomposition for the two Gemma readers. Each cell has five times the sample size of the LongMemEval category cells reported later.

Reader	Memory	Multi-hop (Cat. 1)	Temporal (Cat. 2)	Open-domain (Cat. 3)	Single-hop (Cat. 4)	Overall
E2B	Full context	10.0%	**0.0%**	20.0%	**0.0%**	7.5%
E2B	Graph triples	12.0%	12.0%	26.0%	36.0%	21.5%
E2B	Graph + sources	16.0%	22.0%	34.0%	60.0%	33.0%
E2B	Vector retrieval	32.0%	18.0%	32.0%	62.0%	36.0%
E4B	Full context	12.0%	**0.0%**	30.0%	**0.0%**	10.5%
E4B	Graph triples	12.0%	18.0%	32.0%	48.0%	27.5%
E4B	Graph + sources	16.0%	16.0%	36.0%	62.0%	32.5%
E4B	Vector retrieval	34.0%	20.0%	42.0%	66.0%	40.5%

Table 4.4: LoCoMo accuracy decomposed by question category ($n = 50$ per cell, 200 per row). Category names follow the benchmark. The dataset's numeric field is given in parentheses because its ordering differs from the paper's prose. Bold marks the two categories on which full context answers no question correctly under either reader.

An omnibus supporting analysis tests whether these differences are larger than expected from conversation-level difficulty alone. Within each of the ten conversations, the analysis shuffled correct and incorrect labels across the four category labels 100,000 times and recomputed Pearson's 4×2 chi-square statistic. Holm correction was then applied across the eight Gemma reader-memory cells in Table 4.4. Question category remained associated with correctness in every cell; the largest adjusted p-value was 0.00548. For Vector RAG, the observed statistics were $\chi^2(3) = 22.40$ at E2B and $\chi^2(3) = 23.14$ at E4B, with adjusted $p = 0.00015$ in both cases. This analysis was added after the category decomposition and is not part of the primary strategy-comparison family. It is an omnibus result, so it establishes that at least one category differs within each cell, not that every pair of category percentages is distinguishable. Appendix A gives the full table and reproduction command.

The category table reveals four patterns.

- Full context answers none of the 100 temporal and single-hop questions correctly under either reader. A single-hop answer sits in one session, but truncation often removes that session before inference.
- Single-hop questions account for 15.5 of the 28.5 vector-over-full-context points at E2B and 16.5 of the 30.0 points at E4B. Graph triples score 36% and 48% on this category. Conditions that include source text reach 60% to 66%.
- Temporal reasoning remains weak. Vector retrieval reaches 18% and 20%, and no condition exceeds 22%. LongMemEval-S shows the same 20% result for both vector readers.

[1]The dataset's numeric category label does not follow the order in which the benchmark paper's prose enumerates the five types. The mapping used here was verified against the published category frequencies, against the evidence span of each category. Multi-hop items cite 2.68 distinct sessions on average and are 95.4% multi-session, while single-hop items cite 1.00 and are 0.2% multi-session. The questions were also inspected directly.

- Multi-hop questions do not show the expected graph advantage. Vector retrieval reaches 32% and 34%. The source-restored graph reaches 16%, and graph triples reach 12%, under both readers.

The first and last of these patterns are not confined to Gemma. Across all five readers of Table 4.1, full context answers no temporal question correctly, and it reaches at most 2% on single-hop questions. Vector retrieval also leads the source-restored graph on multi-hop questions in every reader: 14% against 6% at Qwen 0.8B, 22% against 0% at Qwen 2B, and 36% against 18% at Qwen 4B, alongside the two Gemma pairs above. Each cell holds 50 questions, so individual values are coarse, but the direction repeats in five readers across two model families.

The absence of a multi-hop advantage applies to the tested retrieval design. Versions 3 and 4 select triples associated with retrieved turns; they do not traverse the graph. The earlier one-hop retrieval rule performed worse, but this does not test richer traversal methods such as Personalized PageRank [Jiménez Gutiérrez et al., 2024] or community summaries [Edge et al., 2024]. Chapter 6 examines the speaker-hub structure and a traced retrieval failure. Multi-hop evidence coverage was not measured separately, so these scores alone cannot distinguish missing evidence from difficulty using it.

The vector lead reported for every reader cell is an aggregate result and does not hold in every category. At E2B the source-restored graph is nominally ahead on temporal reasoning, 22% against 18%, and on open-domain knowledge, 34% against 32%. Those margins are two questions and one question respectively, well inside the sampling noise of a 50-question cell, and both reverse at E4B. They are recorded here so that the category table is not read as contradicting the aggregate ranking, not as evidence of a category-specific advantage.

Equal category weighting differs from LoCoMo's natural question mix, in which single-hop questions form about 55% of the answerable pool. Section 4.8 reports the reweighted results. The balanced accuracies should not be compared directly with published scores that use the natural category frequencies.

4.3 Model scaling within each memory strategy

Reader size usually improves vector retrieval. Gemma rises from 36.0% at E2B to 40.5% at E4B, a 4.5-point gain. The paired difference does not separate from zero (McNemar $p = 0.108$, Holm-adjusted $p = 0.115$), so it remains an effect-size estimate, not evidence that scaling has no effect. Qwen rises monotonically from 22.0% at 0.8B to 32.5% at 2B and 39.0% at 4B.

Full context has no comparable trend. Qwen scores 4.5%, 7.0%, and 6.5% across 0.8B, 2B, and 4B. Gemma rises from 7.5% to 10.5%. Increasing reader size cannot recover turns removed during prompt construction.

The triple-only graph has a different scaling curve. Qwen stays near the full-context floor at 0.8B and 2B, then rises to 23.5% at 4B. The same model performs extraction and answering, so this jump combines a better graph with a stronger reader. It cannot be assigned to reader scale alone. The reader-extractor crossover in Chapter 6 separates those roles.

Extraction failed on 79.0% and 84.6% of batches for Qwen 0.8B and 2B, compared with 0.6–3.8% for the other three extractors. The two smallest graphs therefore contain little material before retrieval begins. Table 6.5 reports the failure counts and extraction-budget controls. Parse rate alone does not explain answer quality: increasing output volume did not reliably recover accuracy, and E4B built a sparser graph than E2B while scoring higher.

Qwen 4B and Gemma E2B also have similar profiles: 39.0% and 36.0% for vector retrieval, 23.5% and 21.5% for graph triples, 32.0% and 33.0% after source restoration, and 6.5% and

7.5% for full context. This descriptive comparison is not a controlled architecture test. It shows only that the observed ordering appears in both families.

Figure 4.2 compares the scaling curves. Replacing full context with vector retrieval gains 17.5–32.5 points at a fixed reader. Within vector retrieval, doubling Gemma reader size gains 4.5 points; increasing Qwen size fivefold gains 17.0. The graph gains with Qwen scale combine changes in extraction and reading.

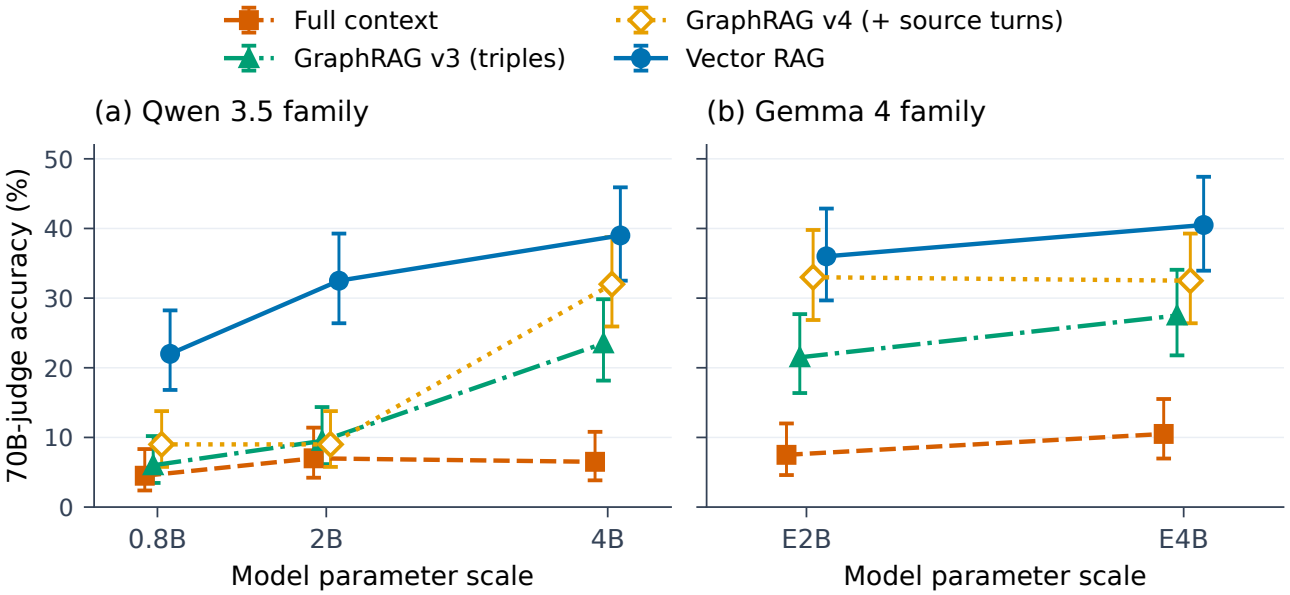


Figure 4.2: LoCoMo accuracy by reader scale and memory strategy ($n = 200$ paired questions per condition). Panels show (A) Qwen3.5 and (B) Gemma 4; whiskers are Wilson 95% intervals. Markers are offset for visibility. Graph conditions use the same model for extraction and answering, so their scaling curves combine changes in graph construction and reader capability.

The retrieval-depth experiment next measures larger changes, but on a different LoCoMo subset.

4.4 Retrieval-depth analysis

The vector-retrieval baseline uses the ten most similar turns. To test whether this setting hides a different accuracy-cost trade-off, retrieval depth k was varied over 2, 5, 10, 20, and 50 on the same balanced set of 100 LoCoMo questions. The retrieved turns and prompt footprints are identical for E2B and E4B at each depth. Only the reader changes.

Coverage is measured against the dataset's labelled evidence turns. "Any" counts questions with at least one such turn in the prompt; "complete" requires all labelled turns. Turn-level recall is the fraction of labelled turns retrieved, pooled over questions. These measures differ when a question requires several evidence turns.

Increasing k from 2 to 50 raises any-evidence coverage from 26% to 85%, complete coverage from 16% to 64%, and accuracy by 15–27 points across the five readers. Mean Gemma prompt length grows from 296 to 3,310 tokens; Qwen tokenizes the same text about 1.5% shorter. Each depth reuses the same embeddings, so the additional work is at query time rather than ingestion.

Over the tested range, retrieval depth changed accuracy more than the reader doublings that were tested. Doubling the reader from Gemma E2B to E4B and from Qwen 2B to 4B gained 4.5 and 6.5 points on the main grid, and 5 and 8 points within this sweep at $k = 10$. The comparison is limited in three ways. It sets the full range of one setting, a 25-fold change

	Evidence coverage				Accuracy				
					Qwen3.5			Gemma 4	
k	Any	Complete	Turn level	Prompt tokens	0.8B	2B	4B	E2B	E4B
2	26%	16%	15%	296	7%	17%	17%	18%	17%
5	52%	36%	34%	488	18%	22%	29%	27%	30%
10	62%	44%	44%	808	21%	28%	36%	30%	35%
20	73%	56%	57%	1,438	22%	27%	36%	39%	38%
50	85%	64%	69%	3,310	26%	32%	42%	42%	44%
				Gain, $k = 2$ to $k = 50$	+19	+15	+25	+24	+27

Table 4.5: Retrieval-depth sweep on 100 paired LoCoMo questions. All five readers receive the same turns at each depth from a shared embedding store. Coverage measures are defined in the text. Prompt tokens are Gemma means; Qwen tokenizes the same text about 1.5% shorter. Accuracy is judged separately for each reader.

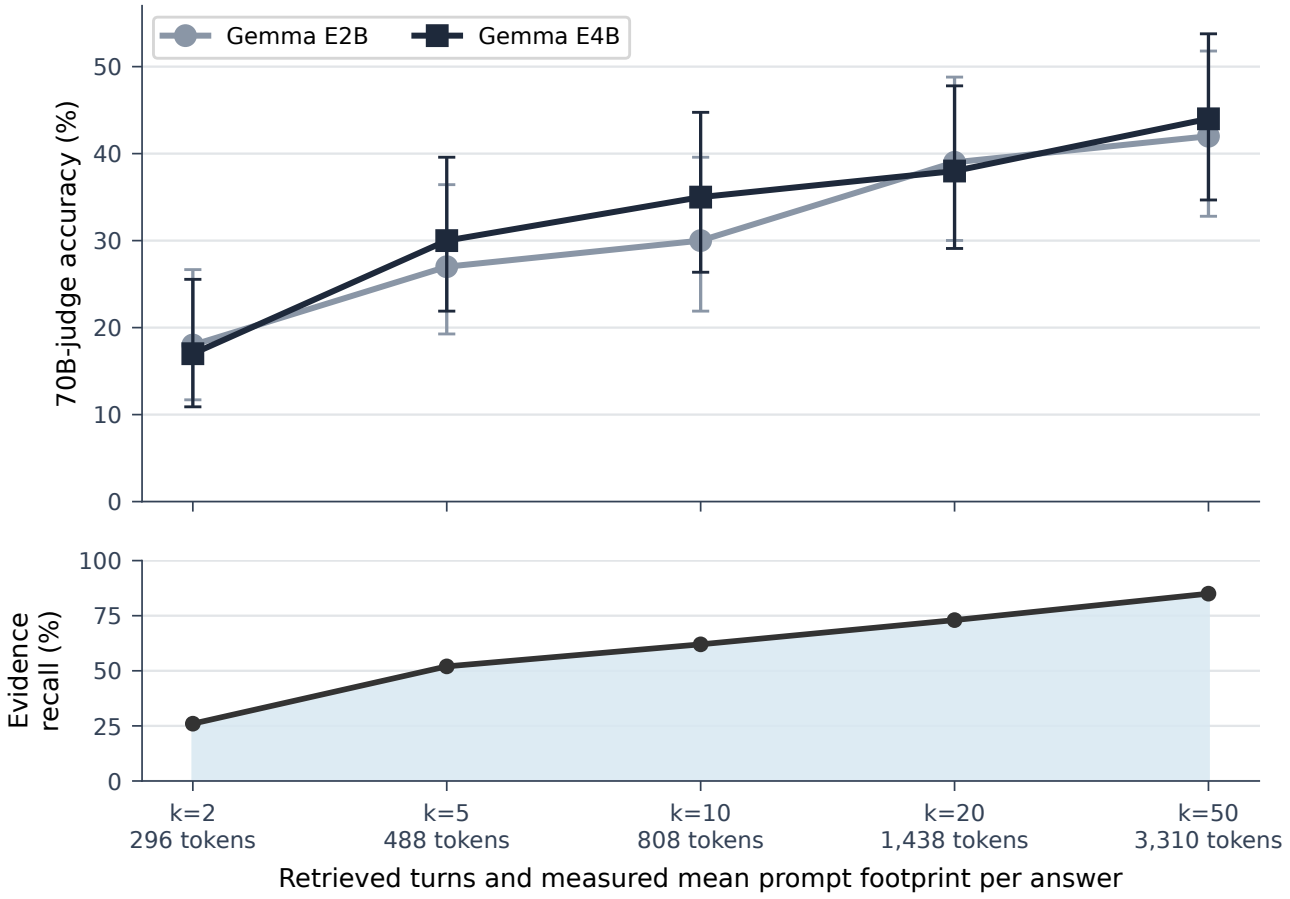


Figure 4.3: Prompt size, evidence coverage and accuracy across retrieval depths for Gemma 4 E2B and E4B on the 100-question LoCoMo subset. Whiskers are Wilson 95% accuracy intervals. "Evidence recall", shared by both readers, denotes questions with at least one labelled evidence turn in the prompt; complete-evidence and turn-level recall appear in Table 4.5. This subset differs from the 200-question strategy grid.

in depth, against a single twofold step in reader size. The depth gain also costs an elevenfold longer prompt. Finally, it covers readers of 0.8 to 4 billion parameters and a 4,096-token history budget, so it does not show that reader size matters less in general. Retrieval depth complements the method, reasoning and capacity choices considered in BudgetMem [Zhang et al., 2026b]. Its phone costs are examined in Chapter 5.

The intermediate changes differ by reader. E2B gains nine points from $k = 10$ to $k = 20$ (raw $p = 0.022$), while E4B and Qwen 4B change little over that interval. Qwen 2B drops one answer before recovering at $k = 50$. Across the full sweep, the three Qwen gains have raw $p = 2.10 \times 10^{-5}$, 0.00593, and 4.65×10^{-6} respectively; these tests are exploratory. The adjacent $k = 5$ to 10 and $k = 20$ to 50 comparisons do not separate on this sample. No universal retrieval depth follows from these curves.

Table 4.6 separates complete, partial, and absent evidence at $k = 50$. Complete evidence outperforms partial evidence in every reader, by between 4.2 and 26.1 points.

		Qwen3.5			Gemma 4	
Evidence in prompt	Questions	0.8B	2B	4B	E2B	E4B
Complete	64	31.2%	37.5%	51.6%	51.6%	54.7%
Partial	21	19.0%	33.3%	38.1%	33.3%	28.6%
None	15	13.3%	6.7%	6.7%	13.3%	20.0%

Table 4.6: Accuracy by evidence completeness at $k = 50$ on the shared 100-question LoCoMo subset. The question counts are shared because all five readers received the same retrieved turns, so the columns differ only in how each reader used them.

Within each evidence group, all five readers receive the same prompts. On complete-evidence questions at $k = 50$, accuracy ranges from 31.2% to 54.7%, showing that reader capability still matters when retrieval succeeds. Comparisons between evidence groups involve different questions and therefore mix evidence access with question difficulty.

Complete-evidence accuracy changes relatively little across the sweep: E2B moves from 50.0% to 51.6%, E4B from 50.0% to 54.7%, and Qwen 4B from 43.8% to 51.6%. This is consistent with aggregate gains coming mainly from more questions receiving complete evidence, although group membership changes with depth. Figure 4.4 shows how any-evidence coverage and correctness change together for the Gemma readers.

The shorter LongMemEval-S sweep follows the same broad pattern, as shown in Table 4.7.

k	Mean prompt tokens	E2B accuracy	E4B accuracy
2	638	36.7%	36.7%
5	1,338	48.3%	45.0%
10	2,488	48.3%	56.7%

Table 4.7: LongMemEval-S retrieval-depth sweep ($n = 60$ paired questions per row). The comparisons are descriptive.

E2B reaches its observed plateau by five turns, while E4B continues to improve at ten. The sample does not establish a universal optimum.

Graph retrieval has a separate depth control. Increasing the limit from twelve to twenty-four facts adds 2.0–8.0 points for the three extractors with low failure rates, but only 0.5–1.0 points for the two smaller Qwen models. Chapter 6 reports the full control and prompt costs in Section 6.5. More retrieved facts cannot recover information that never entered the graph.

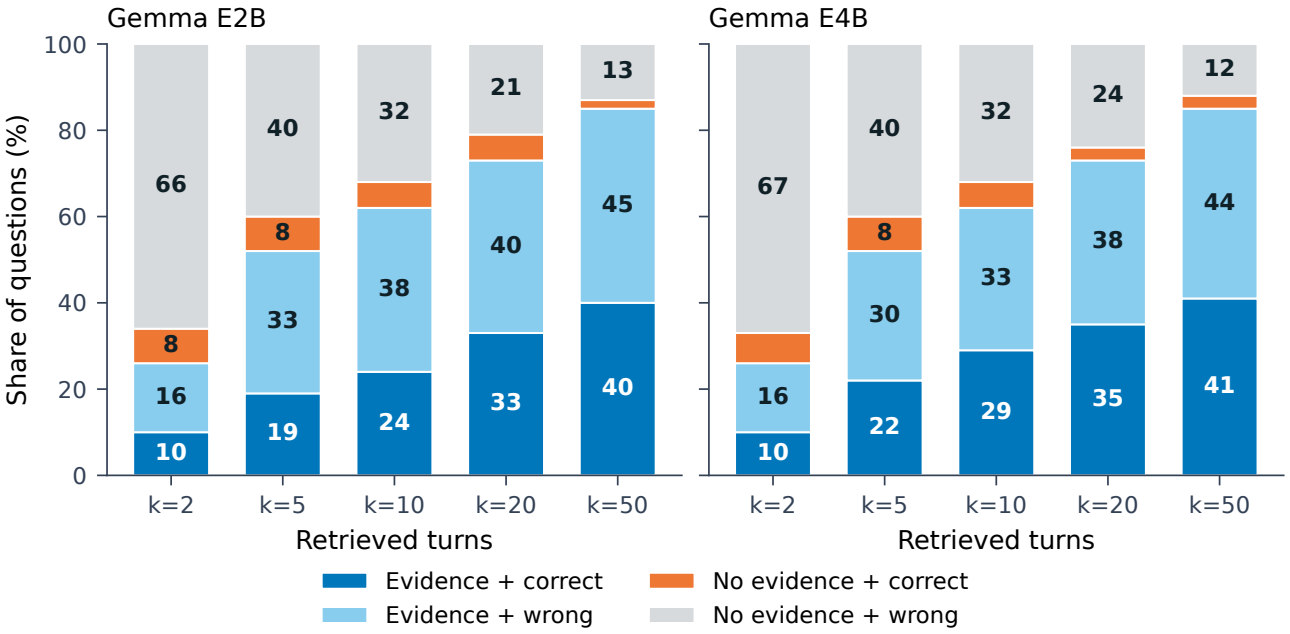


Figure 4.4: Share of questions in each combination of evidence presence and answer correctness across retrieval depths for Gemma 4 E2B and E4B ($n = 100$ paired LoCoMo questions per cell). Evidence is present when at least one labelled evidence turn reaches the prompt. Each question belongs to one of the four combinations; evidence presence does not imply complete evidence.

4.5 Transfer from LoCoMo to LongMemEval-S

LoCoMo contains multi-session conversations with comparatively short turns. LongMemEval-S has much longer histories and six labelled question types: single-session user, assistant, and preference questions, multi-session, knowledge-update, and temporal-reasoning [Wu et al., 2025]. Some items are abstention variants of these types. The transfer evaluation uses a stratified subset of 60 questions, ten from each labelled type. It replaces the earlier 50-question slice, which contained only single-session-user questions.

The two benchmarks handle abstention differently. LongMemEval marks abstention in the question identifier while retaining the item's original type. Its reference answer is a refusal such as "the information provided is not enough." Three of the 60 sampled questions have this marker and are scored under the benchmark's standard protocol. LoCoMo follows the separate four-category QA convention described in Chapter 3.

The three abstention questions are the only group in which the ranking reverses. A strategy that finds no relevant evidence is more likely to refuse and receive credit. E2B full context answers two of the three correctly, while vector retrieval answers none. Removing these items widens the vector-over-full-context gap from 38.3 to 43.9 points for E2B and from 51.7 to 52.6 points for E4B. They remain in the 60-question result. Three cases are too few for a separate abstention analysis.

Vector retrieval reaches 48.3% for E2B, 38.3 points above truncated full context (McNemar $p = 1.52 \times 10^{-5}$, Holm-adjusted $p = 9.14 \times 10^{-5}$). E4B reaches 56.7%, with 32 vector-only wins and one full-context-only win against the 5.0% full-context result (raw $p = 7.92 \times 10^{-9}$, exploratory). The graph sits between the two, as it does on LoCoMo: triples alone reach 25.0% for E2B and 23.3% for E4B, and restoring the source turns raises them to 41.7% and 33.3%.

The three Qwen cells extend the comparison to the full reader ladder. No full-context cell in any reader clears 10%, and Qwen 2B answers none of the 60 questions correctly under full

Reader	Full context	Graph triples	Graph + sources	Vector retrieval
Qwen3.5 0.8B	1.7%	—	—	**26.7%**
Qwen3.5 2B	0.0%	—	—	**38.3%**
Qwen3.5 4B	5.0%	—	—	**50.0%**
Gemma 4 E2B	10.0%	25.0%	41.7%	**48.3%**
Gemma 4 E4B	5.0%	23.3%	33.3%	**56.7%**

Table 4.8: Accuracy on the category-balanced LongMemEval-S subset ($n = 60$, ten questions per type). Retrieval uses corrected embedding prefixes; Gemma graph cells use the repaired pipeline, with v4 adding source turns. All five vector readers share one embedding store. Dashes mark Qwen graph conditions that were not evaluated. Full context is truncated to the 4,096-token window; Section 4.6 reports long-window runs for three readers.

context. Vector retrieval meanwhile tracks reader capacity closely across the Qwen ladder, at 26.7%, 38.3% and 50.0%. The vector-over-full-context difference is between 25.0 and 51.7 points in every reader, with raw $p = 4.63 \times 10^{-7}$ at Qwen 4B and comparable values at the smaller tiers.

The two arms behave differently as the reader grows, and that contrast is the clearest form of this chapter's argument. Truncated full context stays near zero at every scale: on histories this long, truncation leaves the labelled evidence in only about one prompt in thirty for the Gemma readers, so reading capacity has little to work with. Vector retrieval rises with scale, consistent with a larger reader making more use of the evidence that retrieval supplies. Even the smallest reader converts retrieved evidence into 26.7%, against 1.7% on the same questions without it.

The graph repair transfers to this benchmark. The first LongMemEval graph cells were built with the pre-repair pipeline: 79 to 81% of their extraction batches failed on these long histories, and triple-only accuracy was 8.3% for E2B and 10.0% for E4B. Rebuilt with the repaired pipeline and task prefixes, the same histories lose 24.5% and 31.0% of their batches, and triple-only accuracy rises to 25.0% and 23.3% (11 against 1 and 10 against 2 discordant questions, exact McNemar raw $p = 0.0063$ and $p = 0.039$). Both repaired graphs also exceed full context (raw $p = 0.012$ for E2B and $p = 0.001$ for E4B). The failure rate remains far above the roughly 1% measured on LoCoMo (Chapter 6) because LongMemEval turns are longer: 81% of the failed E2B batches and nearly all failed E4B batches end mid-output at the 512-token extraction limit, and the turns in a failed batch contribute no triples.

Source restoration repeats its LoCoMo pattern. For E2B it adds 16.7 points (11 against 1, raw $p = 0.0063$) and leaves vector retrieval 6.7 points ahead, a difference the paired test cannot separate at $n = 60$ (9 against 5, raw $p = 0.42$). For E4B it adds 10.0 points (9 against 3, raw $p = 0.15$), and vector retrieval stays 23.3 points ahead (16 against 2, raw $p = 0.0013$). The graph prompts are the shorter ones, at about 620 tokens with triples alone and 1,600 with sources against about 2,490 for vector retrieval. All of these LongMemEval graph comparisons are exploratory.

The poor full-context result has a direct structural explanation. LongMemEval histories can be far longer than the reader's context window, so suffix truncation discards most of the session. The graph avoids the prompt-length wall and, once repaired, recovers part of the evidence, but only from the turns its extractor processed successfully. Vector retrieval indexes every stored turn and returns source text within the input budget. Section 4.6 tests this explanation by removing the truncation.

Table 4.9 shows where vector retrieval succeeds and where errors remain.

The vector-retrieval profiles share broad strengths and weaknesses. Single-session and

	Qwen3.5			Gemma 4	
LongMemEval-S type	0.8B	2B	4B	E2B	E4B
Single-session user	60%	80%	90%	80%	90%
Single-session assistant	30%	70%	90%	90%	90%
Single-session preference	0%	0%	0%	20%	10%
Multi-session	0%	10%	30%	10%	50%
Knowledge-update	60%	70%	80%	70%	80%
Temporal-reasoning	10%	0%	10%	20%	20%

Table 4.9: Vector-retrieval accuracy by LongMemEval-S question type. Each cell contains ten questions, so differences between adjacent values are descriptive.

knowledge-update scores reach 90% in the larger readers. Preference and temporal-reasoning scores remain at or below 20% for all five readers, while multi-session scores reach at most 50%. Larger readers improve several categories without resolving these weaknesses. Figure 4.5 compares memory strategies overall and by category for Gemma. The overall ordering has category exceptions: on preference questions, E4B graph triples score 40%, above vector retrieval's 10% for E4B and 20% for E2B; with ten questions per cell, this is a descriptive exception rather than an established advantage.

The strong overall vector baseline is consistent with the LongMemEval-S comparison of Jia et al. [2025], although reader, sample, prompt and scoring differences prevent direct numerical comparison.

Absolute accuracy is not comparable across LoCoMo and LongMemEval because their history lengths, question composition, and sample sizes differ. The transfer claim concerns the ordering within each benchmark: vector retrieval leads truncated full context and graph triples in both.

Figure 4.6 illustrates the resulting within-benchmark trade-offs by plotting 70B-judge accuracy against mean prompt tokens across both populations.

4.6 Full context with a long window

Every full-context cell above cuts the history to fit the workstation's 4,096-token window, so it measures what a small window leaves for the reader rather than what the reader can do with the whole conversation. To separate the two, the full-context condition was rerun for Gemma 4 E2B, E4B and Qwen3.5 4B with the server window raised to 131,072 tokens (262,144 for Qwen3.5 4B on LongMemEval-S). Everything else matched the record runs: the same questions, prompt, greedy decoding, 64-token answer cap and 70B judge. No prompt reached the server window. The harness still applies its own history budget, the window less 192 tokens, counted as characters divided by four. Every LoCoMo prompt and every Qwen3.5 4B LongMemEval-S prompt held the complete history. In each Gemma LongMemEval-S run, the budget dropped the oldest 2 to 4 of about 500 turns from four of the 60 questions, and every other prompt held the complete history. Mean prompts were 31,605 to 32,089 tokens for LoCoMo and about 118,000 for LongMemEval-S. These runs are exploratory. Their tests are reported unadjusted and lie outside the Holm family of nine primary comparisons. Table 4.10 sets them against the truncated and vector cells for the same readers.

On LoCoMo, raising the window lifts full context from 6.5–10.5% to 41.5–44.0%, with 68 to 77 questions gained and 2 to 5 lost per reader. Long-window full context then has a higher point estimate than vector retrieval for all three readers. The difference is separated for E2B only, 44.0 against 36.0% (27 against 11 discordant questions, raw $p = 0.014$); for E4B and

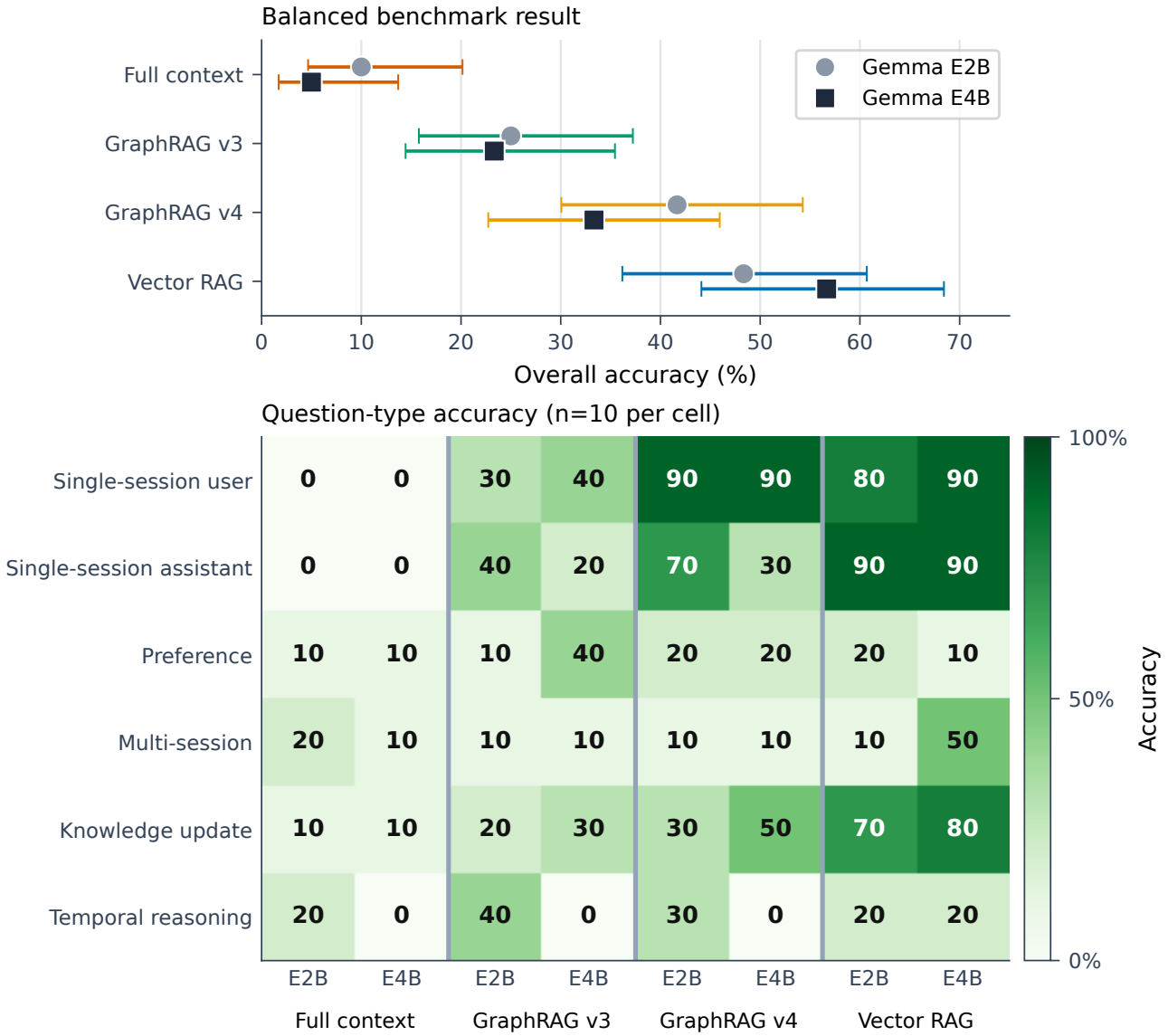


Figure 4.5: LongMemEval-S results for the two Gemma readers ($n = 60$, ten questions per type). Top: overall accuracy by memory strategy, with Wilson 95% intervals. Bottom: descriptive accuracy by question type ($n = 10$ per cell). Retrieval cells use the prefixed record configurations of Table 4.8; Qwen results appear in Tables 4.8 and 4.9.

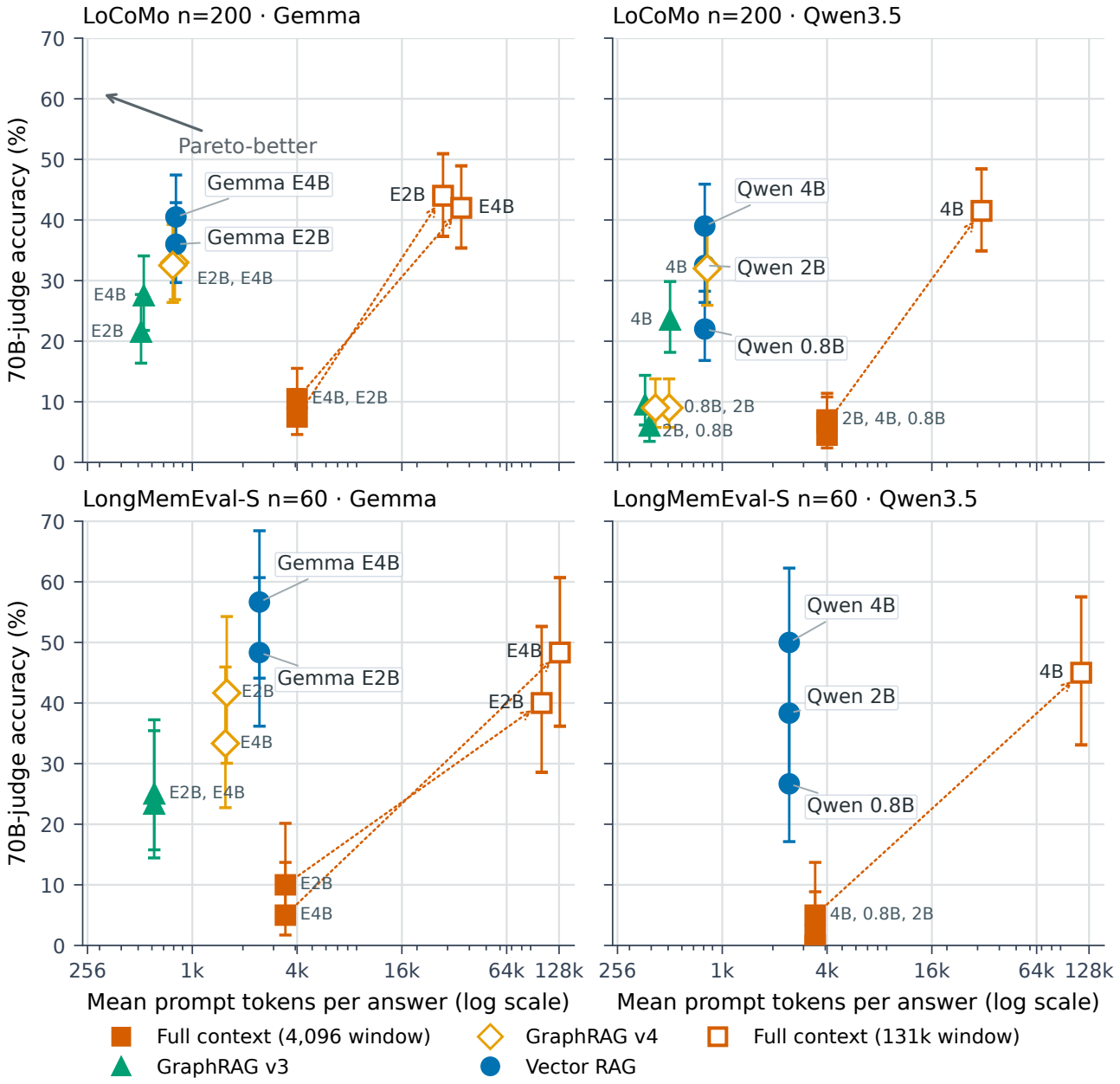


Figure 4.6: Accuracy against mean prompt tokens on LoCoMo ($n = 200$, top) and LongMemEval-S ($n = 60$, bottom). Gemma readers appear on the left and Qwen3.5 on the right. Each point is one reader under one strategy: Vector RAG points carry the reader's name, and graph and full-context points carry a short reader tag (several tags on one label list the readers from top to bottom). Readers that receive the same retrieved context share a prompt length, so their points stack vertically; the LoCoMo values are also given in Table 4.1. Whiskers are Wilson 95% intervals. LongMemEval graph points use the repaired Gemma runs; Qwen graph conditions were not evaluated there. Solid squares are full context cut to the 4,096-token window; hollow squares are long-window full context (Section 4.6), joined by a dotted arrow to the same reader's truncated point. The two Gemma long-window points share one prompt length and are offset horizontally for legibility. Rows represent different populations. Prompt tokens exclude memory-construction work.

Benchmark	Reader	Truncated	Long-window	Vector	Long/vector wins	McNemar p	Prompt tokens
LoCoMo	Gemma E2B	7.5%	**44.0%**	36.0%	27/11	0.014	32,089 / 824
	Gemma E4B	10.5%	**42.0%**	40.5%	17/14	0.72	32,089 / 824
	Qwen 4B	6.5%	**41.5%**	39.0%	24/19	0.54	31,605 / 812
LongMemEval-S	Gemma E2B	10.0%	40.0%	**48.3%**	3/8	0.23	118,324 / 2,488
	Gemma E4B	5.0%	48.3%	**56.7%**	3/8	0.23	118,324 / 2,488
	Qwen 4B	5.0%	45.0%	**50.0%**	3/6	0.51	118,178 / 2,483

Table 4.10: Truncated and long-window full context against vector retrieval ($n = 200$ paired LoCoMo and $n = 60$ paired LongMemEval-S questions per row). "Truncated" is the record cell cut to the 4,096-token window; "Long-window" raises the window to 131,072 tokens (262,144 for Qwen 4B on LongMemEval-S), and no prompt reached it; four of the 60 Gemma LongMemEval-S prompts lost their oldest 2 to 4 turns to the harness budget, and all other prompts hold the complete history. "Long/vector wins" gives discordant questions answered only by long-window full context and only by vector retrieval; McNemar p is the exact two-sided test on them, unadjusted. Prompt tokens are means, long-window full context against vector retrieval. Bold values mark the higher of the long-window and vector point estimates.

Qwen3.5 4B the two are 1.5 and 2.5 points apart, and the paired test cannot separate them. Long-window full context also exceeds the graph with restored sources for all three readers (raw $p \leq 0.0054$).

On LongMemEval-S, where the histories are nearly four times longer, the order reverses. Long-window full context again rises far above its truncated result, to 40.0–48.3% (23 to 26 questions gained and 0 to 5 lost), but vector retrieval keeps the higher point estimate for all three readers, by 8.3, 8.3 and 5.0 points. Most questions are answered the same way under both conditions: only 9 to 11 of 60 are discordant, split 3 against 8, 3 against 8 and 3 against 6, which 60 questions cannot separate (raw $p = 0.23$ to 0.51). Against the Gemma graphs with restored sources, long-window full context is level for E2B (40.0 against 41.7%, 9 against 10) and ahead for E4B (48.3 against 33.3%, 13 against 4, raw $p = 0.049$).

Two conclusions follow. First, the low truncated full-context scores in this chapter measure the 4,096-token budget, not the reader: given the whole LoCoMo conversation, each of the three readers answered at least as many questions as it did with ten retrieved turns. The vector advantage of Section 4.2.1 therefore holds under a small context budget, the setting this thesis targets; it does not show that retrieval beats reading the whole history. Second, retrieval kept most of that accuracy with a small fraction of the prompt. On LoCoMo it stayed within 1.5 to 8.0 points of the whole history with about 2.6% of the prompt tokens (824 against 32,089 for the Gemma readers), and on LongMemEval-S it kept the higher point estimate with about 2.1% (2,488 against 118,324).

These runs were made on the workstation only. The phone's server context was 8,192 tokens (16,384 for its long-window sittings), and no prompt of this length was run there. A rough extrapolation gives the scale. The phone's full-context sittings processed 11.0 to 11.9 prompt tokens per second of total query time (Section 5.6.1). Prefill and decode were not timed separately, so this rate includes the short answers. At that rate, a 32,089-token LoCoMo prompt would take about 45 to 49 minutes per question, and a LongMemEval-S prompt about three hours. The extrapolation assumes the rate holds at prompts several times longer than any run on the phone. It indicates only the scale of the cost and is not a measured time.

4.7 Reasoning experiments

The baseline readers produce short answers using a 64-token generation cap. The reasoning experiment enables the model's step-by-step mode and raises the cap to 768 tokens. This intervention tests whether additional inference can compensate for weak memory access or difficult evidence integration. Table 4.11 reports the all-question result, which keeps unfinished answers in the denominator and scores them as incorrect.

Vector and graph rows use their 64-token baselines because raising the cap alone leaves those conditions effectively unchanged. Full context needs a budget-matched baseline. Reserving 768 tokens for the answer shortens the dialogue history, moving E2B from 7.5% to 9.0% and E4B from 10.5% to 6.5%. The E4B shift has raw $p = 0.03857$. Reasoning is compared with these 9.0% and 6.5% controls.

Benchmark	Reader and memory	Base	Reasoning	Change	Truncated	Mean tokens
LoCoMo	E2B vector	36.0%	41.5%	+5.5	18.0%	535.1
LoCoMo	E4B vector	40.5%	43.5%	+3.0	18.5%	565.6
LoCoMo	E2B graph triples	21.5%	23.5%	+2.0	19.5%	590.7
LoCoMo	E4B graph triples	27.5%	23.5%	−4.0	34.5%	631.0
LoCoMo	E2B full context	9.0%	4.5%	−4.5	12.0%	223.1
LoCoMo	E4B full context	6.5%	6.0%	−0.5	3.5%	204.6
LongMemEval-S	E2B vector	48.3%	58.3%	+10.0	15.0%	500.3
LongMemEval-S	E4B vector	56.7%	60.0%	+3.3	13.3%	522.1

Table 4.11: Accuracy with reasoning disabled and enabled. "Truncated" is the proportion of reasoning runs that exhausted the 768-token generation budget without producing a parseable final answer. Mean tokens counts completion tokens in the reasoning run. The full-context rows use the budget-matched controls described in the text.

The vector condition gains 5.5 and 3.0 points on LoCoMo, then 10.0 and 3.3 points on LongMemEval-S. Raw p-values range from 0.080 to 0.727, and none belongs to the primary family. The measured benefit is possible but variable.

Graph triples gain 2.0 points for E2B and lose 4.0 for E4B once unfinished answers are counted. Full context also declines, from 9.0% to 4.5% for E2B and from 6.5% to 6.0% for E4B. Extra inference cannot reconstruct omitted evidence and may spend the available budget on an under-specified prompt.

The cap-only control moves E2B vector retrieval from 36.0% to 35.5% on one discordant question. E2B graph triples are identical question by question. The larger cap does not explain the reasoning deltas in those rows. Full context is the exception because answer and history tokens share one window. On the workstation that window was already full: the prompt plus the reasoning exceeded 4,096 tokens in 200 of 200 E2B and 199 of 200 E4B full-context reasoning questions, so the server had to discard context while the model reasoned. The full-context reasoning rows therefore mix the effect of reasoning with the effect of that overflow.

Reasoning expands the LoCoMo vector completions from 12.2 to 535.1 tokens for E2B and from 11.5 to 565.6 for E4B. Across source-bearing and graph conditions, 13.3% to 34.5% of outputs reach the 768-token cap without a parseable final answer. Baseline accuracy is much lower on these truncated cases, indicating that difficult questions also tend to consume the most tokens.

Among completed vector outputs, the gains are 7.9 and 7.4 points on LoCoMo and 13.7 and 7.7 points on LongMemEval-S. This sensitivity analysis is selective because reasoning affects whether an output finishes. It shows the effect of budget exhaustion, while the all-question result remains the deployment measure under a fixed cap.

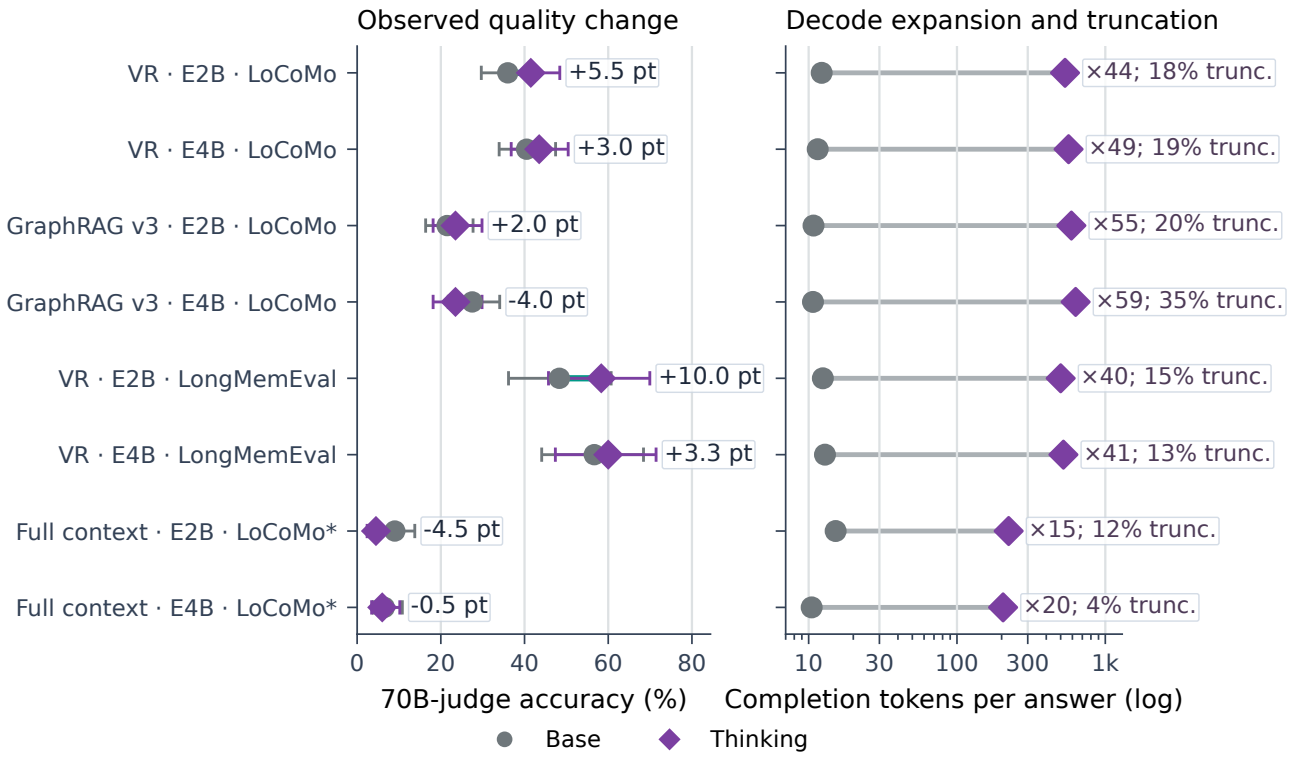


Figure 4.7: Accuracy, completion length and unfinished-output rates with reasoning enabled. Whiskers are Wilson 95% accuracy intervals; unfinished answers at the 768-token cap count as incorrect. Asterisks mark full-context rows, whose baseline uses the same 768-token answer cap. Other rows use the 64-token baseline. VR denotes vector retrieval. Right-panel labels give the ratio of reasoning to baseline completion tokens and the truncated share.

The experiment does not support reasoning as a substitute for retrieval. Its best results occur in the vector condition, but the gains vary and completion length rises by roughly 40 to 60 times.

4.8 Robustness checks

The two prompts wrap the same retrieved context and differ only in their instructions. LoCoMo's official prompt names the two speakers, states that each session begins with its date, and asks for a short phrase using exact words from the context. Our curated prompt (Section 3.4.1) replaces this with seven rules: answer in as few words as possible; give dates only, resolving relative terms such as "yesterday" from the timestamp of the turn; give names only; answer yes/no questions with "Yes" or "No"; do not explain; do not repeat the question; and place the answer between `<answer>` tags. The official prompt has no answer tags, so its first non-empty output line is taken as the answer. The comparison tests the prompt as a whole; the rules were not ablated one at a time, so the contribution of each rule is not measured.

Replacing our answer prompt with LoCoMo's official prompt lowered accuracy by 3–6 points across the Gemma conditions, while preserving the ordering of vector retrieval, graph triples and full context (Figure 4.8). Only the full-context drop separated statistically in this exploratory grid. Prompt wording affects absolute accuracy but did not reverse the strategy ranking in this check.

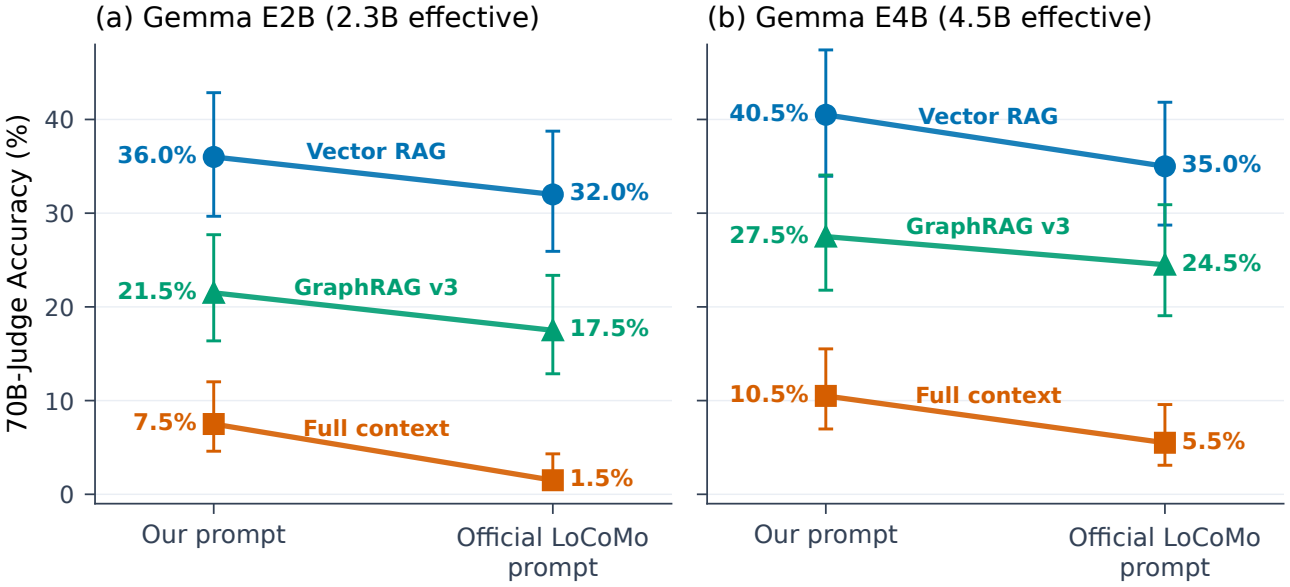


Figure 4.8: Accuracy with our answer prompt and with LoCoMo's official answer prompt, given the same retrieved context ($n = 200$ stratified LoCoMo questions, Gemma 4 E2B and E4B). The official prompt lowers accuracy in every condition, and the order of the three memory strategies is unchanged for both readers. Whiskers indicate Wilson 95% confidence intervals.

Reweighting the same answers to LoCoMo's answerable category frequencies preserves the ordering in Table 4.12.

The full-context-to-vector gap widens to 42.4 points for E2B and 45.0 points for E4B. Reweighting increases the influence of single-hop questions, which form about 55% of the answerable pool. Full context scores zero on this category and retrieval performs best there. The balanced results remain the headline because equal category weight was the sampling design.

The same prompt was deliberately used on LoCoMo and LongMemEval-S to preserve cross-benchmark comparability. LongMemEval's official answer prompt was not evaluated, so its

Reader	Full context	Graph triples	Graph + sources	Vector retrieval
Gemma E2B	3.1%	26.0%	42.40%	45.5%
Gemma E4B	4.1%	34.2%	42.37%	49.1%

Table 4.12: LoCoMo accuracy reweighted to the category frequencies of the answerable pool. The two source-restored values are reported to two decimals because they coincide at one decimal, but they are separate reader measurements.

absolute accuracy values remain prompt contingent. The shared-prompt ordering still holds, but the study does not show how high each strategy might score with benchmark-specific prompt engineering.

The primary LoCoMo comparisons were also checked by resampling entire conversations. The cluster intervals retain the direction of the vector advantage for both Gemma readers. With only ten conversation clusters, these intervals are coarse and can underestimate uncertainty. The paired McNemar tests remain the primary inference. The Qwen grid and most ablations are descriptive replications outside the primary family.

4.9 Experimental limitations

The conclusions are bounded by the experimental design. LoCoMo evaluation covers conversational question answering under the common four-category protocol. It excludes event summarization and multimodal dialogue. The 200 questions come from ten conversations, so outcomes are not independent. Paired tests and the cluster sensitivity analysis preserve the direction of the main comparison.

The supporting analyses are descriptive and have three measurement limits.

- Each LoCoMo category contains only 50 questions. At 30–50% accuracy, the 95% Wilson interval of a 50-question cell spans about ±12 to 13 percentage points, so differences of a few points between categories are within this sampling uncertainty.
- Evidence coverage uses lexical matching. A turn counts only when its normalized text appears verbatim in the prompt, so paraphrases and mid-turn truncation are misses. The figures are lower bounds. Dataset labels identify sufficient evidence, not uniquely necessary evidence, and answers may use unlabelled context.
- Extraction counters come from the evaluation harness. A parseable batch may still contain unusable triples, so these counters are not an independent graph-quality audit.

These measurements describe the prompts and extraction process. They do not change the reported accuracy values.

LongMemEval-S uses a stratified sample of 60 questions, not the complete 500-question benchmark. Each question type contains ten examples, which is enough to reveal large descriptive differences but not to support fine-grained category claims. Three of those 60 are abstention items, retained because they belong to that benchmark's evaluation set. They slightly reduce the reported retrieval advantage and are too few to analyse separately. The LongMemEval full-context and vector conditions were run for all five readers, but no LongMemEval graph cell exists for any Qwen reader, because that would require a cold extraction pass the transfer claim does not rest on. Neither benchmark was run across the complete reader ladder in every condition, so this remains an incomplete benchmark-by-family factorial study, not a full one. The long-window full-context runs cover three readers and were made on the workstation only;

their comparisons with vector retrieval are exploratory, and on LongMemEval-S they rest on 9 to 11 discordant questions per reader. Four of the 60 Gemma LongMemEval-S prompts lost their oldest 2 to 4 turns. For each Gemma reader, full context answered three of the four wrongly, and vector retrieval answered one of those three correctly. In the hypothetical best case for full context, all three would become correct with the complete history; the split would then be 5 against 7, which is still not separated (exact $p = 0.77$).

The judge's limits, including the single-annotator human audit, are stated in Section 3.6 and Section 8.5. Retrieval coverage is measured against dataset-labelled evidence and judge correctness against generated answers. Their association should not be read as a causal diagnosis of every error.

The graph retrieval takes the triples extracted from the retrieved turns and does not traverse the graph. Multi-hop graph walks were not implemented; in these graphs most edges run through the two speakers, so the edges adjacent to the retrieved facts' entities already cover most of the graph (Section 6.6.1). The reasoning experiment uses the two Gemma readers and one 768-token cap, so it does not establish cross-family reasoning behaviour. Its truncation rates show what happened under an on-device-feasible budget, not the maximum accuracy attainable with unbounded inference.

4.10 Summary

Under a 4,096-token history budget, vector retrieval has the highest LoCoMo accuracy point estimate for all five readers; the ordering also holds on the evaluated LongMemEval-S subset. Truncated full context often excludes the required evidence. Long-window full context, which holds the whole LoCoMo history, matches or exceeds vector retrieval on LoCoMo (separated for one of three readers) and stays behind it on LongMemEval-S without separating, at about 39 and 48 times the prompt tokens. Source restoration recovers part of the triple-only graph deficit, while the detailed graph controls in Chapter 6 distinguish extraction, selection and representation limits.

Increasing vector depth from two to fifty turns raises accuracy by 15–27 points and complete-evidence coverage from 16% to 64%, with an elevenfold increase in prompt length. Over the tested range this changes accuracy more than the reader doublings that were tested. Temporal and preference questions remain difficult.

Step-by-step reasoning gives inconsistent gains and uses roughly 40–60 times more completion tokens in the vector conditions. Between 13.3% and 34.5% of retrieval-condition outputs remain unfinished at the tested cap. Chapter 5 turns from these workstation results to measured phone resources.

Chapter 5

Resource-aware long-term memory on a smartphone

On the tested Galaxy S23 Ultra, Vector RAG had higher accuracy point estimates and lower measured energy per query than truncated full context in all five reader pairs. These separate measurements describe answer quality and device cost. Correct answers per kilojoule summarizes their joint effect: the observed Vector-RAG advantage was 5.8× to 26.9× for the four reader pairs with a non-zero full-context denominator, and these multipliers are uncertain with 20 questions per cell (Section 5.5). On this slice, Vector RAG answered every question that full context answered correctly, as well as additional questions. Across the five readers, it produced 40.8 to 68.8 correct answers per full-charge equivalent, against 0 to 8.9 for full context. Query service was also 6.8× to 8.2× faster. These values describe the tested phone workloads, not every mobile agent or question mix.

Contribution of this chapter. Among the studies reviewed in Section 2.5, MobileRAG reports per-query phone energy for document retrieval and SmartRAG reports average phone power for one graph system whose baselines ran off the phone. None compares memory representations of a conversation history by accuracy and energy both measured on one handset. This chapter makes that comparison for vector retrieval and truncated full context with five readers, using accuracy judged from the phone's own answers. It also measures, on the same handset and with the same energy method, what it costs to build a vector memory and an SLM-extracted graph memory for a conversation.

The result continues the strategy comparison in Chapter 4 but changes the unit of analysis. Chapter 4 asks which memory representation produces correct answers under controlled workstation conditions. This chapter asks what happens when the same broad choice must fit a phone's battery, memory and thermal limits. Chapter 7 later returns to graph-memory repair. Here, GraphRAG is limited to the direct phone construction evidence that actually exists.

The chapter addresses the following research question:

> **RQ2.** What trade-offs between answer quality and resource consumption arise when deploying long-term memory strategies on smartphones?

5.1 A resource-aware objective

A continuing mobile agent does more than issue isolated language-model calls. It stores observations, updates its memory and retrieves evidence while sharing a finite battery and thermal envelope with the rest of the phone. A memory design that is accurate but repeatedly fills the

context window can be too slow or expensive to use. Low energy is not sufficient either if the reader returns no correct answers.

Accuracy and energy per query are reported first as separate measures of answer quality and device cost. Correct answers per unit of energy, G_E in Equation 3.3, adds a combined measure: how many correct answers the measured energy buys. It is reported per kilojoule as the number of judge-scored correct phone answers divided by the voltage-weighted battery energy of the recorded workload, including energy spent on incorrect answers. The combined measure complements the separate results; a high value alone does not establish adequate accuracy or low query cost. Correct answers per full-charge equivalent, G_Q, are computed separately from the direct charge-counter delta.

This avoids mixing a voltage-weighted joule numerator with a nominal-voltage battery denominator. Neither measure predicts daily battery life because the screen, radios, background applications and the rest of the agent loop sit outside the measured span.

Section 2.5 explains why these costs must be measured for the tested configuration rather than inferred from token counts or from other devices [Lu et al., 2025, Xu et al., 2024].

5.2 Experimental boundary

Chapter 3 defines the phone protocol, voltage-weighted Coulomb counting, question selection and judging procedure. Inference ran on the CPU of the Galaxy S23 Ultra described there, with 12 GB of RAM and a 5,000 mAh battery. The local inference server used one slot, an 8,192-token slot context and five model threads. Workloads ran unplugged with the screen off.

Five standard Q4_K_M readers were evaluated: Qwen3.5 0.8B, 2B and 4B, and Gemma 4 E2B and E4B. Vector retrieval used the Q4_K_M Nomic Embed Text v1.5.

The primary comparison uses the same balanced 20-question LoCoMo slice under Vector RAG and truncated full context [Maharana et al., 2024]. It contains five questions from each of the four evaluated question families. Decoding was deterministic, and every accuracy in the phone tables was judged from a phone output. The energy unit is a complete sitting divided by its number of scored answers. Vector RAG has three complete energy sittings for every reader. Gemma E2B full context also has three, one of which completed 19 answers. The other full-context cells have one complete sitting. Appendix A gives the replicate values and uncertainty details.

The device audit searched the experiment locations accessible through ADB and Termux: shared storage, Termux's private results directory and the local mirror of pulled results. Protected Android sandboxes and restricted kernel interfaces were not readable without root. The evidence claim is therefore limited to accessible experimental storage.

5.3 The two statistical results used in this chapter

5.3.1 Workstation accuracy cannot substitute for phone accuracy

The phrase "the workstation result does not transfer to the phone" has a precise meaning in this thesis. It does not mean that the phone was proved less accurate. It means that the predefined agreement rule did not license the two environments as interchangeable.

The parity run used the same 100 question identifiers and the same E2B Vector-RAG configuration on the workstation and phone. Each answer was reduced to a paired binary correctness label. The two devices agreed on 83 of 100 labels, below the required 90% agreement. Cohen's κ was 0.621. Of the 17 disagreements, five questions were correct only on the workstation and 12 only on the phone. An exact McNemar test on those discordant pairs gave $p = 0.1435$.

These numbers answer different questions. Agreement measures whether the same questions receive the same labels. McNemar's test asks whether the disagreements consistently favor one device; this run did not establish a directional accuracy shift. The agreement criterion still failed. Workstation accuracy is therefore not joined to phone energy anywhere in the main result.

5.3.2 Question family changes the observed difficulty

The second result concerns the benchmark, not the device. The agreement rule forbids substituting workstation *accuracy* for phone accuracy; it does not prevent using the workstation record to characterize the *questions* both environments answer. That use assumes only that the relative difficulty of LoCoMo's question families carries across environments, which is weaker than the per-question label agreement the parity gate tested and failed. On the workstation sample, question family (the LoCoMo category) remained associated with correctness in all eight primary Gemma reader-memory cells after Holm correction (Section 4.2.4). Phone accuracy must therefore be tied to its sampled question composition. The balanced phone slice gives each family equal weight and should not be read as an estimate under LoCoMo's natural family frequencies.

5.4 Measured phone comparison

Table 5.1 presents the direct phone measurements.[1] Repeated cells use the mean across complete sittings. Single-sitting values are point estimates without a replication claim.

Reader	Memory	Accuracy	J/answer	Query time (s)	Sittings
Qwen3.5 0.8B	Vector RAG	20%	306	15.0	3
Qwen3.5 0.8B	Full context	0%	330	114.5	1
Qwen3.5 2B	Vector RAG	30%	433	34.8	3
Qwen3.5 2B	Full context	10%	850	262.0	1
Qwen3.5 4B	Vector RAG	35%	569	94.0	3
Qwen3.5 4B	Full context	10%	2166	641.9	1
Gemma 4 E2B	Vector RAG	50%	551	54.5	3
Gemma 4 E2B	Full context	5%	1395	420.7	3[a]
Gemma 4 E4B	Vector RAG	40%	737	93.0	3
Gemma 4 E4B	Full context	5%	2589	759.7	1

Table 5.1: Phone-measured accuracy, voltage-weighted energy and query latency on the balanced 20-question LoCoMo slice. Energy is the mean joules per answer across complete sittings where repeated measurements exist. Latency is the mean of the per-sitting medians. [a]One Gemma E2B full-context sitting completed 19 answers.

Vector RAG had a higher accuracy point estimate, lower energy per answer and shorter query time in every reader pair. For Qwen3.5 0.8B the energy difference is small, 306 against 330 J, and lies within the spread of that reader's three vector sittings (248 to 345 J); for the other four readers full context used 2 to 4 times more energy per answer. Reader size alone did not order the systems. Qwen3.5 0.8B used the least energy for a Vector-RAG answer but reached only 20% accuracy. Gemma E2B reached the highest measured accuracy and correct

[1]All 20-question phone Vector RAG runs in this chapter, including Figures 5.1 and 5.2, use the unprefixed embedding path. On the same 20 questions, the prefixed 100-question runs answered 8 (E2B) and 10 (E4B) correctly, against 10 and 8 here (Section 3.5).

answers per unit of energy, while the larger Gemma E4B used more energy and scored ten percentage points lower on this slice. The small sample does not support a general ranking of the readers. It does show why a phone policy cannot use parameter count as its only decision variable.

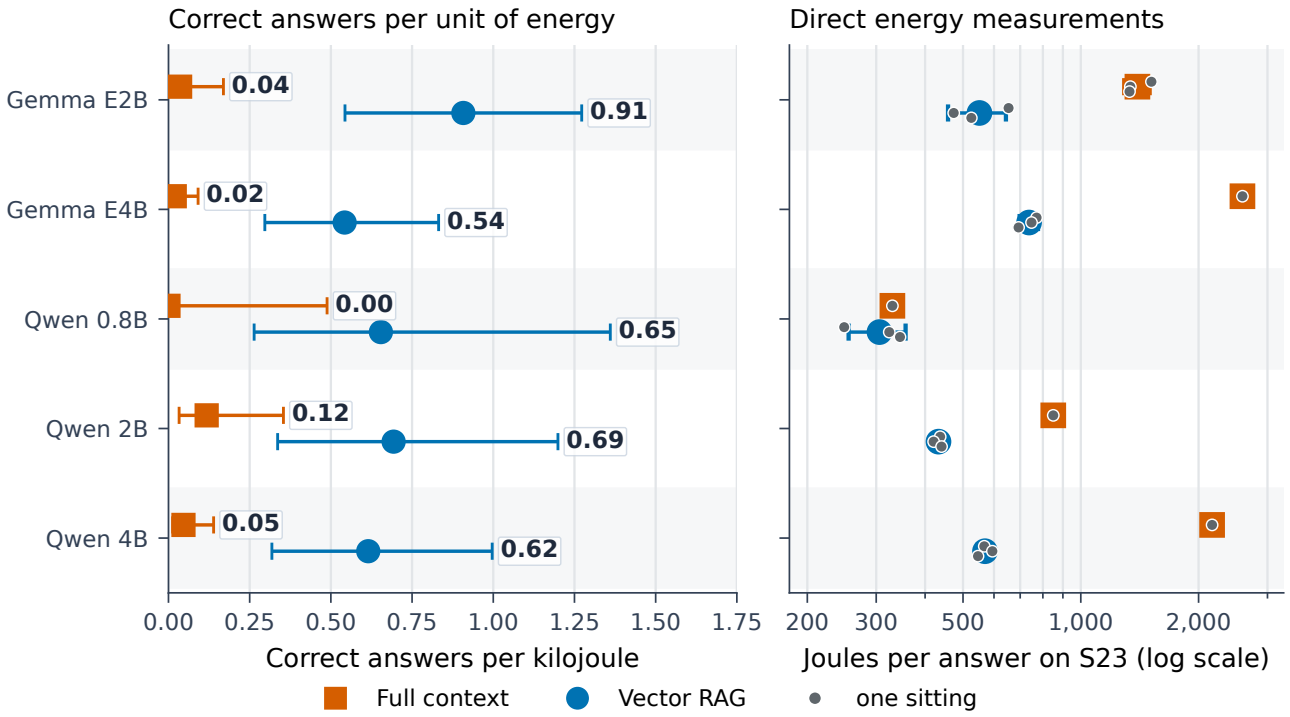


Figure 5.1: Correct answers per unit of energy and energy per answer across the five phone readers. The left panel divides phone-measured accuracy by replicate-mean energy; the right panel shows mean joules per answer, with each cell's individual sittings as grey dots above its marker. The Qwen3.5 0.8B full-context cell produced zero correct answers, so no finite Vector/full-context efficiency ratio is assigned to that pair.

5.5 Correct answers per charge

Table 5.2 converts the Vector-RAG charge-counter measurements into correct answers per 5,000 mAh full-charge equivalent. It also reports the observed within-sitting advantage over full context.

Reader	Vector correct/full charge	Paired Vector/FC efficiency
Qwen3.5 0.8B	50.2	Undefined[a]
Qwen3.5 2B	52.8	5.8×
Qwen3.5 4B	46.7	13.4×
Gemma 4 E2B	68.8	25.1×
Gemma 4 E4B	40.8	26.9×

Table 5.2: Correct answers per full-charge equivalent for Vector RAG and its observed advantage over full context. Vector values use mean charge per answer across complete sittings. The efficiency ratios are computed within matched sittings and are therefore not obtained by dividing the rounded means in Table 5.1. [a]Full context produced zero correct answers, so the ratio is undefined.

The range in the opening paragraph is now visible directly in Table 5.2: Gemma E4B is the 40.8 floor and Gemma E2B is the 68.8 ceiling. The smallest reader does not maximize correct answers per charge because its lower accuracy offsets part of its energy saving. The largest Gemma reader also does not maximize it because its extra energy did not buy a higher score on this slice.

The complete full-context sittings produced 8.9 correct answers per charge for Qwen3.5 2B, 3.3 for Qwen3.5 4B, 2.4 in the primary Gemma E2B sitting, 1.4 for Gemma E4B and zero for Qwen3.5 0.8B. These are workload-normalized comparisons, not a recommendation to drain a handset or a forecast of daily use.

Sensitivity of the efficiency ratio to small-sample denominators. The 25.1× and 26.9× ratios rest on very small denominators. On a 20-question slice, a 5.0% full-context accuracy is one correct answer (1/20), with a Wilson 95% confidence interval of $[0.9\%, 23.6\%]$. One more correct full-context answer ($2/20 = 10.0\%$) would have cut the E2B ratio from 25.1× to about 12.5×, and zero correct answers, as observed for Qwen 0.8B, leave the ratio undefined. Resampling the 20 observed question pairs with mean energy held fixed gives 2.5th-percentile ratios ranging from 2.6× to 8.8× across the four readers with defined ratios (Appendix A.3). These are conditional sensitivity results: they do not include between-sitting energy variation or charge-counter bias, and resampling cannot introduce full-context-only successes absent from the original sample. The observed efficiency ordering holds in all five reader pairs, but the multipliers describe this slice and should not be read as general constants.

5.6 Latency, RAM and thermal headroom

Figure 5.2 places accuracy and energy beside query latency, the peak temperature of the hottest thermal zone and peak system RAM.

Vector retrieval shortened the mean of the per-sitting median query times from 114.5 to 15.0 seconds for Qwen3.5 0.8B, from 262.0 to 34.8 seconds for Qwen3.5 2B, from 641.9 to 94.0 seconds for Qwen3.5 4B, from 420.7 to 54.5 seconds for Gemma E2B and from 759.7 to 93.0 seconds for Gemma E4B. Retrieval adds ingestion, but repeated queries avoid paying full-context prefill on every request.

Peak RAM did not follow the same ordering. Measured as system-wide memory in use (total minus available), the cell means lay between about 5.5 and 6.7 GB, with retrieval lower for some readers and higher for others. Model weights and the resident runtime occupy much of this footprint at the tested context size. The evidence supports reduced repeated context processing, not a claim that retrieval always reduces peak resident memory.

The displayed mean of per-sitting hottest-zone peaks was higher for Vector RAG in all five reader pairs, by 0.5 to 3.7 degrees Celsius, despite much shorter query times. Four comparisons combine a three-sitting Vector-RAG mean with one full-context sitting, so this 5-of-5 pattern is descriptive. It does not establish that retrieval inherently runs hotter, and the recorded traces do not explain it. Vector-RAG queries were short, with medians of 15 to 94 seconds, against about 2 to 13 minutes for full context, so a peak reading may reflect a short burst rather than cumulative thermal load. However, prefill and decode were not timed separately, governor decisions were not recorded, and the core-frequency medians discussed below show no consistent pattern between the strategies. Testing the burst account would require a controlled experiment with matched starting temperature and frequency-residency logging.

Mean power over a sitting separates the two strategies as well, and in the opposite direction to duration. Table 5.3 gives the whole-device average over each measured span. Vector retrieval drew more average power than full context in four of the five reader pairs, by 0.13 to 1.22 W,

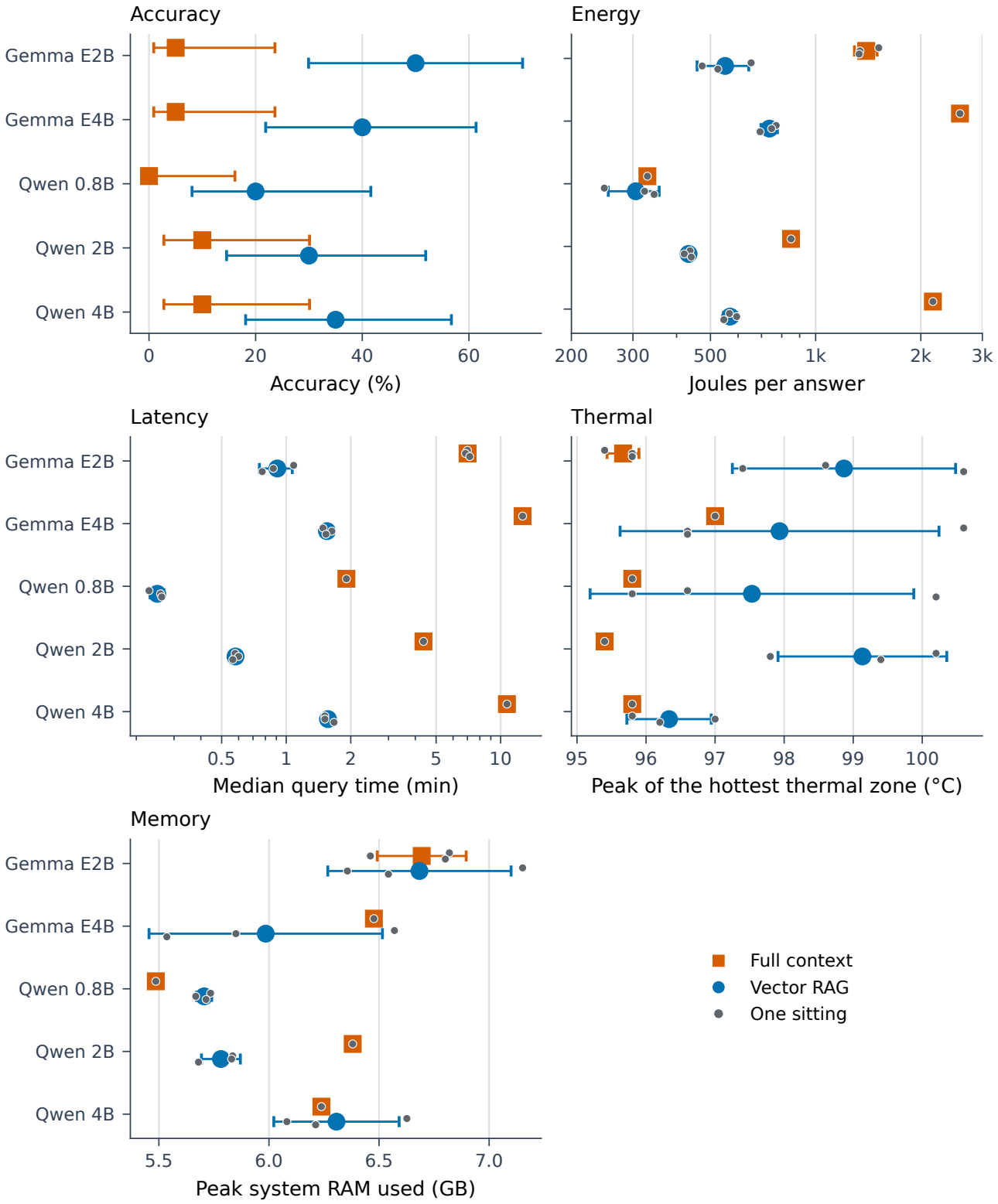


Figure 5.2: Phone workload comparison across five readers. Accuracy uses 20 balanced LoCoMo questions with Wilson 95% intervals. Resource whiskers show between-sitting standard deviations; grey dots are individual sittings. Vector retrieval has three sittings per reader; full context has three for E2B and one otherwise. Energy and latency axes are logarithmic; thermal and RAM axes do not start at zero. Resources summarize complete run spans. Temperature is the maximum across readable thermal zones, not battery temperature; RAM is system-wide. Neither peak isolates a model component.

and the same power to within 0.01 W in the fifth, while its median query took about one seventh as long.

Reader	Vector RAG (W)	Full context (W)
Qwen3.5 0.8B	2.88	2.89
Qwen3.5 2B	3.97	3.27
Qwen3.5 4B	3.44	3.31
Gemma 4 E2B	4.11	3.59
Gemma 4 E4B	4.71	3.49

Table 5.3: Mean whole-device power over each sitting's measured span, from the same charge-counter energy and span as the energy column of Table 5.1. One sitting per cell, not a replicate mean. The construction pilot of Section 5.7 averaged 3.96 W over its span.

Two explanations were considered for that difference. The first is the work of building the vector memory. Each vector sitting embeds six conversation histories, one before each history's questions, and these embedding windows took 22.8 to 31.7 minutes of a 29 to 55-minute sitting. They drew 3.4 to 5.1 W, more than the question stretches of the same sitting in four of the five readers (by 0.4 to 0.9 W; Qwen3.5 4B was level), so building the memory accounts for part of the difference. The question stretches alone drew 3.1 to 4.4 W, against 2.9 to 3.6 W over the whole full-context sittings: a gap of 0.9 W for Gemma E4B and 0.3 W or less for the other readers. Four of those full-context sittings started at the highest possible charge (100%), and in three of them the charge counter paused, which makes their power read low, so only the E4B gap is clearly larger than that effect. The second explanation is the duty cycle. The median of the maximum core frequency during the question stretches was 1,594 to 1,978 MHz in the vector sittings, against 1,843 MHz in every full-context sitting, so the clock data show no consistent pattern that would support it. The traces do not record governor decisions, so the remaining difference is not explained here. Vector stretches that began while the charge counter was paused at full charge are left out of these phase figures.

The recovered second Gemma E4B full-context sitting illustrates another device outcome: failure to complete. It produced 8 of 20 output records and no final summary during approximately 113 minutes. The battery level fell from 96% to 65% over the surviving trace. Because the boundary cannot be matched to the sealed cells, this run is excluded from the result tables. Its value is operational: checkpointing and recoverability are part of a phone memory system, not merely conveniences for the experimenter.

5.6.1 Prefill dominance and KV-cache size

The latency and memory measurements in the dashboard reflect the prefill–decode difference and the growth of the KV cache with context length, both introduced in Section 2.1. Figure 5.3 shows the measured query times of the Gemma 4 E2B phone runs (llama.cpp on the Snapdragon 8 Gen 2 CPU cluster, 5 pinned threads, CPU mask `0xf8`).

Figure 5.3 plots every answered E2B phone question against its prompt length. Full-context prompts of about 4,900 tokens took a median of 411 to 431 seconds per sitting. Ten retrieved turns (about 680 tokens) took 46 to 65 seconds, two retrieved turns (about 290 tokens) about 21 seconds, and the four graph-pilot questions (about 510 tokens) about 34 seconds. Query time follows prompt length: the median full-context prompt is 17 times as long as the median two-turn prompt (4,905 against 294 tokens) and took 20 times as long (418.5 against 20.8 seconds). The answers are short, a median of 10 tokens (8 to 21) across the 243 answers. The phone logs only the total generation time, so the time spent decoding the answer cannot be

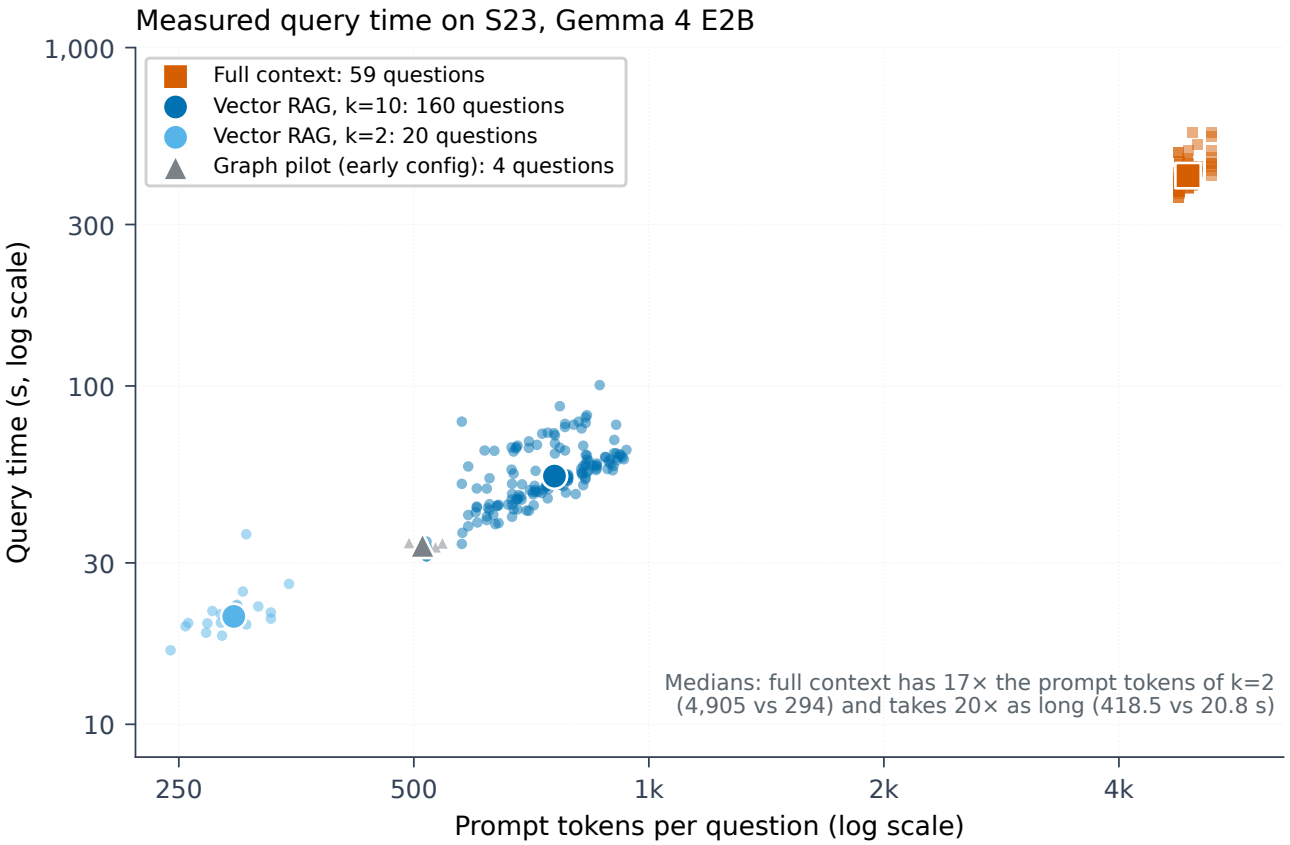


Figure 5.3: Measured query time (retrieval plus generation) of every answered Gemma-4 E2B question on the Samsung Galaxy S23 Ultra against its prompt length: three full-context sittings, four vector-retrieval sittings with ten retrieved turns (one of them 100 questions long), one sitting with two retrieved turns, and the four questions of the early graph pilot. Large markers are medians. The phone records only the total generation time of each question, so prefill and decode time are not separated.

separated in these runs; the near-proportional scaling indicates that processing the prompt sets the cost of a question. Measured per sitting, the phone processed 10.9 to 15.4 prompt tokens per second of query time, and 11.0 to 11.9 in the full-context sittings. Averaged over sittings, full context took 7.7 times as long per question as vector retrieval, 420.7 against 54.5 seconds (Section 5.6).

Memory was measured as system-wide RAM in use (total minus available memory), not per process. Peak use in the E2B sittings averaged about 6.7 GB under both strategies (Figure 5.2). The server reserves the key–value cache for its whole 8,192-token context when it loads, so the reservation was the same for every strategy: prompt length changed time, not memory. The size of that cache follows from the model files. Twenty of Gemma-4 E2B's 35 layers reuse the KV cache of earlier layers, and 12 of the 15 layers that keep their own cache use a 512-token sliding window, so only three global layers grow with context. Calculated from the GGUF header shapes, the E2B cache is 57 MB at 8,192 tokens and 0.81 GB at 131,072 tokens, against 4.4 GB still free at the highest measured E2B peak (Figure A.2). The layout behind that calculation is the published one: the Gemma 4 technical report states a four-to-one ratio of local to global attention blocks for E2B and five-to-one for E4B, and KV-cache sharing across 20 of 35 layers in E2B and 18 of 42 in E4B [Gemma Team, Google DeepMind, 2026], which are the counts used here. Its Table 3 also reports the footprint of an int8 cache at a 32,768-token context, 0.05 GB for E2B and 0.14 GB for E4B, about half of what the calculation above gives for the same contexts at half precision. The calculated figures are therefore treated as an upper bound on the cache; the comparison against free memory only becomes more favourable if the published footprints are the operative ones. Apart from a four-question pilot that doubled the history budget to 8,192 tokens and was not pursued further (Appendix B), no longer context was run on the phone and no long-context study was done on it; the long-window full-context runs of Section 4.6 were made on the workstation. The runtime's other buffers were not measured at longer contexts, so this comparison bounds the cache alone rather than showing that a longer context would fit. Bounded KV growth does not make a long context cheap either, because prefill time and energy still grow with every prompt token, as the query times show.

5.7 The measured graph-construction boundary

Graph memory remains preliminary on the phone. Figure 5.4 shows the one direct construction pilot. Building an SLM-extracted graph for 419 dialogue turns took 92.1 minutes and consumed 21.9 kJ under the same voltage-weighted integration used elsewhere in this chapter, or 52.3 J per turn. Fifty-eight of 84 extraction batches failed. The pilot then answered zero of four questions correctly. Four answers do not support a comparative accuracy estimate.

The timeline in Figure 5.4 shows how the handset behaved during extended extraction. The temperature trace is the hottest of all readable thermal zones. Its first sample, taken after the model servers had loaded, read 47.9°C, with the battery at 33.3°C. The hottest zone peaked at 95.4°C 74 seconds after launch, 29 seconds into extraction, and the Cortex-X3 prime core reached its 3.36 GHz maximum only within the first minute. For most of the 92.1-minute construction span the fastest core then ran at 1.59 GHz, and the hottest zone had a median of 74.6°C, with ten-minute medians between 73.4 and 77.8°C after the first ten minutes. The charge counter gives an average of 3.96 W over the span; with the clock held down, the heat input and with it the temperature stayed nearly constant. A steady clock ceiling under a CPU-bound workload is consistent with thermal or power throttling, but the trace does not record governor state, so it cannot confirm the cause. Battery temperature rose steadily from 33.3°C to 44.3°C. The two traces diverge throughout, which is why hottest-zone peaks are not reported as battery or chassis temperatures.

The construction cost is the most informative measurement from this pilot. Table 5.4 shows

how it is compared with building a vector memory. Each side is the whole-device battery energy spent while the phone built memory, measured with the same charge-counter method as the rest of this chapter, divided by the number of dialogue turns built. Graph construction is the Gemma 4 E2B reader writing triples for batches of five turns. Vector ingestion is the Nomic embedder embedding each turn once. Neither side includes answering questions.

	Graph construction	Vector ingestion
Work done per turn	Triple extraction by Gemma 4 E2B, five turns per call	One embedding by Nomic Embed v1.5
Measured runs	One pilot build	Three Gemma E2B vector sittings
Histories built	One conversation (`conv-26`, 419 turns)	Six per sitting; two windows left out[a]
Duration and mean power	92.1 min at 3.96 W	31.7 min at 4.29 W in the first sitting
Energy per dialogue turn	52.3 J	2.04 J (1.85 to 2.37 by sitting)
Ratio	About 26× (22× to 28× against each vector sitting)	

Table 5.4: How the construction ratio is computed. The graph column is the single early phone build (256-token extraction limit, 58 of 84 batches failed), not a usable repaired graph, and the ratio applies to that build only. Both columns are direct phone measurements over the build windows only (`results/golden_metric.json`, `construction` entry; duration and power from `results/phone_power_phases.json`). [a]Two embedding windows began while the charge counter was paused at full charge and are left out.

The two builds drew a similar average power, so the ratio comes almost entirely from time. Writing triples took about 13 seconds per dialogue turn (92.1 minutes for 419 turns), while embedding took about half a second per turn in the first vector sitting (31.7 minutes for 3,435 turns). Neither side depends on judged accuracy, on token counts or on the workstation record.

The ratio describes this one attempted build, not the cost of constructing a usable graph, and three limits apply to it. The graph side is a single build of one conversation, so it has no replicate spread; the 22–28× range reflects only the variation between vector sittings. The build used the early 256-token extraction limit, and 58 of its 84 batches failed; failed batches still spent their decoding energy, so they are included in the cost, but they contributed no triples. Finally, the repaired 512-token configuration of Chapter 6 was not built on the phone, so its construction energy was not measured.

The asymmetry matters because a memory policy that inspects only query prompts cannot observe either term: construction is paid before the first question is asked.

The pilot supports one narrow conclusion: a memory policy must account for construction as well as query service, as in the accounting of Equation 2.3. It does not support a phone routing comparison among graph, vector and full-context memory because no complete phone GraphRAG QA cell exists. Figure 5.5 places the build time of this directly measured 419-turn phone build (21.9 kJ, 52.3 J/turn) beside workstation build times across dialogue lengths, which grow linearly with the number of turns. The figure shows time only; the energy values are the ones given here.

The earlier token-routing grid is therefore removed from the main argument. Appendix A records why it is exploratory and keeps the long-context and amortization calculations separate from direct measurements.

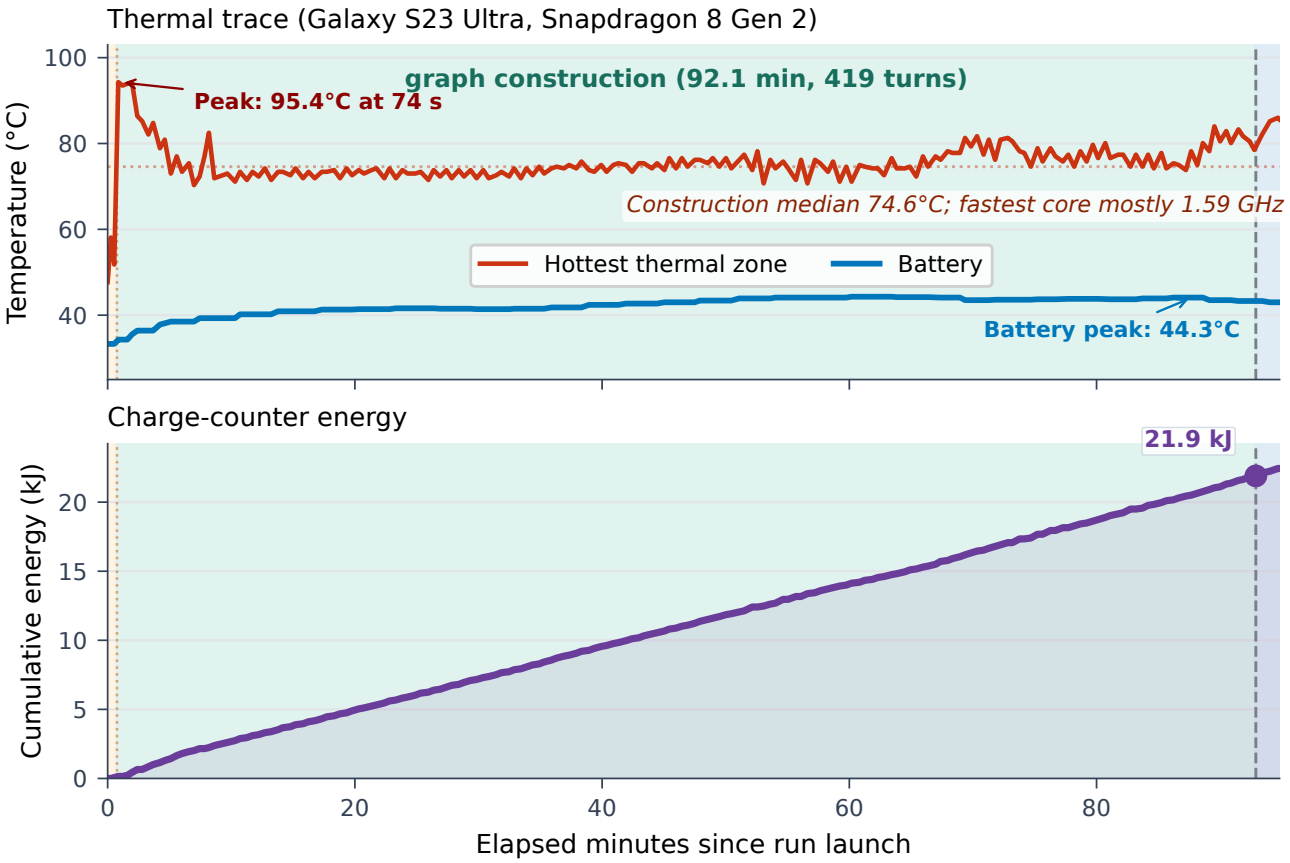


Figure 5.4: Early graph-construction pilot on the Galaxy S23 Ultra: 419 turns, 92.1 minutes and 21.9 kJ (52.3 J/turn), with 58 of 84 extraction batches failing. The red trace is the hottest readable thermal zone, not battery temperature; sampling begins after server loading. Core frequencies and charge use describe the recorded workload. The four subsequent answers do not constitute a comparative graph-QA evaluation.

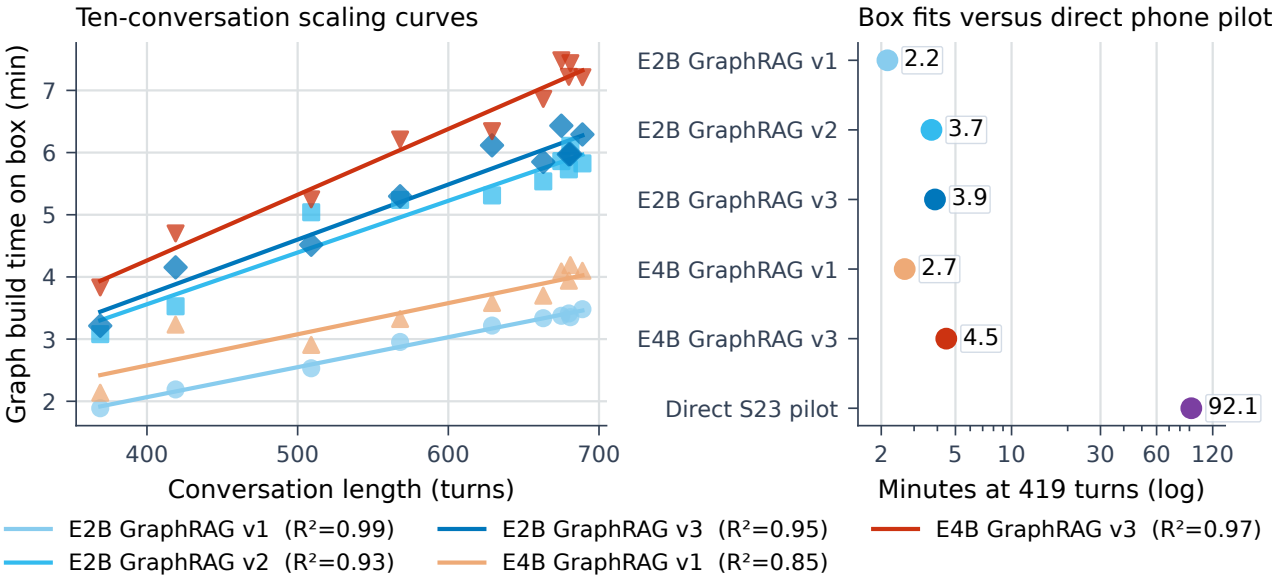


Figure 5.5: Graph construction time. Left: 50 workstation builds across ten LoCoMo histories and five Gemma extractor configurations (E2B v1–v3; E4B v1 and v3), with a linear fit per configuration. Right: fitted workstation times at 419 turns and the directly measured 92.1-minute phone build, on a logarithmic axis. The early phone pilot differs from the repaired workstation configurations.

5.8 Position relative to mobile systems literature

Section 2.5 reviews the mobile studies closest to this chapter and the gap they leave. Two of them report measurements that bear directly on the results above. SmartRAG reports average power of 2.37 to 4.77 W during triplet extraction and 2.12 to 5.36 W during question answering on its two OnePlus handsets, but not energy per answer [Jiang et al., 2026]. It also finds that retrieval-conditioned prefill dominates latency: on a Snapdragon 8 Elite Gen 5 with the 1.7B backbone, median time to first token was 1.32 s for question decomposition but 36.9 s for answering, consistent with the prompt-driven query times observed here in Section 5.6.1. MobileRAG reports real-phone energy for document RAG on a Galaxy S24, from 24.71 J per query to more than 200 J in some configurations [Park et al., 2025]. In both cases the handset, model, corpus, retrieval work and measurement span differ, so their numbers are not pooled with the measurements in this chapter.

5.9 Implications for resource-aware long-term memory

The measurements support four design choices for a phone-native agent.

1. **Optimize correct answers per unit of energy rather than one counter.** Energy, latency and accuracy changed together. Correct answers per kilojoule and per charge reveal cases in which the lowest-energy reader is not the most useful one.

2. **Use bounded retrieval as the default path for the tested workload.** Vector RAG occupied a better measured accuracy, energy and latency point than full context in all five reader pairs. The conclusion is limited to this retriever, device and balanced question slice.

3. **Keep a lifecycle ledger.** Embedding, graph extraction, prefill, decode and persistent storage are different operations. Prompt tokens describe reader input but omit memory construction entirely, and on this phone construction is where the strategies separate most sharply: the early graph build cost about 26 times more energy per turn than embedding (Table 5.4). A ledger that prices only the query path misses that difference before the first question is served.

4. **Treat completion and provenance as system properties.** A run that consumes battery but cannot be resumed has delivered no sealed work. Checkpoints, model digests, runtime identity and explicit measurement spans are necessary if a memory policy is to learn from device telemetry.

On a phone, "long term" does not mean only a larger store or a longer prompt. It means recovering useful history across repeated interactions without exhausting the resources that allow the agent to continue operating.

5.10 Experimental limitations

One device and runtime. The measurements come from one Galaxy S23 Ultra, one Android and kernel build and a CPU-oriented llama.cpp configuration. A newer SoC, accelerator support, another thread count or another operating-system governor may change the ordering.

Small phone sample and uneven replication. Each accuracy cell contains 20 balanced questions. Question family is associated with correctness in the larger workstation sample, but the phone slice is too small for family-specific phone claims. Vector energy has three sittings for every reader, while full-context replication is complete only for Gemma E2B. Appendix A reports this asymmetry instead of hiding it in main-text error bars.

Short-workload overhead. In the 20-question Gemma E2B vector sittings, building the memory for six histories took about 60% of the energy (Appendix A.4). A separate 100-question E2B Vector-RAG run, which spread ten histories over 100 questions, used less energy per answer, but it used the task prefixes and its question composition and execution time differ. It is evidence that the number of questions per history matters, not a matched estimate of a universal amortization factor.

Automated judging. Phone answers are scored by the same automatic judge as the workstation answers, with the limits stated in Section 3.6.

Incomplete and inaccessible evidence. The incomplete E4B sitting is excluded from the sealed tables. The audit covered all experimental locations accessible through ADB and Termux, but not protected Android sandboxes or root-only kernel interfaces.

Graph evidence. Only one early phone graph construction pilot was measured, and no complete phone graph QA cell exists. The main chapter makes no graph-routing or graph-accuracy claim.

5.11 Chapter conclusion

The phone results support a direct systems conclusion. Vector retrieval produced higher accuracy point estimates, lower energy per answer and much shorter query times than full context under all five readers. It delivered 40.8 to 68.8 correct answers per full-charge equivalent, and its finite observed advantage over full context ranged from 5.8× to 26.9× correct answers per kilojoule. This combined measure summarizes the separate quality and energy results.

Two statistical results set the limits of that claim. The workstation-to-phone parity gate failed, so phone accuracy must be measured on the phone. Question family was also associated with correctness in every primary Gemma cell, so the balanced phone result must remain attached to its question mix. RAM and thermal peaks do not reduce to the same ordering as energy and latency, and the graph pilot shows that construction can be a large part of the memory lifecycle. A resource-aware memory policy therefore needs these device measurements before it can route between strategies.

Chapter 6

Repairing the graph memory pipeline: what improved and what remained

On 200 LoCoMo questions, the initial Gemma 4 E2B graph baseline scored 15.0%, against 36.0% for vector retrieval.[1] We investigated whether the graph deficit came from implementation problems or from the evidence supplied by the graph. A repair sequence changed retrieval, increased the extraction budget and restored source dialogue. Further controls varied graph depth and separated the extractor from the reader.

The repairs improved accuracy, with the largest single gain following source restoration. This chapter examines where evidence was lost and which repairs recovered it, motivating the budgeted source-selection policy evaluated in Chapter 7.

Contribution of this chapter. Han et al. [2026] compare triples, triples with source text and plain retrieval using larger models, and Kelesis et al. [2026] show that small-model extraction lowers graph-reasoning accuracy. This chapter adds a repair sequence that changes one stage of a small-model graph pipeline at a time, on the same 200 questions. It uses that sequence to locate where evidence is lost: in extraction, in selection, in the triple representation or in the reader. The individual repairs are not new. Using a graph together with passage retrieval has precedents (Section 6.1), and attaching source text to triples is one of the conditions compared by Han et al. [2026]. The contribution is applying them one at a time to a small-model pipeline, so that each loss of evidence can be assigned to one stage. Two controls complete the sequence. An extraction-budget control shows a floor below which retrieving more facts does not recover answers, and a reader–extractor crossover separates the model that builds the graph from the model that answers.

6.1 What the graph baseline represents

The graph arm is the SLM-extracted dialogue pipeline defined in Chapter 3, not an unchanged Microsoft GraphRAG installation. It stores extracted triples as directed edges, merging repeated relations. Its results depend both on which facts survive construction and on how evidence reaches the reader. Graph-guided retrieval and presenting triples as evidence are distinct choices: HippoRAG, for example, uses a graph to rank source passages [Jiménez Gutiérrez et al., 2024]. The repairs below test these choices within our pipeline.

[1] The earlier unprefixed vector-retrieval runs scored 32.5%.

6.2 The repair sequence

The versions are called GraphRAG v1 to v4 because each repair changes one stage of the whole retrieval pipeline (construction, selection or reader input), not the form of the graph. Every version stores (subject, relation, object) edges in a directed multigraph. Table 3.2 lists what each version changes. Version 2 changes how triples are selected, and at the same time stores source links and standardizes names. Version 3 raises the extraction budget, which rebuilds the graph with more triples. Version 4 keeps the graph and the selected triples of version 3 and changes only the text given to the reader.

Version 1 was the original implementation. Each later version addresses the failure that remained visible in the one before it: selection that depended on exact name matches (v2), cut-off extraction output (v3) and missing detail in the triples (v4). Source restoration therefore came last. While selection and extraction still lost evidence, its effect could not have been separated from theirs.

The main comparisons use 200 category-balanced LoCoMo questions under the workstation protocol of Chapter 3; earlier 100-question runs guided development. Version 1 performs no embedding; the other conditions follow the reporting convention in Section 3.5.

6.2.1 GraphRAG v1: entity-match retrieval

The original implementation extracted triples from batches of up to five dialogue turns. Each extraction call had a 256-token output limit. At question time, the system searched for graph nodes whose names matched terms in the question, then collected relationships within one link of the matching nodes. It ranked those relationships using term matches and simple rules for dates and session information, and supplied up to 12 triples to the reader.

This baseline could fail during extraction or during selection. Failed extraction removed facts before retrieval, while name matching and score ties could exclude useful stored relationships. The following case illustrates both problems.

A traced retrieval failure

Figure 6.1 traces "What did Caroline research?" on LoCoMo `conv-26`, using the early phone graph with a 256-token extraction limit and v1 one-hop retrieval. That graph has 148 nodes and 154 edges; the repaired workstation *v3* graph of the same conversation has 537 nodes and 551 edges. Phone extraction failed on 58 of 84 batches, compared with 9 of 1,179 across all ten workstation E2B v3 graphs. Speaker-incident edges account for 85% of this phone graph and 81% of this single workstation v3 graph. The ten-history v3 aggregate is reported separately in Section 6.6.1; the phone pilot is not the graph behind the workstation accuracy grid.

The labelled evidence turn says "Researching adoption agencies", but no extracted relation records that fact. Retrieval matched the speaker hub and tied 72 of its 77 candidate facts. The recorded prompt contained nine exchanges between the speakers among its twelve facts, omitted the tied adoption fact, and produced the wrong answer. The figure traces each stage; adding the adoption fact would not itself restore the missing research relation.

The tie order follows Python's per-process hash seed. Repeating the query under 50 seeds kept the adoption triple in 31 runs and reproduced the pilot's twelve facts in 19. In the workstation v1 runs, 164 of 200 questions per Gemma reader had ties across the 12-fact cutoff, including 138 shared cases. Their reported v1 accuracies are single draws from that variation. Vector-first retrieval from v2 onward returned identical contexts under three tested seeds. Workstation vector retrieval also missed this question's evidence at every tested k from 2 to 50; this case does not demonstrate that vector retrieval would answer it correctly.

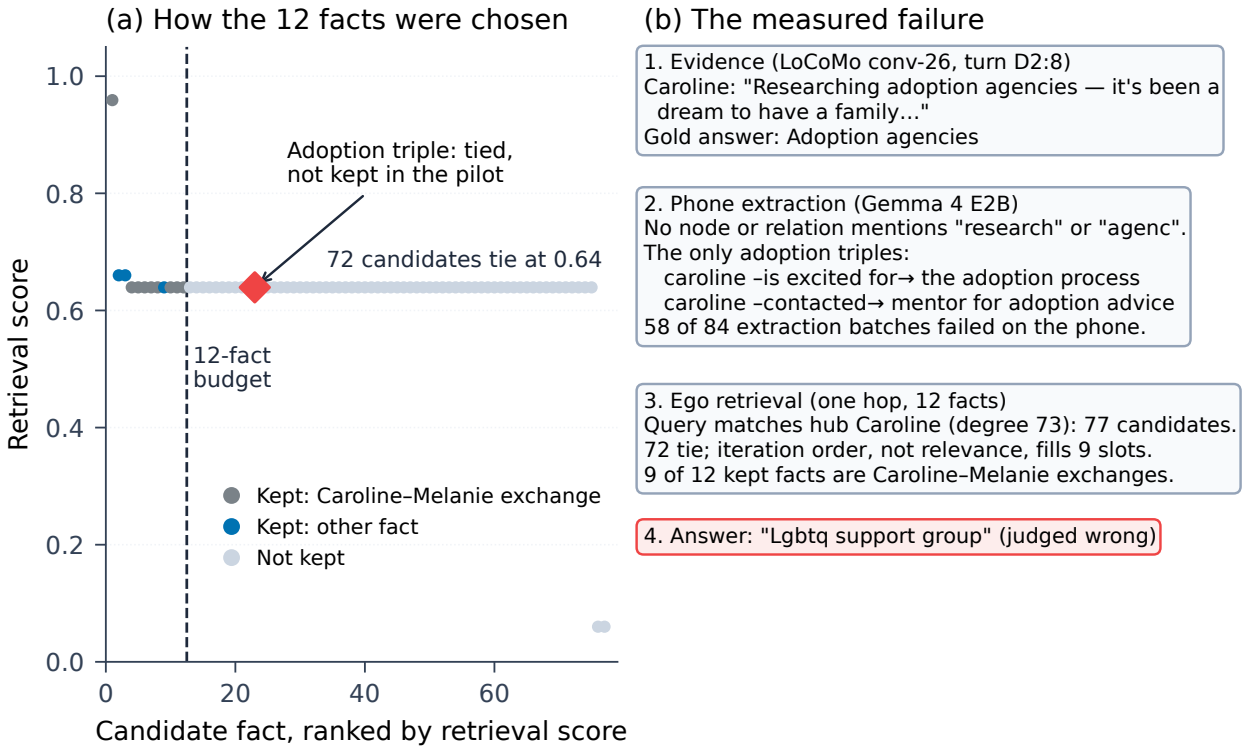


Figure 6.1: Retrieval failure with the v1 one-hop rule in the early Gemma 4 E2B phone graph for LoCoMo `conv-26`. (A) Scores of 77 one-hop candidate facts; colours identify the 12 facts included in the recorded prompt. Seventy-two candidates tie, making selection depend on iteration order. (B) The path from omitted evidence to the incorrect answer. Workstation vector retrieval also misses the evidence turn, so this case does not demonstrate vector superiority.

6.2.2 GraphRAG v2: turn-first retrieval

Version 2 began with the same kind of vector search used by the vector-retrieval baseline. The system embedded each dialogue turn, selected the ten turns most similar to the question, and considered graph edges whose recorded source batches overlapped those turns. It ranked candidate edges by the similarity of their best matching turn, multiplied by the edge's stored weight, and kept up to 12.

This change reduced dependence on matching the question to the exact wording of graph nodes. It also stored links back to the dialogue and standardized some names, for example by treating "mom" and "mother" alike. Because these changes were introduced together, version 2 is a combined retrieval and naming repair. Its accuracy change cannot be assigned to vector search alone.

The source links have a further limitation. Each edge records the range of turns in the batch from which it was extracted. It does not record which individual sentence supports that edge. A selected edge can therefore be associated with a retrieved turn from the same batch even when that turn expresses a different fact. This detail affects both source restoration in version 4 and the later SPE design.

Version 2 still showed only triples to the answer model. On the 200 E2B questions, accuracy rose from 15.0% to 18.5%. The extraction counters remained at 507 failed batches out of 1,179. Better selection could only work with the facts that had entered the graph.

6.2.3 GraphRAG v3: larger extraction budget

The failure logs included structured responses cut off before a triple or the enclosing object was complete. An output limit of 256 tokens could be too short for the relationships in five dialogue turns. Version 3 retained version 2's retrieval method and doubled the extraction limit to 512 tokens.

For E2B, failed batches fell from 507 to 9 out of 1,179, or from 43.0% to 0.8%. The number of extracted triples rose from 3,531 to 7,699. Accuracy moved from 18.5% to 21.5%, a modest difference that did not separate statistically. The clear effect of version 3 was on graph construction, not on the answer score.

The large reduction in failed batches establishes that the output budget was a practical problem for this extractor. It does not establish that every original failure had the same cause. The original version 2 log contains 504 JSON decoding errors, where the response could not be read as a structured object, and three type errors caused by fields with the wrong value type. The version 3 log contains four JSON decoding errors and five type errors. These counters record whether the extraction pipeline could process a response, not whether every resulting fact was correct or complete.

6.2.4 GraphRAG v4: source turns restored

After version 3, E2B could build a substantially larger graph with few failed batches, yet its answer accuracy remained below vector retrieval. Version 4 tested whether useful details were missing from the representation given to the reader. It kept the same graph and selected triples as version 3, then appended retrieved dialogue turns whose indices overlapped the selected edges' source batches. Each source turn was included once.

This comparison changes the text available for answering without changing the graph contents or changing which triples are selected. It can test whether the attached text helps. Because the source links are recorded by batch, it cannot establish that each added detail was a faithful expansion of one particular selected triple.

The reported runs use the corrected graph loader; an earlier attempt that failed to apply source expansion was excluded (Appendix B.4).

6.3 How much the repairs recovered

Figure 6.2 shows the E2B sequence on 200 questions. Accuracy increased from 15.0% in version 1 to 33.0% in version 4. The largest individual increase, 11.5 percentage points, followed source restoration. Mean prompt length stayed near 520 tokens through version 3 and rose to 813 tokens in version 4. Vector retrieval scored 36.0% with a mean prompt length of 824 tokens.

Source restoration helped four of five readers (Table 6.1). For E2B, 27 answers changed from wrong to correct and four from correct to wrong, giving an 11.5-point gain that survives the designated Holm correction ($p = 0.00017$). The E4B gain is five points (exploratory, raw $p = 0.0129$).

Qwen 4B gained 8.5 percentage points, whereas Qwen 0.8B remained weak and Qwen 2B was essentially unchanged. The latter's extraction failed on 84.6% of batches (Table 6.5), limiting the evidence available for restoration.

By question category, source restoration mainly helps single-hop questions: it fixes 13 and breaks one for E2B, and fixes seven and breaks none for E4B. These category comparisons are exploratory.

No source-restored point estimate reached vector retrieval. E2B's remaining three-point gap did not separate statistically; E4B retained an eight-point gap (exploratory $p = 0.0070$).

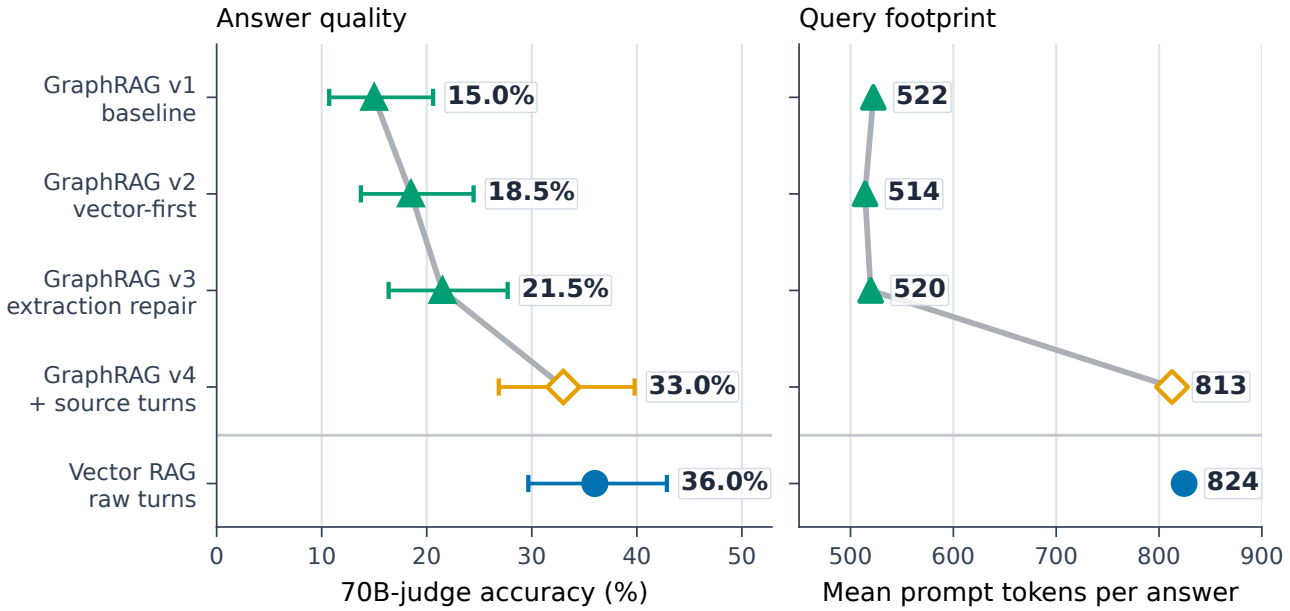


Figure 6.2: E2B repair sequence on the same 200 LoCoMo questions, using the record runs. Version 1 retrieves by matching names and embeds nothing; versions 2 to 4 and vector retrieval use the corrected task prefixes. The accuracy intervals are Wilson 95% intervals. Prompt tokens are the measured whole-prompt counts on the workstation. Table 6.1 extends versions 3 and 4 and vector retrieval to the other four readers.

Model	v3 triples	v4 triples + text	Vector retrieval	v4 gain
Gemma 4 E2B	21.5%	33.0%	36.0%	+11.5
Gemma 4 E4B	27.5%	32.5%	40.5%	+5.0
Qwen3.5 0.8B	6.0%	9.0%	22.0%	+3.0
Qwen3.5 2B	9.5%	9.0%	32.5%	-0.5
Qwen3.5 4B	23.5%	32.0%	39.0%	+8.5

Table 6.1: Primary workstation results with corrected embedding prefixes, 200 paired LoCoMo questions per condition. Versions 3 and 4 use a 512-token extraction limit and allow up to 12 selected triples. The final column is the change from v3 to v4 in percentage points. Qwen rows are descriptive checks in a second model family.

E2B mean prompt length rose from 519.6 to 812.7 tokens, close to vector retrieval's 824.4.

6.4 What reached the answer model

The saved prompts allow a more direct inspection than the accuracy totals alone. Consider the LoCoMo question "What martial arts has John done?" The reference answer names kickboxing and taekwondo. Version 3 supplied 12 triples, none of which named either sport, and E2B replied "No information available." One selected triple referred only to a question about a martial arts place or yoga studio.

Version 4 supplied the same 12 triples and added five dialogue turns. Two contained John's statements "I'm doing kickboxing" and "I'm off to do some taekwondo!" E2B then answered "kickboxing and taekwondo," which the judge accepted. This example shows information absent from the selected triple text becoming available through source restoration. It also shows why batch-level source links must be described carefully: useful text can accompany a triple without expressing that triple's particular relationship. The example illustrates a mechanism; it does not estimate how often that mechanism explains the full set of errors.

We also checked whether the prompts contained dialogue turns identified as evidence by LoCoMo. This analysis covered 199 questions with usable evidence labels; one question was excluded from this diagnostic, while all 200 remained in the accuracy calculation. Table 6.2 reports the proportion of questions for which at least one labelled evidence turn appeared in the prompt. It does not measure whether all the evidence needed for an answer was present.

E2B condition	At least one evidence turn	Reference answer string
v3 triples	1/199 (0.5%)	33/199 (16.6%)
v4 triples + text	97/199 (48.7%)	40/199 (20.1%)
Vector retrieval	127/199 (63.8%)	42/199 (21.1%)

Table 6.2: Prompt checks for the primary E2B conditions. The first diagnostic searches for the text of dataset-labelled evidence turns. The second searches the normalized prompt for the reference answer string. Both are text-matching checks, not semantic assessments of whether the prompt is sufficient to answer.

The near-zero evidence-turn match for version 3 is expected because triples reword the dialogue. It does not mean the graph contains no useful facts. The answer-string check is also limited: it can miss paraphrases and differently written dates, and it searches the whole prompt, including the question. We therefore use these measures as supporting diagnostics. The paired source-restoration experiment and the inspected example provide stronger evidence that useful detail was absent from the text supplied by version 3.

All five readers' graph and vector conditions start from the same ten retrieved turns. For E2B, these contain labelled evidence on 127 questions, of which the twelve-triple selection retains evidence-linked triples on 97. On those 97 questions, v3 answers 26 correctly, while v4 and vector retrieval both answer 44 correctly. Within the 127-question subset, the remaining gap is in the 30 questions whose evidence the graph selection drops: v3, v4 and vector retrieval answer 4, 6 and 10 correctly. Separately, on the 72 questions with labelled evidence that retrieval misses, v4 answers 15 correctly and vector retrieval 17. These group counts locate the observed shortfall; they do not isolate the causes of errors.

When complete labelled evidence is present, the two methods have similar conditional accuracy, without a consistent advantage across readers (Table 6.3). The groups contain different questions, so the comparison is descriptive.

	Graph + sources		Vector retrieval	
Reader	Complete cases	Accuracy	Complete cases	Accuracy
Gemma E2B	67	56.7%	88	50.0%
Gemma E4B	67	53.7%	88	56.8%

Table 6.3: Accuracy when the complete labelled evidence is present. The case counts differ because graph source restoration and vector retrieval do not cover the same number of questions.

6.5 Would more triples be enough?

The default limit of 12 triples was another possible constraint. We therefore reused the version 3 graphs and varied the maximum number of triples placed in the answer prompt. The main extraction settings stayed fixed. Table 6.4 shows the results with corrected embedding prefixes.

	4 triples		12 triples		24 triples		Vector retrieval	
Reader	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens
Gemma E2B	13.0%	287	21.5%	520	29.5%	870	36.0%	824
Gemma E4B	16.0%	295	27.5%	539	29.5%	909	40.5%	824
Qwen 4B	11.5%	283	23.5%	513	29.0%	864	39.0%	812
Qwen 0.8B	n/a	n/a	6.0%	—	7.0%	—	22.0%	—
Qwen 2B	n/a	n/a	9.5%	—	10.0%	—	32.5%	—

Table 6.4: Graph-depth controls on 200 paired LoCoMo questions. Each reader's v3 graph is fixed; only the maximum number of retrieved triples changes. Tokens are mean whole-prompt counts. Vector retrieval is the reference from Chapter 4. The four-triple condition was not run for the two smallest Qwen models (n/a); dashes indicate token means omitted from this table.

Increasing the limit from twelve to twenty-four triples gives E2B 18 improvements and two regressions, an eight-point gain (exploratory $p = 0.00040$). Its mean prompt grows from 519.6 to 870.2 tokens, yet its 29.5% accuracy remains below source restoration's 33.0% at 812.7 tokens. This is a descriptive comparison between different evidence sets. E4B gains two points and Qwen 4B gains 5.5; the smaller Qwen models change by only one or two answers (raw $p = 0.5$ for 0.8B and $p = 1$ for 2B).

Reducing the limit from twelve to four costs E2B 23 correct answers while gaining six (raw $p = 0.002316$). E4B loses 28 and gains five; Qwen 4B loses 26 and gains two. With losses at four and gains at twenty-four for E2B and Qwen 4B, twelve triples does not saturate observed accuracy for those readers. None of the tested limits for these three readers reaches the accuracy of source restoration.

The answer-cap and reasoning controls show no consistent recovery from additional output tokens under the tested settings (Section 4.7).

6.6 Extraction quality and model choice

The Gemma repair did not establish that a 512-token limit would suit every model. Table 6.5 compares extraction failures and output volume across the five models, including the 1,024-token controls for Qwen 0.8B and 2B. All conditions processed the same 1,179 batches from the ten LoCoMo conversations.

Doubling the Qwen 2B extraction allowance nearly tripled its output, from 1,925 to 5,683 triples. The number of stored graph edges rose from 1,853 to 5,323. Accuracy changed from

Extractor	Output limit	Failed batches	Extracted triples	Stored edges	QA accuracy
Gemma 4 E2B	512	9 (0.8%)	7,699	7,458	21.5%
Gemma 4 E4B	512	7 (0.6%)	6,264	6,228	27.5%
Qwen3.5 4B	512	45 (3.8%)	7,699	7,292	23.5%
Qwen3.5 0.8B	512	931 (79.0%)	1,898	1,678	6.0%
Qwen3.5 0.8B	1,024	889 (75.4%)	2,742	2,427	6.5%
Qwen3.5 2B	512	998 (84.6%)	1,925	1,853	9.5%
Qwen3.5 2B	1,024	802 (68.0%)	5,683	5,323	11.0%

Table 6.5: Extraction-budget controls over 1,179 LoCoMo batches. Output limits are tokens per call. Failed batches cannot be processed; valid empty returns are separate. Triples are counted before repeated edges merge. E2B and Qwen 4B happen to produce the same number of triples; their stored edges differ. QA uses v3 retrieval, corrected prefixes, and the same model for extraction and answering. No transport errors were recorded in the inspected logs.

9.5% to 11.0%, a net gain of three answers, which did not separate statistically and is too small to interpret as a recovery. Qwen 0.8B rose only from 6.0% to 6.5% with the larger allowance. Neither result reproduced the near-elimination of failed batches seen with Gemma E2B.

These controls show that extraction volume alone was insufficient to recover answer quality in the tested small Qwen configurations. They do not prove that extraction quality is unimportant. Many batches still failed at 1,024 tokens, and we did not measure the correctness or completeness of every surviving triple. Within Qwen, the 4B condition was the first tested size with a low failure rate at the 512-token setting. This is a boundary observed for this prompt, budget, model family, and runtime, rather than a universal model-size threshold for graph construction.

6.6.1 Structure of the repaired Gemma graphs

Table 6.6 describes the workstation E2B and E4B *v3* caches across ten LoCoMo conversations. These repaired graphs also supply the SPE evaluation.

Extractor	Nodes ($\|V\|$)	Edges ($\|E\|$)	Degree ($\bar{k}$)	Components	Giant %	Empty %	Extraction failure %
E2B	676.6 ± 140.1	745.8 ± 151.0	2.21	36.5 ± 12.2	86.1%	0.00%	0.76%
E4B	588.2 ± 108.0	622.8 ± 115.2	2.12	24.3 ± 8.6	89.7%	7.12%	0.59%

Table 6.6: Properties of the repaired Gemma 4 v3 graphs used in the SPE evaluation across 10 LoCoMo conversations. Nodes, edges and components are per-conversation means ± standard deviations. Degree is $2|E|/|V|$ counting every stored edge; read as a simple undirected graph, the mean degree is 1.96 (E2B) and 1.97 (E4B). Empty % and Extraction failure % are pooled over all 1,179 extraction batches.

Across the ten repaired workstation E2B v3 graphs, the mean share of edges touching either speaker is 83%. For each question, `graph_audit.py` takes the entities of all triples whose source batches overlap the ten retrieved turns, then counts graph edges incident to those entities (one step). This edge share averages 87% across questions. Excluding speakers as stepping stones, the first step adds under four percentage points to the seed triples' edge share and the second adds less than one. These observations describe graph reachability; a traversal policy designed for this structure was not tested.

E4B has fewer disconnected components and a larger giant-component share than E2B. Only 1.7% of E2B edges are reinforced across multiple dialogue turns, so repeated relations

contribute little consolidation in this representation. These measurements describe the graph inputs; they do not establish why one extractor answers better.

6.6.2 Empty responses and discarded values

The original E4B v3 extraction returned 84 empty batches out of 1,179 (7.12%), against none for E2B. Because the original counter did not retain those responses, all 1,179 E4B calls were replayed with instrumentation. Total extraction volume reproduced within 0.6%; all 96 replayed empty batches were explicit `{"triples": []}` returns. The count difference is treated as runtime variation.

Replacing each original empty batch with E4B's mean non-empty yield would recover about 484 of the 1,435-triple difference from E2B, roughly one third. E4B also produces fewer triples in non-empty batches. Inspection found both appropriate omissions of social exchanges and missed facts such as named pets, purchases and dated activities. Empty returns therefore do not establish better precision, nor does this audit estimate extraction recall or explain downstream accuracy.

The replay also found that a rule removing list numbers discarded three bare-numeric triples out of 6,231 returned triples. It caused none of the empty batches. The stored graphs were not rebuilt with a revised normalizer, so this diagnostic bounds the defect's observed effect in the replay rather than establishing that all implementation errors were removed.

6.6.3 Separate the model that builds the graph from the model that answers

E4B extracted fewer triples than E2B but achieved higher version 3 accuracy. In the usual runs, however, the same model both built the graph and answered the questions. We separated those roles by giving each reader the graph produced by each extractor (Table 6.7). The retrieval settings and question set stayed fixed.

Answer model	Graph built by E2B	Graph built by E4B
Gemma 4 E2B	21.5%	26.0%
Gemma 4 E4B	22.0%	27.5%

Table 6.7: Reader-extractor crossover on the same 200 LoCoMo questions, using version 3 and corrected embedding prefixes. Rows hold the reader fixed; columns hold the graph producer fixed. With the graph fixed, both readers received the same graph cache and retrieved context, and retrieval metadata matched for all 200 questions.

Changing the extractor from E2B to E4B gains 4.5–5.5 points with the reader fixed; changing only the reader gains 0.5–1.5 points. Most of the difference in these four cells is therefore associated with the graph producer. The crossover does not identify whether fact selection, naming or edge ranking is responsible, and it does not show that reader size is generally unimportant. Graph size and empty-batch frequency alone cannot explain the advantage.

6.7 What the study establishes and where it stops

The experiments distinguish failed or omitted extraction, evidence excluded during selection, detail missing from selected triples, and errors after evidence reaches the reader. These constraints should not be collapsed into one explanation. Larger-model work likewise finds that graph retrieval design and the supplied representation both affect outcomes [Han et al., 2026].

The depth controls are not exactly token-matched. Source links identify batches, normalization defects remain, and there is no complete manually verified reference graph. The results therefore do not assign all remaining error to triple compression.

Repairs were developed while inspecting this benchmark, so the repair sequence is a development record on LoCoMo, not a test held out from it; Section 4.5 repeats the repaired pipeline on LongMemEval-S. The repaired graph was not built on the phone; the available phone pilot is reported in Chapter 5.

6.8 From full source restoration to selective expansion

Source restoration improves answers but increases prompt length. This motivates choosing whether each retrieved fact should be omitted, retained as a triple or accompanied by source text under a shared context budget. Chapter 7 develops Selective Provenance Expansion and evaluates that decision against criteria fixed before its runs.

Chapter 7

Selective Provenance Expansion

Graph triples give a compact index of past dialogue, but extraction can remove qualifiers, temporal context and other details a question needs. Restoring the source turn keeps those details at a higher token cost. We designed and pre-registered *Selective Provenance Expansion* (SPE) to choose between the two: under a retrieval-context token budget, it decides whether to omit a retrieved fact, keep it as a triple, or include its source dialogue turn. This chapter describes the two versions of the selection rule, evaluates the repaired implementation against the corrected baselines, and examines why the expected trade-off did not appear.

Contribution of this chapter. SPE is proposed in this thesis. The closest systems also return from a compact memory unit to its source text (Section 2.3). SEEM does so for every retrieved event frame [Lu et al., 2026], and LeanMem does so when an LLM planner judges that a question needs detail [Liao et al., 2026]. SPE differs in where the decision is made. It weighs the two representations of each retrieved fact against each other under one shared token budget, and it was evaluated against criteria registered before its record runs. The result is negative. The chapter's second contribution is the explanation of that result: the rule's per-token scoring, together with the measured token costs, sets a bar for source text that the tested inputs did not clear (Section 7.3.4).

7.1 Why source text may matter

A long-term conversational memory has to trade compactness against fidelity:

1. **Compactness**: Longer prompts increase prefill work and KV-cache storage. Within the operating range measured in this thesis, they also increased inference energy. In the SPE runs, a selected triple cost 17.5 to 19.2 tokens on average, while upgrading it to its source text cost about 75. The shorter representation allows more retrieved facts to fit in the context window.

2. **Fidelity**: A triple does not preserve every detail of its source turn. Qualifiers, temporal expressions, speaker intent, and surrounding context may disappear during extraction, leaving the reader with less evidence than the original dialogue contained.

The tension is not an artefact of small extractors: Han et al. [2026] find the same pattern with a 70B reader, while small-model extraction itself lowers graph-reasoning accuracy [Kelesis et al., 2026].

The current E2B results in Chapter 6 show the trade-off: v3 reaches 21.5%, source-restored v4 reaches 33.0%, and vector retrieval reaches 36.0%. SPE was specified before these reruns.[1] Because v3 and v4 select the same graph edges, the gain comes from what the reader is shown.

This tension motivated the core question of this chapter: *Under an explicit retrieval-context budget, can an algorithm selectively decompress only the facts that require verbatim evidence, retaining unambiguous facts as compact triples, to achieve high accuracy at a lower resource footprint?*

SPE uses the repaired Gemma 4 E2B and E4B v3 graph caches described in Section 6.6.1; this chapter concerns how to select representations from those fixed inputs.

SPE is therefore not a separate method to be combined with GraphRAG v4. It runs on the graph and the source links that v4 uses. v3 shows every selected fact as a triple only, and v4 attaches the source turns of every selected fact; SPE makes that choice fact by fact. It selects from a larger candidate pool (up to 40 edges) under a token budget instead of a fixed twelve, and v4 is the accuracy reference it was required to approach at a lower prompt cost.

7.2 SPE design

SPE was designed as a small change to the existing graph retrieval path, not as a learned routing system. The existing pipeline already stored three useful pieces of information: the similarity between the query and each retrieved dialogue turn, the triples extracted from that turn, and the link from each triple back to its source. We used those values to decide how much of each fact to place in the prompt.

Candidate construction follows the implementation. Dense retrieval first selects the top-k dialogue turns. SPE then collects graph edges whose provenance overlaps those turns, orders the edges by the similarity of their best matching turn, and keeps at most 40 candidates. Each candidate can be omitted, shown as a triple, or shown as a triple together with the retrieved source turn. Source quotes shared by several triples are included only once, so the cost of a choice depends on what has already been selected.

Algorithm 1 summarizes the resulting greedy selection rule.

Algorithm 1 Selective provenance expansion using a greedy density rule

Require: Query q, candidate facts $\mathcal{E}$, retrieved source turns, token budget B
Mark every candidate as omitted and set the spent budget to zero.
while some representation change fits within B and has $\Delta U > 0$ **do**
  for all valid changes: omit→triple, omit→expanded, triple→expanded **do**
    Compute the additional tokens ΔC, excluding source quotes already present.
    Compute the additional utility ΔU supplied by the change.
  end for
  Among the changes that fit and have $\Delta U > 0$, choose the one with the largest $\Delta U / \Delta C$.
  Apply the change and update the spent tokens, included source turns, and covered terms.
end while
return the selected triples followed by the selected source quotes.

The two SPE versions differ only in how they define utility.

[1] At the design freeze, the earlier v3 and v4 baselines were 22.0% and 30.0%; the registered criteria retain those values.

7.2.1 SPE-1: a simple heuristic score

The first version combined signals already available in the retrieval pipeline. Similarity measured query relevance, lexical overlap rewarded uncovered query terms, a binary flag favored timestamped facts for temporal questions, and a redundancy term discouraged repeated evidence:

$$U_1(e, r, \mathcal{S}, \mathcal{C}) = \text{sim}(q, s_e) + \lambda_1 \cdot \text{Cov}(e, q \setminus \mathcal{C}) + \lambda_2 \cdot \text{Temp}(e, q) - \lambda_4 \cdot \text{Red}(e, \mathcal{S}) \tag{7.1}$$

Here, $\mathcal{S}$ is the set of source turns already included and $\mathcal{C}$ is the set of query terms already covered. The similarity, temporal and redundancy terms were identical for a fact's triple and expanded forms; only the coverage term could differ, because the expanded form also contains the source text. This issue is examined below. The three weights were set to 0.3 before evaluation and were not tuned on the test questions. This was an intentionally simple starting point: it tested whether readily available metadata was enough to choose which facts deserved more context.

7.2.2 SPE-2: score the two representations separately

SPE-1 had a basic problem. The triple and expanded versions of a fact shared the same similarity term even though they contained different text. Apart from lexical coverage, the selector therefore had little information about whether paying for the source turn was worthwhile. Before running the SPE-2 cells, we replaced that shared similarity with one for the triple and another for the source turn:

$$u_{\text{triple}}(e) = \cos(\mathbf{e}_q, \mathbf{e}_{\text{triple}(e)}) \tag{7.2}$$

$$u_{\text{source}}(e) = \cos(\mathbf{e}_q, \mathbf{e}_{\text{source}(e)}) \tag{7.3}$$

$$\text{gap}(e) = u_{\text{source}}(e) - u_{\text{triple}}(e) \tag{7.4}$$

The difference between the two scores is a proxy for detail that may have been lost during extraction. An upgrade from a selected triple to its source turn competes using $\text{gap}(e)/\Delta C$. SPE-2 removes the three heuristic weights, but it retains the candidate cap and token budget. The change was recorded before the SPE-2 evaluation and no further selection rule was introduced afterward.

7.3 Evaluation against the pre-registered criteria

The registered evaluation used Gemma 4 E2B on 200 LoCoMo questions, with answers judged by a 70B model. A frozen addendum repeated the comparison with E4B. The retrieval-context budget B covers retrieved triples and source quotes; total prompt length also includes the instructions, answer format, and question.

The registered criteria used the baseline values available when the design was frozen:

- **Criterion 1**: reach at least 30.0% accuracy with at least 25% fewer total prompt tokens than the expand-all baseline;
- **Criterion 2**: exceed 22.0% accuracy while keeping total prompt length below 600 tokens.

The fixed criteria remain unchanged; the comparisons below use the current baselines under the convention in Section 3.5.

7.3.1 The selector repair and why the repaired rule is the one evaluated

After the first SPE-2 runs and before the record runs, a code audit found two bookkeeping defects in the selector. The registered design states that choosing the expanded form of a fact *supersedes* its triple, and that the greedy step takes the remaining option with the highest utility per token (Algorithm 1). The code departed from that design in two ways:

1. Choosing the expanded form of an already selected fact appended a second copy of the fact instead of replacing the triple, so the triple line appeared twice in the prompt.

2. An upgrade from triple to expanded form was charged only its extra tokens, but it was credited with the full utility of the expanded form, u_{source}. The triple's utility, u_{triple}, had already been counted when the triple was selected, so the upgrade's value was counted partly twice.

The repair made the expanded form replace the triple and credited an upgrade with what it adds, $\text{gap}(e) = u_{\text{source}} - u_{\text{triple}}$, divided by what it adds in tokens. The first change implements the supersession that the recorded design states. The second is the implementation's reading of that design: the design sets the expanded form's utility to u_{source} and selects greedily by utility per token, but it does not write out how an upgrade from an already selected triple is credited. In a greedy budgeted selection, every step must be scored by the value it adds per token it adds. When an upgrade is credited with value the prompt already contains, the selector pays for source text for an accounting reason, not because the source text is worth its cost. The repaired rule can still choose source text. It does so whenever an upgrade's gap per extra token exceeds what the best remaining triple offers per token, or once every candidate triple has been taken and budget remains. That it never did so is therefore a result about these inputs, and Section 7.3.4 explains it.

The earlier version, which did quote source turns, also did not succeed. At $B = 370$ it quoted at least one source turn in 39 of the 200 E2B prompts, and it reached 26.0%, against 27.0% for the repaired rule; both are below the 30.0% that Criterion 1 requires.[2] Neither version met Criterion 1. Because the embedding prefixes changed at the same time, the repair's own effect on accuracy cannot be isolated. The repair defines the valid implementation used in this chapter and is not a new SPE variant; the registered design had already declared SPE-2 the final variant.

7.3.2 Registered E2B results

Table 7.1 reports the registered E2B comparison.

Criterion 1 failed. At $B = 370$, SPE-2 reached 27.0%, compared with 33.0% for GraphRAG v4, and reduced mean prompt length by only 9.3%. The paired difference did not separate (McNemar $p = 0.0576$, Holm-adjusted $p = 0.1152$), but the configuration missed both registered thresholds.

Criterion 2 passed numerically but remained inconclusive. The tighter configuration reached 23.0% within the prompt limit, yet its 1.5-point gain over GraphRAG v3 was not statistically distinguishable ($p = 0.711$).

SPE-1, run on the same pipeline as an exploratory comparison, did worse at both budgets: 15.0% at $B = 240$ and 20.5% at $B = 370$. Like SPE-2, it selected no source turns. At the

[2] At $B = 240$ the earlier version reached 24.0% for E2B, and 24.5% and 27.0% for E4B at the two budgets. These earlier runs also used embeddings without the task prefixes, so they differ from the record runs in two respects and are not used elsewhere in this chapter.

Configuration	Retrieval control	Total Prompt Tokens	Kept Triples	Source Turns	Accuracy
Full context	4096-token window	4096.0	n/a	n/a	7.5%
GraphRAG v3 (triples)	cap: 12 triples	519.6	12.00	0.00	21.5%
GraphRAG v4 (triples + sources)	cap: 12 triples	812.7	12.00	4.50	33.0%
Vector RAG	top-10 dialogue turns	824.4	0.00	10.00	**36.0%**
SPE-2 tight	context budget: 240 tokens	545.7	13.81	0.00	23.0%
SPE-2 registered	context budget: 370 tokens	737.1	20.68	0.00	27.0%
SPE-1	context budget: 240 tokens	547.4	14.03	0.00	15.0%
SPE-1	context budget: 370 tokens	738.3	20.85	0.00	20.5%

Table 7.1: Corrected baseline, repaired SPE-2 and SPE-1 results on LoCoMo ($n = 200$, Gemma 4 E2B reader, 70B judged accuracy). The baselines use fact-count or context-window controls, not SPE's explicit retrieval-context budget. Token and retrieval counts are means over 200 questions. The SPE-1 rows are exploratory and outside the registered criteria.

tighter budget it fell below GraphRAG v3 itself (exact McNemar $p = 0.029$, unadjusted), so the simpler scoring rule was not merely unable to recover the benefit of source text but spent its budget less well than the triple cap it replaced.

7.3.3 E4B addendum and reader–extractor factorization

The frozen scale addendum tested whether E4B could make better use of selectively compressed evidence. Because E4B initially served as both extractor and reader, this was an end-to-end comparison rather than a reader-only test. Table 7.2 places the registered E2B result beside the E4B addendum.

Memory Configuration	Gemma 4 E2B (2B)	Gemma 4 E4B (4B)
GraphRAG v3 (triples)	21.5%	27.5%
SPE-2 repaired ($B = 240$)	23.0%	25.5%
SPE-2 repaired ($B = 370$)	27.0%	25.0%
SPE-1 ($B = 240$)	15.0%	19.0%
SPE-1 ($B = 370$)	20.5%	22.5%
GraphRAG v4 (triples + sources)	33.0%	32.5%
Vector RAG	**36.0%**	**40.5%**

Table 7.2: End-to-end scale sensitivity between Gemma 4 E2B and E4B on LoCoMo ($n = 200$). For graph-based configurations, each column initially used the same model for extraction and reading, so the cross-column difference is not a reader-only effect. The E4B SPE comparisons are exploratory.

E4B did not rescue SPE. Its two SPE-2 conditions reached 25.5% and 25.0%, and its two SPE-1 conditions 19.0% and 22.5%, all below GraphRAG v4 at 32.5% and Vector RAG at 40.5%. The larger configuration therefore preserved the main result rather than narrowing the representation gap. Figure 7.1 places these record cells, with the post hoc oracle of Section 7.4, on one plane of accuracy against prompt length for both Gemma scales.

The two columns change extractor and reader together; the crossover in Table 6.7 associates most of the v3 difference with the graph producer rather than reader scale.

7.3.4 Why SPE retained only triples

Across both models and both budgets, none of the 800 pre-specified SPE-2 prompts (the registered E2B study and its frozen E4B addendum) selected a source turn.[3] The exploratory SPE-1 variant selected none either. At $B = 370$, E2B retained 20.7 triples on average and

[3]Only the repaired record runs are counted here; the earlier selector is described in Section 7.3.

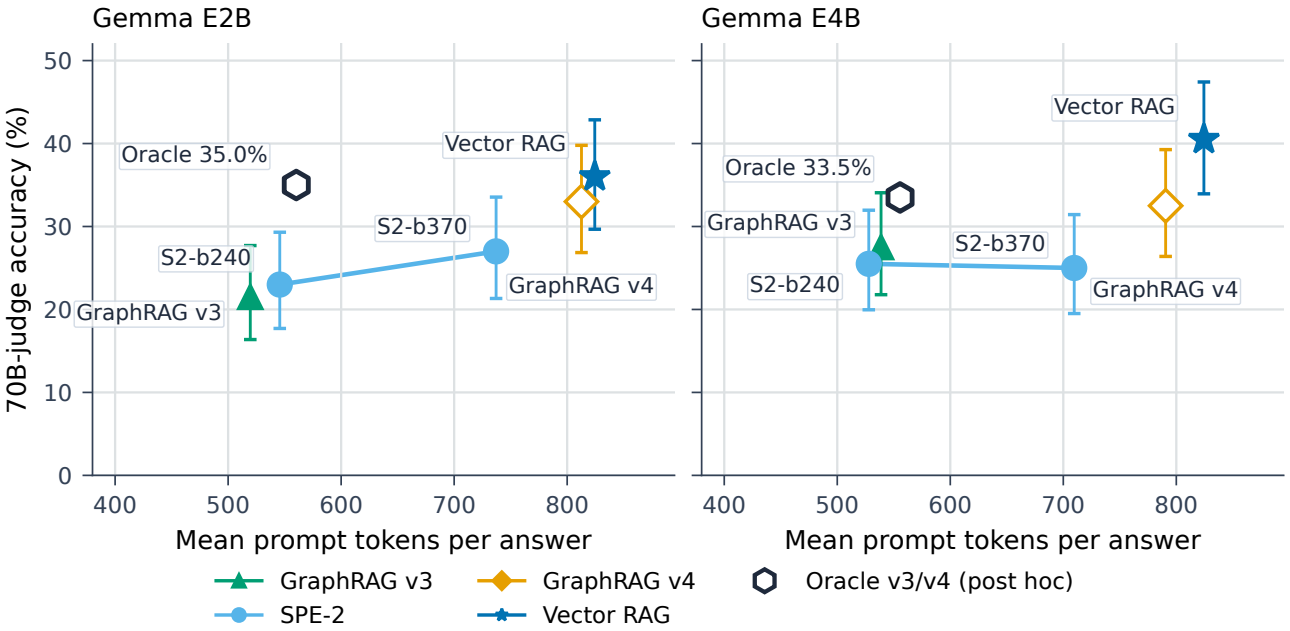


Figure 7.1: Accuracy against mean prompt tokens for Gemma 4 E2B and E4B on 200 LoCoMo questions, with corrected prefixes and the repaired selector. Whiskers are Wilson 95% intervals. Hexagons show the post hoc oracle selecting between v3 and v4 using judged labels; they bound those choices rather than represent an achievable policy. S2-b240 and S2-b370 mark SPE-2 at budgets $B = 240$ and $B = 370$. SPE-1 is omitted for legibility and reported in Tables 7.1 and 7.2.

E4B retained 18.9. SPE-2 was therefore a larger triple-only retriever, not a selective expansion policy.

Three properties of the inputs explain this outcome: what source text costs, what an upgrade adds, and how many triples were available.

Cost of source text. Source text was much more expensive than a triple. Measured offline from the logged prompts and graph caches, in the selector's own token units, the 4,136 triples that E2B SPE-2 selected at $B = 370$ cost 17.5 tokens each on average. Upgrading one of them would have cost 75.4 more tokens (median 68), because a quarter of these facts pointed to two or more retrieved turns. The median ratio $r = \Delta C/C$ of upgrade cost to triple cost was 3.9 for E2B and 3.5 for E4B, and r was below two for only 10% of E2B upgrades and 13% of E4B upgrades.

What an upgrade adds. At each step the selector compares what each option adds per token. A new triple with similarity u' to the query and cost C' adds its full similarity, u'/C'. An upgrade adds only the gap, $\text{gap}(e)/\Delta C$, because the triple's similarity is already in the prompt. For a competing triple of similar length, the upgrade wins only if

$$\text{gap}(e) > r \cdot u'. \tag{7.5}$$

The figure of about 0.26 below follows from this inequality under four stated assumptions:

1. the cost ratio r takes its median value of 3.9 (E2B); for an individual fact it can be smaller or larger;

2. the competing triple is about as long as the triple being upgraded, so $C' \approx C$;

3. both the triple and its source turn have non-negative similarity to the query, so the gap, which is the source turn's similarity minus the triple's, is below the source turn's similarity and therefore below 1;

4. the greedy step compares the upgrade with the best remaining triple that still fits.

Under these assumptions an upgrade can win only when every competing triple has a similarity below $\text{gap}(e)/r < 1/3.9 \approx 0.26$, and in practice below a quarter of the actual gap. Adding a fact directly in expanded form faces a similar bar: it costs the triple and the source text together, about 5.3 times a triple, so under the same assumptions the competing triples would need a similarity below about 0.19. The threshold is a consequence of the scoring rule, not a measured value: the per-candidate similarities were not logged.

Number of triples. An upgrade could also have won by default if the triples had run out with budget left, but they never ran out. Including every candidate triple would take at least 727 tokens on average for E2B and 814 for E4B, about twice $B = 370$. The selector logs confirm this for every prompt: in none of the 800 SPE-2 prompts did it take every candidate triple, and it stopped only after spending at least 220 of 240 and 348 of 370 tokens (`spe_spent_tokens` and `candidate_edges` in the run records). The logs show only that this default route never opened. They do not show why each upgrade lost its comparison with a triple; that part of the explanation rests on the scoring calculation above. Finally, the selector discards options whose utility is not positive, so a fact whose triple was at least as similar to the query as its source turn had no upgrade option at all.

The first expansion is the difficult step. Once one source turn is in the prompt, upgrading another fact that points to the same turn costs only one token, but under the repaired rule that first expansion never happened. The earlier version did reach it in 39 E2B prompts because it credited an upgrade with the source turn's full similarity instead of the gap (Section 7.3.1). Its condition was $u_{\text{source}} > r \cdot u'$, which is easier to meet because the source turn's similarity is larger than the gap.

This explanation follows from the rule and the measured costs. The runs did not log the utilities, so how often gaps were non-positive, and how close the best upgrades came to the bar, remain unmeasured; establishing either requires logged utilities or an ablation of the selector.

The representation-specific scoring also added query-time work. Mean retrieval time on the workstation was about 7.1 seconds per question for both SPE-2 budgets, compared with 0.21 seconds for the SPE-1 cells, 0.19 seconds for E2B GraphRAG v4 and 0.24 seconds for E2B Vector RAG. This implementation of the selection rule therefore carried a substantial latency cost.

7.4 Discussion

In both Gemma configurations, the selector did not recover the value of the omitted dialogue text. That conclusion concerns the selection rule, not whether a useful middle point existed; a post hoc oracle bound separates the two. For each question it keeps the v4 answer when v4 was judged correct and v3 was not, and the cheaper v3 answer otherwise. For E2B this reaches 35.0% at a mean of 560.3 prompt tokens, 31.1% fewer than v4, which would satisfy Criterion 1; for E4B it reaches 33.5% at 555.6 tokens, 29.7% fewer than v4. Expanding the same share of questions at random would give 23.1% and 27.8%. The oracle reads the judged labels, so it is an upper bound on question-level selection between two existing configurations rather than an achievable policy. Its value is to show where the headroom lay: source restoration turned 27 E2B answers and 12 E4B answers from wrong to right while reversing 4 and 2, so a selector had

to find a small minority of questions. The tested rule never spent its budget on source text, so it could not find them.

For these graphs and this dialogue QA task, the tested greedy policies did not identify a useful middle point between compact triples and source text. Text-attributed graph nodes might perform differently, although adding longer text to each node would narrow the practical distinction from chunk-based retrieval while retaining the graph-construction cost measured in Section 5.7.

SEEM and LeanMem also return from a compact memory unit to its source text (Section 2.3): SEEM for every retrieved event frame, within twice the initial retrieval budget [Lu et al., 2026], and LeanMem only when an LLM planner judges that a query needs detailed evidence [Liao et al., 2026]. Both are evaluated with server-scale backbones. SPE differs in where the decision is made: it weighs the two representations of each retrieved fact against each other under a shared token budget. That per-fact comparison is what failed here. A query-level decision of the kind LeanMem makes, for which the oracle bound above shows headroom, was not tested.

7.5 Limitations

The limits of the LoCoMo sample, the automatic judge and graph extraction stated in Chapters 4 and 6 apply here, and SPE ran on the workstation only. Three limits are specific to SPE:

1. **Selection policies**: SPE tested one heuristic and one representation-specific cosine score, both ranked by utility per token. That normalization was never ablated. Cosine similarity is not a ratio-scale measure of usefulness, so dividing it by token cost favors the shorter representation (Section 7.3.4). Calibrated utilities, learned policies, cross-encoder reranking, a budget share reserved for source turns and a query-level expansion decision were not tested.

2. **Scope**: All SPE runs used Gemma 4, E2B for the registered study and E4B for the frozen addendum, at budgets of 240 and 370 tokens. The negative result does not establish cross-family generality; testing another family now would be a new replication, not part of the frozen evaluation.

3. **Post hoc analyses**: The oracle bound reads judged labels, and the reader–extractor crossover was added after the scale comparison exposed a confound. Both are exploratory diagnostics, not deployable policies or confirmatory tests.

7.6 Summary

Selective Provenance Expansion did not provide a reliable middle point between triple-only retrieval and source dialogue for the tested Gemma configurations. On E2B, Criterion 1 failed, and Criterion 2 cleared its numerical threshold without a statistically distinguishable gain; E4B also stayed below the source-bearing baselines. None of the 800 pre-specified SPE-2 prompts selected a source turn: the density rule spent the budget on additional triples. A post hoc oracle shows question-level headroom between v3 and v4, so the negative result applies to the tested selection rule, not to selective expansion as a whole.

Chapter 8

Discussion

This chapter reads the measurements of Chapters 4 to 7 as answers to the three research questions, then states what those answers do not cover. Phone and workstation results stay separate throughout: the phone gives accuracy and device cost on one handset, and the workstation tests the same memory modules on more questions, readers and benchmarks, including the graph repairs and SPE.

Tables 8.1 and 8.2 list the main claims of Chapters 4 to 7, each with its sample size, its test and its status under the labels defined in Section 3.8.

8.1 RQ1: how memory strategy affects answer quality

Under a 4,096-token history budget, the strategy ordering was stable across every reader tested. On the phone, vector retrieval answered 20% to 50% of the 20 LoCoMo questions correctly, against 0% to 10% for full context. On the workstation's 200-question LoCoMo set it had the highest accuracy point estimate for all five readers, 28.5 to 32.5 points above truncated full context in the primary comparisons. Triple-only graph memory sat between the two, trailing vector retrieval by 13.0 to 15.5 points where extraction worked. LongMemEval-S repeated the ordering: vector retrieval scored 26.7% to 56.7%, no truncated full-context cell exceeded 10%, and the repaired Gemma graphs again sat between the two (Section 4.5).

For a system designer, the sizes of the two settings behind that ordering are also informative. Raising retrieval depth from two to fifty turns raised complete-evidence coverage from 16% to 64% and accuracy by 15 to 27 points in every reader. The tested reader doublings were worth 4.5 points from Gemma 4 E2B to E4B and 6.5 points from Qwen 3.5 2B to 4B. For readers of 0.8 to 4 billion parameters under a 4,096-token history budget, what the memory system put in front of the reader therefore moved accuracy more than the tested increases in reader size. The comparison sets a 25-fold range of depth against one doubling of the reader, and the larger depths cost longer prompts. It does not rank the two settings in general, and it does not apply to readers that can hold the entire history.

Two qualifications belong with this answer. First, full context scores low because the history does not fit: the workstation truncated LoCoMo histories to a 4,096-token window, and the phone kept only the most recent turns (79 of 419 in the conversation of Figure 3.3). With a long workstation window, full context rose to 41.5–44.0% for the three readers tested on LoCoMo, where it held the whole history, at or above vector retrieval, and to 40.0–48.3% on LongMemEval-S, where vector retrieval kept a 5- to 8-point lead that 60 questions could not separate (Section 4.6). Those long-window prompts were about 39 and 48 times longer than the retrieval prompts. The vector advantage is therefore an advantage in evidence per prompt token under a small context budget; on LoCoMo, a reader with a long enough window matched or passed it. On the phone that window is the constraint: a rough extrapolation from the

Claim	Sample	Test	Status	Section
Measurement				
The automatic judge agrees with a blind human annotator	100 judged answers, 86 questions; 83 decisive labels	Agreement 82/83; Cohen's $\kappa = 0.976$	Descriptive	3.6
Answer quality on the workstation (RQ1)				
Vector retrieval beats truncated full context on LoCoMo	200 paired; Gemma E2B, E4B, Qwen 4B	McNemar, Holm; cluster bootstrap	Primary	4.2.1
Vector retrieval beats triple-only graph memory (GraphRAG v3)	200 paired; Gemma E2B, E4B	McNemar, Holm	Primary	4.2.2
GraphRAG v3 $\rightarrow$ v4 (source turns restored) raises E2B from 21.5% to 33.0%	200 paired	McNemar, Holm	Primary	4.2.3
E2B to E4B reader under vector retrieval: +4.5 points, not separated	200 paired	McNemar, Holm	Primary	4.3
Vector retrieval beats truncated full context on LongMemEval-S	60 paired; Gemma E2B	McNemar, Holm	Primary	4.5
The same ordering holds for the other readers and the LongMemEval-S graph cells	60–200 paired per reader	McNemar, unadjusted	Exploratory	4.5
Long-window full context matches or exceeds vector retrieval on LoCoMo; separated for E2B only	200 paired; Gemma E2B, E4B, Qwen 4B	McNemar, unadjusted	Exploratory	4.6
Vector retrieval stays 5–8 points ahead of long-window full context on LongMemEval-S, not separated	60 paired; same three readers	McNemar, unadjusted	Exploratory	4.6
Retrieval depth $k = 2 \rightarrow 50$ adds 15–27 points, more than the tested reader doublings	100 paired; five readers	McNemar, unadjusted; magnitudes not tested	Exploratory	4.4
Question category is associated with correctness	Eight Gemma cells of 200	Permutation χ^2 within conversations, Holm over eight	Exploratory	4.2.4
Reasoning mode gives variable gains; 13–35% of retrieval-condition outputs unfinished	200 (LoCoMo), 60 (LongMemEval-S)	McNemar, unadjusted	Exploratory	4.7
The official LoCoMo prompt lowers accuracy without changing the ordering	200 paired; Gemma E2B, E4B	McNemar, unadjusted	Exploratory	4.8

Table 8.1: Main claims with their evidence: measurement and workstation answer quality (RQ1). *Pre-registered*: the criterion was fixed before the run that tests it; only these rows are confirmatory. *Primary*: one of the nine paired comparisons Holm-corrected as a family that was designated during the analysis. *Exploratory*: a test reported with its unadjusted p-value. *Descriptive*: a measurement reported without a test. McNemar tests are exact and two-sided.

Claim	Sample	Test	Status	Section
Phone resources (RQ2)				
Embedding prefixes have no detectable effect on phone vector accuracy (E2B 10→8, E4B 8→10)	20 paired; Gemma E2B, E4B	McNemar, unadjusted	Exploratory	3.5
Workstation accuracy cannot stand in for phone accuracy (83% agreement, threshold 90%)	100 paired; Gemma E2B vector	Agreement threshold; κ; McNemar	Pre-registered	5.3.1
Vector retrieval has higher accuracy, lower energy per answer and shorter query time than full context in all five readers	20 questions per cell; 3 vector sittings per reader; 1–3 full-context sittings	None	Descriptive	5.4
5.8–26.9× more correct answers per kilojoule	As above; ratios within matched sittings	Resampling sensitivity: lower end 2.6–8.8×	Descriptive	5.5
Peak RAM and temperature do not follow the energy ordering	As above	None	Descriptive	5.6
The early phone graph build (256-token limit, 58/84 batches failed) cost about 26× more energy per turn than vector ingestion	One graph build against three vector sittings	None; range 22–28× by sitting	Descriptive	5.7
Graph memory and SPE (RQ3)				
Small Qwen extractors fail on 79–85% of batches; more retrieved facts recover 1–2 answers	1,179 batches; 200 paired	McNemar, unadjusted	Exploratory	6.6
The graph producer, more than the reader, sets GraphRAG v3 accuracy	Four cells of 200	None	Descriptive	6.6
SPE misses Criterion 1 and stays below GraphRAG v4	200 paired; E2B (E4B addendum)	Registered thresholds; McNemar, Holm vs v4	Pre-registered	7.3
SPE passes Criterion 2 numerically, without a gain separable from GraphRAG v3	200 paired; E2B	Registered thresholds; McNemar vs v3	Pre-registered	7.3
SPE-2 selects no source turn	800 prompts	Count	Descriptive	7.3.4
A per-question choice between GraphRAG v3 and v4 has headroom (oracle)	200; uses judged labels	None; post hoc bound	Descriptive	7.4

Table 8.2: Main claims with their evidence: phone resources (RQ2) and graph memory and SPE (RQ3). Status labels as in Table 8.1. Phone rows use phone-measured accuracy only.

measured query time per prompt token puts a whole LoCoMo history at about 45 minutes per question, a length never run there. Second, the ordering holds for overall accuracy, not in every category: temporal questions never exceeded 26% in the main LoCoMo grid, and none of the tested memory designs reliably repaired that weakness.

8.2 RQ2: what memory strategy costs on a phone

On the Galaxy S23 Ultra, the quality and cost orderings agreed. Vector retrieval had a higher accuracy point estimate, lower energy per answer and shorter query time than truncated full context in all five reader pairs, and 5.8 to 26.9 times more correct answers per kilojoule where that ratio is finite. On this slice, every question that full context answered correctly was also answered correctly by vector retrieval. Bounded retrieval was therefore not a compromise between quality and cost on this device: it was better on both.

That result is whole-workload efficiency: each vector sitting's energy includes building its memory, about 60% of an E2B vector sitting. Construction is also where the strategies separate most sharply. In the phone pilot, extracting a graph cost about 26 times more energy per dialogue turn than embedding (Table 5.4). This cost is paid before the first question and is invisible to prompt-token accounting. Server-side evaluations of memory frameworks likewise find ingestion and retrieval dominating processing time [Jia et al., 2025]. Device comparisons should therefore price construction separately from each query, as the ledger in Chapter 5 does.

The resource results do not collapse into one ordering. Peak system RAM and peak hottest-zone temperature did not follow energy and latency, so a policy that optimizes only joules can still violate a memory or thermal limit. This is why the individual traces are reported alongside the joint measure.

Two boundaries limit how far these results travel. The workstation-to-phone parity gate failed at 83% agreement against a 90% threshold, so every phone claim rests on the 20-question phone cells. The graph strategy appears on the phone only as one early construction pilot, with 58 of 84 extraction batches failing, so this thesis makes no claim about the accuracy of a repaired on-device graph.

8.3 RQ3: what graph memory can and cannot do here

Here the mechanism, rather than the ranking, is the contribution. The repair sequence separated four places where evidence is lost: facts that never entered the graph because extraction failed or omitted them, stored facts excluded by the retrieval rule or triple budget, selected triples missing detail from their source turn, and evidence in the prompt that the reader did not use. Restoring the source text behind the selected triples raised Gemma 4 E2B from 21.5% to 33.0% ($p = 0.000170$ after the thesis's Holm correction) and E4B from 27.5% to 32.5%.

Both conditions started from the same ten retrieved turns, so the E2B gain points to the triple representation rather than retrieval as the main place the loss arises. For E4B, restoration recovered 5 of the 13 points that separated triples from vector retrieval.

SEEM and LeanMem also return from a compact memory unit to its source text (Section 2.3): SEEM for every retrieved event frame [Lu et al., 2026], and LeanMem when a planner judges that a question needs detail [Liao et al., 2026]. Neither reports energy. This thesis measures how much that return is worth for small readers under a small budget, and the device cost of building the compact unit in the first place.

SPE tried to keep the restoration gain while controlling its token cost. Criterion 1 failed, and Criterion 2 cleared its numerical thresholds without a distinguishable improvement over version 3; none of the 800 pre-specified SPE-2 prompts selected a source turn. Under per-token

scoring, an upgrade to source text had to earn roughly four times what a competing triple offered, and the budget ran out on triples first (Section 7.3.4). The utilities were not logged, so this explanation follows from the rule and the measured costs rather than from a logged trace. A post hoc oracle choosing between v3 and v4 per question with judged labels reaches 35.0% for E2B and 33.5% for E4B at about 560 total prompt tokens. It is an upper bound, not an implementable policy, but it shows headroom at the question level, so the negative result applies to the tested per-fact rule rather than to selective expansion in general.

The extraction floor sets the outer limit. The smaller Qwen extractors failed on roughly four batches in five and the phone pilot on 58 of 84, so for those configurations the question is less which retrieval rule to use than how much of the history extraction captures. Where extraction worked, raising the graph retrieval budget from twelve to twenty-four facts was worth 2.0 to 8.0 points; where it had failed, the same change moved one or two questions out of 200.

8.4 What it took to make these measurements trustworthy

Three lessons from the measurement procedure apply beyond this study.

Device accuracy has to be measured on the device. The pre-registered parity check failed, so phone accuracy comes only from phone answers and is never substituted from the workstation. The smaller phone cells are the price of that separation.

A budgeted selection rule needs a defined tie-break. The first graph retrieval rule gave most edges of a matched speaker hub the same score, so the facts that filled the budget depended on the process's hash seed; 164 of 200 version-1 questions had such a tie at the cutoff. The vector-first retrieval used from version 2 onward returned identical contexts under different seeds, so the record results are unaffected.

The energy measurement is simple and cross-checked, with one known gap. Energy comes from the phone's charge counter, weighted by the measured voltage. It agrees with the reported battery-level drop, and with integrated current to within 10% on sittings longer than 50 minutes. These are consistency checks, not independent instruments. Sittings that started at the highest possible charge (100%) under-count because the counter pauses at full charge; Appendix A quantifies the effect, and no comparison changes direction.

8.5 Threats to validity and what the results do not show

The chapter-level threat sections state the specific limits; the consolidated ones are these.

Sample and independence. The 200 LoCoMo questions come from ten conversations and the 60 LongMemEval-S questions from a stratified sample of one benchmark, so question-level observations are not independent. Paired tests and cluster sensitivity checks preserve the direction of the main comparisons, and category cells of 50 or 10 questions support only coarse statements.

Judging. All reported accuracies come from one automatic judge. Re-judging identical predictions changed no verdict within one judging session and one verdict in 100 across sessions. A blind human audit of 100 judged answers agreed with the judge on 82 of the 83 decisive labels

(Cohen's $\kappa = 0.976$), with no correct answer marked wrong; 17 answers were too ambiguous to label, and counting them all as disagreements would lower agreement to 82 of 100 (Section 3.6). The audit had one annotator, so it does not measure agreement between humans, and no second judge model was compared. The primary strategy gaps are far larger than the variation from re-judging. The audit counts only decisive labels from a sample drawn across conditions, so it does not bound judge bias within any single strategy comparison.

One device, one runtime, uneven replication. All device measurements come from one Galaxy S23 Ultra, one Android build and a CPU-oriented llama.cpp configuration, with three sittings for each vector cell and complete full-context replication only for Gemma 4 E2B. Each phone cell has 20 questions, so the efficiency multipliers are uncertain: resampling the question pairs puts their lower end at 2.6 to 8.8× (Section 5.5). A different SoC, an accelerator path or a different thread and governor configuration may change the ordering.

Cross-study comparison. External accuracy numbers in this thesis are context, not comparators. Readers, judges, retrieval units, context limits and question populations differ across the cited studies, so their effect sizes are not pooled with those measured here.

Four statements are excluded deliberately, because the evidence in this thesis does not support them.

1. **That knowledge graphs fail for conversational memory.** The evidence concerns graphs extracted by small local models under a small generation budget and queried by turn-anchored triple selection, without multi-hop traversal; in these speaker-centred graphs most edges already lie next to the retrieved facts' entities. Community-summary designs, much larger extractors and curated knowledge graphs were not evaluated. SmartRAG demonstrates a graph-based mobile memory system built with a distilled extractor [Jiang et al., 2026], and the token-matched comparison of Han et al. [2026] retains a temporal-accuracy advantage for graph retrieval at server scale.

2. **That selective expansion cannot work.** The tested per-fact rule failed, and the oracle shows that a question-level choice had headroom on the same cells. A query-level policy of the kind LeanMem applies [Liao et al., 2026] was not tested here.

3. **That the device draws a fixed power.** The pilot's average of about 4 W is the quotient of one workload's energy and its duration. The frequency and temperature traces are consistent with thermal or power throttling; they do not identify a governor policy or establish an invariant power figure.

4. **That prefill dominates the measured query time.** The phone logs record total query time, and prefill and decode were not timed separately. The relationship between prompt length and query time is consistent with prompt processing dominating, which is weaker than a measured phase split.

Stated positively, for conversational question answering with small local readers under a small context budget, retrieving a bounded amount of source text was the strongest quality-cost point measured, and on the workstation it stayed within 1.5 to 8 points of reading the whole history on LoCoMo, and ahead of long-window full context on LongMemEval-S, with 2 to 3% of the prompt tokens; the compact graph representation lost evidence that its own source spans still contained; restoring that text recovered part of the loss at workstation scale; the tested rule for spending a token budget on that restoration did not work; and building the compact representation carries a device cost that prompt-level accounting does not show.

Chapter 9

Conclusion

This thesis asked what a small language model running on a phone should do with a conversation it can no longer fit in its context window. The question was answered by measuring, in one pipeline, what each memory strategy delivers in answer quality and what it costs on the device that has to run it.

9.1 Findings

Retrieval choices mattered as much as reader size. Vector retrieval answered more questions correctly than truncated full context for all five readers on the phone, had the highest accuracy point estimate for every reader on the workstation under the same truncation, and kept that ordering on LongMemEval-S. Full context failed because of the truncation: with a long window, the three readers tested matched or exceeded vector retrieval on LoCoMo, where they read the whole history, and trailed it, without separating, on LongMemEval-S, with prompts about 39 and 48 times longer. For readers of 0.8 to 4 billion parameters under a 4,096-token history budget, the tested range of retrieval depth moved accuracy more than the tested doublings of reader size. For readers of this size, the evidence placed in front of the reader is therefore worth tuning alongside the choice of model.

On the phone, the better answer was also the cheaper one. Vector retrieval had higher accuracy point estimates, lower energy per answer and shorter query times than truncated full context in all five reader pairs, with 5.8 to 26.9 times more correct answers per kilojoule where the ratio is finite. Building a graph in the phone pilot cost about 26 times more energy per dialogue turn than embedding, a cost paid before the first question that prompt-token accounting does not show, and peak RAM and temperature did not follow the energy ordering.

The graph lost evidence that its own sources still held. On the same retrieved turns, restoring the source text behind the selected triples raised Gemma 4 E2B from 21.5% to 33.0%: adding the associated source text recovered much of the observed deficit. Below that sits an extraction floor: where most extraction batches failed, as for the smaller Qwen extractors and the phone pilot, retrieving more facts did not recover the answers.

The proposed selection rule did not work. Selective Provenance Expansion failed its first pre-registered criterion, and its second passed numerically without a distinguishable gain; the repaired selector chose no source turn in any of the 800 pre-specified prompts. A post hoc oracle using judged labels shows headroom at the question level, so the negative result bounds this per-fact rule rather than selective expansion itself.

9.2 Implications for on-device memory

The most general implication is that answer quality and device cost have to be measured in the same experiment, on the device, because prompt-level proxies miss the cost of building memory. Three design consequences follow.

1. **Price the whole lifecycle, not the prompt.** Construction, retrieval, prefill and decode are separate operations, and the strategies separated most sharply in construction, which never appears in a prompt-token count.

2. **Default to bounded retrieval of source text, and keep device limits visible.** On this phone it had higher accuracy point estimates, shorter query times and lower energy per correct answer than truncated full context. On the workstation it trailed long-window full context by up to 8 points on LoCoMo and stayed ahead of it on LongMemEval-S, with 2 to 3% of the prompt tokens. A compact derived representation must earn its construction bill, and peak memory and temperature still decide whether a configuration can run at all.

3. **Instrument for provenance and repeatability.** Digests, explicit measurement spans and checkpoints turn a battery-consuming run into evidence. A budgeted selection rule also needs a defined tie-break: in the first graph retrieval rule, 164 of 200 questions had a score tie at the fact budget.

9.3 Future work

Close the measured gaps first. The most direct next steps address the limits stated in Chapter 8: a second annotator and a second judge model for the human audit, adding matched full-context phone sittings so that every efficiency ratio rests on comparable replication, and running a complete repaired graph-QA cell on the phone with separate meters for construction and query service. A few whole-history LoCoMo prompts should also be timed and metered on the phone, so that the cost of long-window full context rests on a measurement rather than on the arithmetic in Section 4.6. Because building the memory took about 60% of the energy in a 20-question E2B vector sitting, the same questions should also be repeated at 20, 50 and 100 queries per history, to measure how construction amortizes with reuse.

Cheaper construction. Graph memory cost 52.3 J per turn on the phone because a general-purpose small model wrote it. A specialized extractor, such as the distilled entity recognizer of SmartRAG [Jiang et al., 2026], or a construction pipeline distilled into one compact model, as in MemoryCPT [Lei et al., 2026], would attack that term directly. Such a component should be judged by the ledger used here: joules for construction and joules per correct answer, not tokens alone.

A better representation to store. The measured deficit belongs to one representation: short triples produced by a small model, with their source spans reachable but not present. Representations that keep text with their structure, or store a retrieval gist beside a pointer to the original turn, should preserve the qualifiers and temporal detail that source restoration returned, and can be tested against the same matched comparison.

Decide expansion once per question. The SPE analysis points to a utility that is comparable across representations and to a decision taken once per question instead of once per fact. Reserving part of the budget for source text, or deciding at query level as LeanMem does with a planner [Liao et al., 2026], are the variants the oracle bound justifies testing.

Beyond question answering. The measured task is memory-dependent question answering. A phone agent also acts: it calls tools, changes device state and observes the result. Extending this evaluation to tool use means scoring whether the memory supplied the evidence behind a correct action, at a measured cost.

Appendix A

Supplementary analyses for on-device memory

This appendix keeps the main argument of Chapter 5 separate from the details needed to reproduce it. The chapter uses two formal statistical results: the workstation-to-phone agreement rule and the omnibus question-family test. The remaining material below describes replicate variation, run-length sensitivity, thermal peaks and analytical routing scenarios. These secondary calculations are not additional primary claims.

A.1 Cross-device agreement

The parity cell used the same 100 stable question identifiers on both devices. Both runs used Gemma 4 E2B with Vector RAG, `top_k=10`, `num_ctx=4096`, prompt style v3 and the Nomic embedding prefix. The workstation used Ollama, while the phone used the quantized llama.cpp path.

Quantity	Result
Paired questions	100
Matching correctness labels	83 (83.0%)
Required agreement	at least 90%
Cohen's κ	0.621
Workstation-only correct	5
Phone-only correct	12
Exact McNemar p	0.1435
Exact answer-string matches	40 (40.0%)

Table A.1: Components of the predefined workstation-to-phone agreement check. The rule required at least 90% label agreement and McNemar $p > 0.05$. It failed on agreement.

Agreement and McNemar's test serve different purposes. The first asks whether the two environments label the same questions the same way. The second uses only the 17 discordant pairs and asks whether their direction is imbalanced. The observed $p = 0.1435$ does not demonstrate equivalence. It only fails to establish a directional shift. The 83% agreement therefore remains the operative result, and all Chapter 5 accuracy numerators come from phone outputs. Figure A.1 illustrates the predefined parity threshold, measured label agreement, and the paired 2×2 decision matrix.

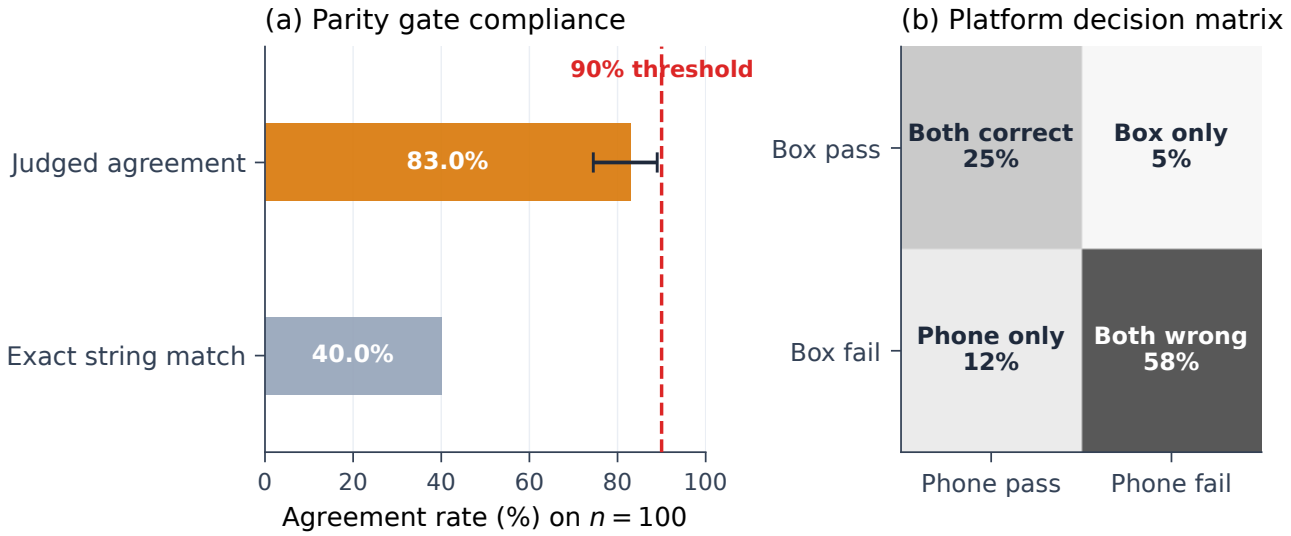


Figure A.1: Predefined workstation-to-phone parity gate diagnostic ($n = 100$ paired LoCoMo questions). Left: required threshold (90%) vs measured label agreement (83.0%, Wilson 95% CI [74.5%, 89.0%]) and exact string match (40.0%). Right: 2×2 concordance matrix, where "Box" is the workstation. The phone scored 37% and the workstation 30%, and the two runtimes worded many answers differently. Under the predefined rule the gate failed, so phone accuracy is always measured on the phone.

A.2 Question-family randomization test

Chapter 4 reports 50 LoCoMo questions from each of four families in every primary Gemma cell. To test the visible category separation without treating the 200 answers as independent, the analysis works within the ten source conversations.

For a given reader-memory cell, the observed statistic is Pearson's chi-square from the 4×2 table of question family by correct or incorrect outcome. Under the null, correctness is exchangeable across family labels within a conversation. Each of 100,000 randomizations shuffles the observed binary labels only within their source conversation. This keeps the conversation's total number correct and its category composition fixed. The randomization p-value is the proportion of shuffled statistics at least as large as the observed statistic, with the standard plus-one correction. Holm's procedure adjusts the eight cell-level p-values as one family. The test was added after the category decomposition was inspected. It is a supporting analysis, not a member of the primary strategy-comparison family.

Every adjusted value is below 0.05. The omnibus test establishes that correctness is associated with question family within each row. It does not identify which pairs of families differ and does not turn the adjacent percentages in Chapter 4 into pairwise findings.

A.3 Phone energy replication

Table A.3 gives the repeated sitting values that are collapsed to means in Chapter 5. The standard deviation describes variation between complete sittings. It is not a per-query error bar.

Gemma E2B is the only strategy pair with three complete sittings on both sides. Its within-sitting Vector/full-context correct-per-kilojoule ratios are 23.18, 26.86 and 25.37. Their mean is 25.14 and their standard deviation is 1.85. The second full-context sitting completed 19 answers, so its accuracy is calculated as 1/19. The ratio generator now reads the exact scored labels rather than reusing the first sitting's 5% accuracy. This is why the paired mean is not

Reader	Memory	C1	C2	C3	C4	$\chi^2(3)$	Holm p
E2B	Full context	5	0	10	0	19.82	0.00018
E2B	Graph triples	6	6	13	18	12.18	0.00548
E2B	Graph + sources	8	11	17	30	25.78	< 0.0001
E2B	Vector RAG	16	9	16	31	22.40	0.00015
E4B	Full context	6	0	15	0	32.08	< 0.0001
E4B	Graph triples	6	9	16	24	19.34	0.00050
E4B	Graph + sources	8	8	18	31	32.52	< 0.0001
E4B	Vector RAG	17	10	21	33	23.14	0.00015

Table A.2: Omnibus question-family tests for the eight primary Gemma cells. C1 to C4 are the numbers correct out of 50 for multi-hop, temporal, open-domain and single-hop respectively. The chi-square value is the observed separation statistic. Each p-value comes from conversation-stratified randomization and is Holm-adjusted across all eight rows.

Reader	Memory	Sitting 1	Sitting 2	Sitting 3	Mean	SD
Qwen3.5 0.8B	Vector RAG	248.5	323.9	345.1	305.8	50.8
Qwen3.5 2B	Vector RAG	437.1	420.6	440.7	432.8	10.7
Qwen3.5 4B	Vector RAG	566.7	594.3	546.1	569.0	24.2
Gemma 4 E2B	Vector RAG	653.8	473.4	525.0	550.7	92.9
Gemma 4 E2B	Full context	1515.6	1338.7	1331.9	1395.4	104.1
Gemma 4 E4B	Vector RAG	770.9	748.3	693.3	737.5	39.9

Table A.3: Voltage-weighted joules per scored answer in cells with three complete sittings. The other full-context cells have one complete sitting.

equal to the quotient of the rounded energy means in the main tables.

Repeated sittings reduce the uncertainty in energy, not in accuracy: greedy decoding gives the same answers in every sitting, so the accuracy of each cell still rests on 20 questions. A question-level sensitivity analysis resampled the 20 observed paired questions with replacement 20,000 times, with energy per answer held at its measured mean. The 2.5th percentile of the vector-to-full-context ratio is 2.6× for Qwen3.5 2B, 5.7× for Qwen3.5 4B, 7.6× for Gemma E2B and 8.8× for Gemma E4B; for Qwen3.5 0.8B full context has no correct answer, so no finite ratio is reported. The range 2.6× to 8.8× summarizes four separate lower percentiles, not one interval for the whole experiment. These percentiles are conditional on fixed energy: they exclude between-sitting energy variation and charge-counter bias, and the question-level resampling does not account for dependence within a conversation. In all five readers, every question that full context answered correctly was also answered correctly by vector retrieval. This describes answer quality on the observed sample; it does not validate the energy measurements. Resampling these pairs cannot introduce full-context-only successes that were absent from that sample. The percentiles therefore are not guaranteed lower bounds for new workloads.

Accuracy intervals displayed in Figure 5.1 are Wilson 95% intervals for 20 binary outcomes. They describe uncertainty in each marginal accuracy estimate. They do not test whether the efficiency ratio differs across reader families.

A.4 Run-length observation

The separate E2B Vector-RAG run over 100 phone questions consumed 29.76 kJ, scored 37 of 100 answers correct and used 297.6 J per answer. The three 20-question sittings used 550.7

J per answer on average. The observed ratio is 1.85. Splitting each run into its memory-building windows and its question stretches accounts for most of it. Building the memory for six histories took 62% and 60% of the energy in the two 20-question sittings that did not start at full charge, against 32% in the 100-question run, which built ten histories (its first two, begun while the charge counter was paused at full charge, are left out of that share), while the questions themselves cost about 198 J per answer at 100 questions against 205 to 246 J at 20. The lower figure at 100 questions therefore comes mainly from spreading the memory build over more questions. It is still not a matched run-length intervention: the 100-question run used the task prefixes, and its question composition and execution time differ. No main-table value is rescaled by 1.85.

A.5 Thermal peak detail

The mean of the within-sitting maxima of the hottest thermal zone is higher for Vector RAG in all five reader pairs. Table A.4 reports the observation directly.

Reader	Full context (°C)	Vector RAG (°C)	Difference
Qwen3.5 0.8B	95.80	97.53	1.73
Qwen3.5 2B	95.40	99.13	3.73
Qwen3.5 4B	95.80	96.33	0.53
Gemma 4 E2B	95.67	98.87	3.20
Gemma 4 E4B	97.00	97.93	0.93

Table A.4: Displayed means of per-sitting peaks of the hottest thermal zone. Vector RAG has three sittings in every row. Full context has three for E2B and one for the other four readers. These values are descriptive and are not battery temperatures.

The repeated 5-of-5 direction is worth reporting, but unequal replication makes its uncertainty asymmetric. Peak temperature also ignores exposure duration. The table therefore does not support a causal claim that retrieval runs hotter.

A.6 Long-context and routing sensitivities

Figure A.2 sets the KV-cache size of the two Gemma readers against the memory the phone had left. The curves are calculated from the layer shapes in the headers of the phone's own GGUF files, not measured at run time; the horizontal lines are measured. No context longer than the 8,192-token server context was run on the phone, so the figure describes how the cache would grow, not how long prompts would perform.

KV sharing and 512-token sliding windows leave only three E2B and four E4B layers whose cache grows with context. These counts agree with the Gemma 4 technical report [Gemma Team, Google DeepMind, 2026]. Its Table 3 gives int8 cache sizes of 0.05 and 0.14 GB at 32,768 tokens, about half the f16 estimates here. The plotted estimates are conservative relative to those lower-precision footprints; they are not bounds on the runtime's other allocations.

The exploratory routing grid combines the direct per-turn construction anchors with projected graph query energy and workstation accuracy. It now makes two like-for-like comparisons. Raw query tokens are compared with raw lifecycle energy, producing different choices in 38 of 40 history-length and query-count profiles. Query tokens per correct answer are compared with lifecycle energy per correct answer, producing different choices in 20 of 40 profiles. These

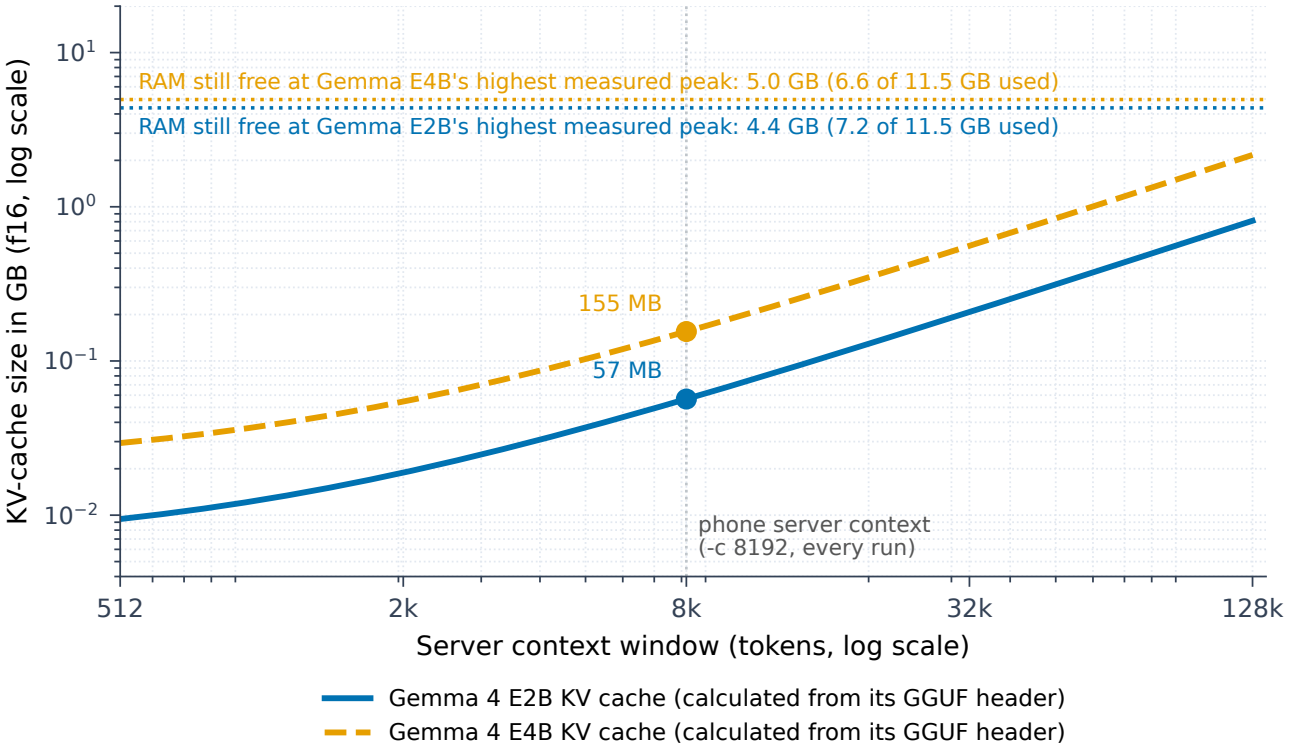


Figure A.2: Calculated f16 KV-cache size versus context length for Gemma 4 E2B and E4B, using the phone GGUF layer shapes. Markers show the reserved 8,192-token cache (57 and 155 MB). Dotted lines show measured free RAM at each model's highest system-memory peak (4.4 and 5.0 GB). The curves describe cache growth alone; they do not establish total runtime memory or the feasibility of longer-context inference.

are scenario counts, not statistical tests. Figure A.3 visualizes this dynamic break-even frontier as user query volume scales.

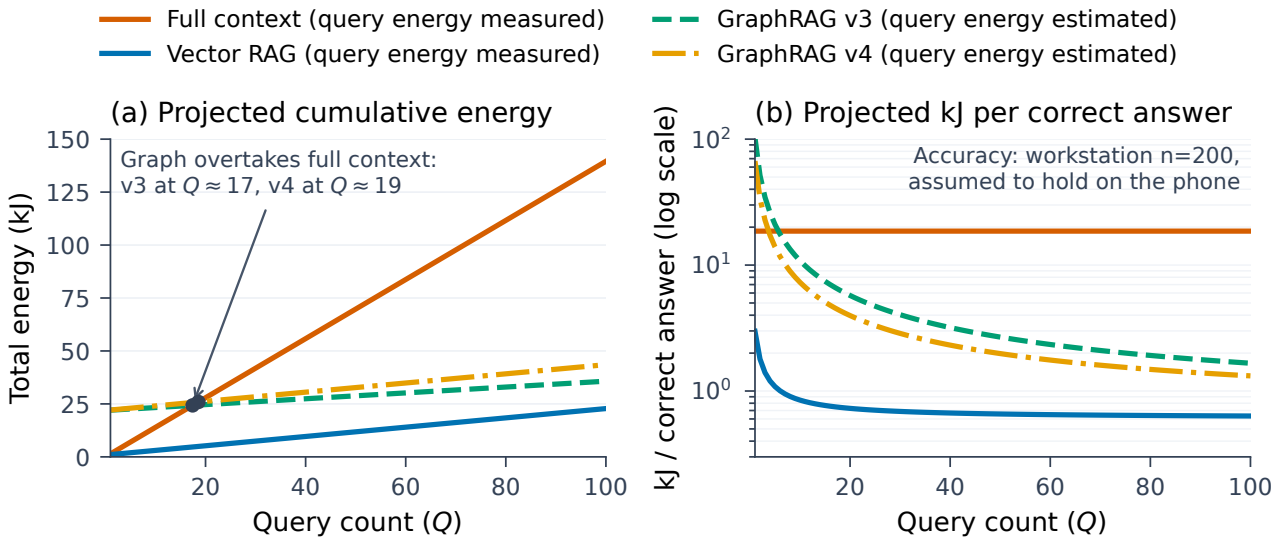


Figure A.3: Illustrative amortization for Gemma 4 E2B over a 419-turn history. (A) Cumulative energy including construction; markers show projected crossings with full context. (B) Energy per expected correct answer, on a logarithmic scale. Construction and vector/full-context query energies are phone measurements; graph query energy is estimated from prompt tokens. Workstation accuracy is assumed to transfer despite the failed parity criterion. The phone graph-build pilot and workstation graph-accuracy runs also use different configurations. These curves describe a scenario, not measured on-device graph efficiency.

The full-context and vector query-energy anchors average three sittings. Vector query energy excludes embedding windows, so construction is counted once. Graph query energy is scaled from vector query energy using workstation prompt tokens; accuracy uses the workstation $n = 200$ record. The graph build is the early phone pilot, with 58 of 84 failed batches, whereas the workstation graphs had under 1% failures. These mismatched inputs and the failed parity criterion limit the calculation to an illustrative scenario.

Under these assumptions, GraphRAG v3 and v4 cross full context's cumulative energy after about 17 and 19 queries. Neither overtakes vector retrieval per expected correct answer over the plotted 1–100 queries: vector construction costs 0.85 kJ against the graph's 21.9 kJ, and its assumed accuracy is higher. These are properties of the projection.

The former 40-of-40 statement is retired because it compared raw prompt tokens with energy per correct answer. Introducing accuracy normalization changed the result. More importantly, no complete phone graph-QA cell exists. The only conclusion carried into the main chapter is that query-token counts omit memory construction.

A.7 Energy and charge conventions

Phone energy integrates each positive charge-counter interval against the mean recorded battery voltage for that interval. The direct graph construction span is 21.898 kJ for 419 turns, or 52.263 J per turn. Its charge-counter decrease is 1495.305 mAh, which implies an effective voltage of 4.068 V over that span.

The battery readings provide two consistency checks. In every sitting, the charge-counter decrease matches the drop in reported battery level, at 47.47 mAh per percentage point, to within about one percentage point, which is the resolution of the level reading. Over the construction span, for example, the counter fell 1,495 mAh while the level fell from 91% to 60%, or about 1,472 mAh. Integrating the logged instantaneous current against voltage agrees with the counter energy to within 10% in every sitting longer than 50 minutes (median +2%; 22.6 against 21.9 kJ for construction). The instantaneous current is used only for this check. Termux:API reports it in inconsistent units: values near -1 and near $-1{,}000$ describe the same draw of about one ampere.

The counter has two limits. It reports in steps of 0.1% of capacity (4.747 mAh, about 68 J at 4 V), updated about every 30 s, which is negligible over a whole sitting. It also holds its full value while the battery reads 100%. Eleven sittings started at the highest possible charge, 100%. In nine of them the counter stayed flat for 161 to 380 s while the logged current averaged 0.9 to 1.5 A, and it did not catch up once it began to fall, so that energy is missing. Estimated from the logged current over the flat interval, the shortfall is about 2 to 5% for the long sittings: E2B full context in its first and third sittings, Qwen3.5 4B full context, and the two $n = 100$ runs. It is about 15 to 25% for the short ones: E2B vector retrieval in its second sitting, Qwen3.5 0.8B vector retrieval in its first sitting, Qwen3.5 0.8B full context, and the $k = 2$ run. The reported values are not corrected, because the estimate rests on the inconsistently scaled instantaneous current. With the estimate applied, the E2B vector-to-full-context ratio would move from 25.1× to about 24.5×, and the Qwen3.5 0.8B energies would move the most. No direction reported in Chapter 5 changes.

Correct answers per full-charge equivalent use measured mAh per answer and the phone's 5,000 mAh capacity. They do not divide by a fixed energy-per-charge constant. The earlier 3.88 V routing convention and its 73% graph projection are therefore not used in the chapter. This separation keeps the energy and charge calculations internally consistent.

Appendix B

Reproducibility record

This appendix records the configuration values that the main chapters rely on but do not state, so that a reader can interpret the reported numbers and repeat the experiments. It holds the prompt texts, question sets, settings, artifact identifiers, a worked prompt example and a glossary. The measurements themselves are in Chapters 4 to 7 and the statistical detail is in Appendix A.

B.1 Prompts

Each prompt below was held fixed across every condition in which it was used, so that a difference between conditions cannot be a difference in instructions. The texts are reproduced from the harness source. Long lines are wrapped for display, and placeholders in braces are filled at run time.

Reader answer prompt (`qa_short_answer_v3`)

Used for every memory strategy, every reader and both benchmarks. `{context}` holds the memory text built by the strategy (Section B.4 shows examples). `{date_line}` is empty on LoCoMo; on LongMemEval it gives the question date ("Today's date is …"), so that relative terms can be resolved. The answer is read from between the `<answer>` tags.

```
You are answering a question based on a conversation history.

Conversation history:
{context}

{date_line}Question: {question}
Rules:
1. Answer the question in as few words as possible.
2. For dates, give the date only (e.g., "2022" or "7 May 2023"). Resolve
   relative terms like "yesterday", "last week", "last year" mathematically
   using the timestamp of the fact.
3. For names, the name only.
4. For yes/no questions, answer "Yes" or "No".
5. Do not explain.
6. Do not repeat the question.
7. Put the final answer between <answer> and </answer>.
Answer:
```

Official LoCoMo prompt (prompt-robustness check only)

Used only for the substitution test of Section 3.8. It is reproduced verbatim from the LoCoMo repository, including its original spelling of "wriiten"; `{s1}` and `{s2}` are the two speakers.

```
Below is a conversation between two people: {s1} and {s2}. The conversation
takes place over multiple days and the date of each conversation is wriiten
at the beginning of the conversation.

{context}

Based on the above context, write an answer in the form of a short phrase
for the following question. Answer with exact words from the context
whenever possible.

Question: {question} Short answer:
```

Graph extraction prompt (`triple_json_v2`)

Sent once per batch of five turns, each rendered as "speaker: text". The reply is parsed as JSON under an extraction budget of 256 tokens (baseline) or 512 tokens (repaired pipeline).

```
Identify factual relationships in this dialogue segment, preserving all
dates, times, locations, and specific details.

Dialogue:
{dialogue}

Respond with a JSON object containing a 'triples' key which is a list of
objects. Each object in 'triples' must have exactly three string keys:
'subject', 'relation', and 'object'.
Do not use pronouns (he, she, it, I) as subjects; resolve them to concrete
names if clear from context.
Capture key attributes of the speakers and any temporal details.
Example output structure:
{"triples": [{"subject": "alice", "relation": "works at", "object":
"north clinic"}, {"subject": "bob", "relation": "graduated in", "object":
"2019"}]}
```

Judge prompts (LongMemEval `get_anscheck_prompt`)

The judge receives the question, the reference answer and the reader's parsed answer. A reply beginning with "yes" counts as correct and one beginning with "no" as incorrect; a reply that cannot be parsed is set aside for review rather than scored as wrong. LoCoMo answers use the standard form:

```
I will give you a question, a correct answer, and a response from a model.
Please answer yes if the response contains the correct answer. Otherwise,
answer no. If the response is equivalent to the correct answer or contains
all the intermediate steps to get the correct answer, you should also answer
yes. If the response only contains a subset of the information required by
the answer, answer no.

Question: {question}
```

```
Correct Answer: {gold}

Model Response: {pred}

Is the model response correct? Answer yes or no only.
```

LongMemEval answers use the same standard form except for four question types, as in the benchmark's own evaluation:

- **Temporal reasoning** adds: "In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18 days), the model's response is still correct."
- **Knowledge update** replaces the equivalence sentences with: "If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer."
- **Single-session preference** supplies a rubric in place of a correct answer: "Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user's personal information correctly."
- **Abstention** questions ask whether the model "correctly identifies the question as unanswerable"; the model may say that the information is incomplete, or that other information is given but the asked information is not.

B.2 Question sets

LoCoMo comparisons use 200 questions from the released ten-conversation set, 50 from each of Categories 1 to 4, taken in turn from each conversation (round-robin). The LongMemEval subset is a stratified sample of 60 questions, ten from each of the six question categories, drawn with seed 42 and stored as a fixed file so that every run sees the same questions in the same order. Question identifiers are assigned per run and depend on the sample size, so cross-run joins use the stable identifier carried in each record.

Each run records a SHA-256 hash of its loaded question set. The 200 LoCoMo questions hash to `8f7200d571bd6391`…, the 60 LongMemEval questions to `64f5c89412584b98`… and the 20-question phone slice to `231afbb0f95fac1b`… (first 16 hexadecimal digits).

The 20-question phone slice

The primary phone comparison uses the 20 questions below, drawn by the same round-robin procedure with five questions per category. Every phone accuracy in Chapter 5 rests on them. Questions and reference answers are reproduced as they appear in the dataset, including its spelling errors.

Multi-hop (Category 1)

1. `conv-26`: What did Caroline research? *Reference:* Adoption agencies
2. `conv-30`: What do Jon and Gina both have in common? *Reference:* They lost their jobs and decided to start their own businesses.

3. `conv-41`: What martial arts has John done? *Reference:* Kickboxing, Taekwondo
4. `conv-42`: What kind of interests do Joanna and Nate share? *Reference:* Watching movies, making desserts
5. `conv-43`: what are John's goals with regards to his basketball career? *Reference:* improve shooting percentage, win a championship

Temporal reasoning (Category 2) 1. `conv-26`: When did Caroline go to the LGBTQ support group? *Reference:* 7 May 2023
2. `conv-30`: When Jon has lost his job as a banker? *Reference:* 19 January, 2023
3. `conv-41`: Who did Maria have dinner with on May 3, 2023? *Reference:* her mother
4. `conv-42`: When did Joanna first watch "Eternal Sunshine of the Spotless Mind? *Reference:* 2019
5. `conv-43`: In which month's game did John achieve a career-high score in points? *Reference:* June 2023

Open-domain knowledge (Category 3) 1. `conv-26`: What fields would Caroline be likely to pursue in her educaton? *Reference:* Psychology, counseling certification
2. `conv-41`: What might John's financial status be? *Reference:* Middle-class or wealthy
3. `conv-42`: Is it likely that Nate has friends besides Joanna? *Reference:* Yesteammates on hisvideo game team.
4. `conv-43`: Would Tim enjoy reading books by C. S. Lewis or John Greene? *Reference:* C. S.Lewis
5. `conv-44`: What is an indoor activity that Andrew would enjoy doing while make his dog happy? *Reference:* cook dog treats

Single-hop (Category 4) 1. `conv-26`: What did the charity race raise awareness for? *Reference:* mental health
2. `conv-30`: How do Jon and Gina both like to destress? *Reference:* by dancing
3. `conv-41`: What is John's main focus in local politics? *Reference:* Improving education and infrastructure
4. `conv-42`: What is one of Joanna's favorite movies? *Reference:* "Eternal Sunshineof the Spotless Mind"
5. `conv-43`: Which team did John sign with on 21 May, 2023? *Reference:* The Minnesota Wolves

B.3 Memory configuration

- **Truncated full context.** The most recent turns that fit the reader's context budget, which is 4,096 tokens throughout, less a reserve for the prompt template and the answer.
- **Vector retrieval.** Turn-level embeddings with mean pooling, cosine similarity, and the ten most similar turns by default; Chapter 4 sweeps this from 2 to 50. Retrieved turns are re-sorted into chronological order before they enter the prompt. The embedding model expects task prefixes: stored turns are indexed as `search_document:` and questions are encoded as `search_query:` . Runs that predate this repair are identified where they appear. On the workstation the repair raised vector accuracy by 3.5 points for E2B and 6.0 for E4B, and changed triple-only graph accuracy by at most half a point.

- **Graph memory.** Triples are extracted from batches of five turns and stored in a directed multigraph. Edges start at weight 0.6 and are reinforced by 0.3 on repetition, capped at 1.0, with a decay factor of 1.0 and a pruning threshold of 0.15. Retrieval returns at most twelve triples. The extraction token budget is 256 in the baseline configuration and 512 in the repaired pipeline of Chapter 6.

B.4 Worked prompt example

The four context formats below come from the recorded Gemma 4 E2B workstation prompts for the LoCoMo question "What martial arts has John done?" (`conv-41`; reference answer: kickboxing and taekwondo), which Chapter 6 also discusses. Each block replaces `{context}` in the reader prompt; "[…]" marks omitted lines, and long lines are wrapped for display.

Truncated full context. 85 turns were sent and the server trimmed the oldest to fit 4,096 tokens. The retained suffix begins in August 2023, after every turn that mentions kickboxing or taekwondo. Answer: "John has not mentioned any martial arts." (judged incorrect).

```
[5:19 pm on 5 August, 2023] John: I appreciate it! Your words are really
motivating.
[...]
[11:08 am on 16 August, 2023] John: Yeah, Maria, let's keep each other and
everyone else motivated to make a difference! Together, our impact will
surely last.
```

Vector retrieval, $k = 10$. Ten turns re-sorted into chronological order, 628 prompt tokens. Answer: "kickboxing and taekwondo" (judged correct).

```
[11:01 am on 17 December, 2022] John: Woah, Maria, that sounds cool! I'm
doing kickboxing and it's giving me so much energy.
[6:10 pm on 22 December, 2022] John:  Yep, let's keep making great memories
with our loved ones and cherishing the time we have. I'm off to do some
taekwondo!
[...]
[6:21 pm on 22 July, 2023] John: Yeah, they offer a a bunch, like yoga,
kickboxing, and circuit training. It keeps things interesting!
[...]
```

Graph triples, version 3. Twelve selected triples rendered as timestamped statements, 511 prompt tokens. None names either sport. Answer: "No information available" (judged incorrect).

```
[6:03 pm on 6 March, 2023] System: Maria volunteering at Homeless shelter.
[11:08 am on 16 August, 2023] System: John expressed gratitude to Maria.
[...]
[6:21 pm on 22 July, 2023] System: Maria asked about location Martial arts
place or yoga studio.
[6:21 pm on 22 July, 2023] System: Shared photo depicts Two women standing in
a room full of black mats.
[11:01 am on 17 December, 2022] System: John returned from Family road trip
yesterday.
```

Graph triples with sources, version 4. The same twelve triples followed by five source turns, 774 prompt tokens. Answer: "kickboxing and taekwondo" (judged correct).

```
[... the twelve triples of version 3 ...]

Source: [11:01 am on 17 December, 2022] John: Woah, Maria, that sounds cool!
I'm doing kickboxing and it's giving me so much energy.
[...]
Source: [6:10 pm on 22 December, 2022] John:  Yep, let's keep making great
memories with our loved ones and cherishing the time we have. I'm off to do
some taekwondo!
[...]
```

Source-expansion loader correction. An early source-restoration run loaded a cached graph without reapplying the source-append option. The loader was corrected to restore the requested query settings, and that invalid attempt was excluded from the repair results.

B.5 Model artifacts

Every reported cell uses a community-distributed GGUF build at standard `Q4_K_M` quantization, with uniform per-layer bit widths. The single exception is the quantization check described in Chapter 3, which repeated one phone cell on an adaptive `UD-Q4_K_XL` build and reproduced its verdicts exactly. Table B.1 gives the identity of each file as recovered from the device cache.

Role	Repository	Size (GB)	SHA-256 (first 16)
Gemma 4 E2B reader	`unsloth/gemma-4-E2B-it-GGUF`	3.11	not recorded
Gemma 4 E4B reader	`unsloth/gemma-4-E4B-it-GGUF`	4.98	`85a896a047553e84`
Qwen 3.5 0.8B reader	`unsloth/Qwen3.5-0.8B-GGUF`	0.53	`bd258782e35f7f45`
Qwen 3.5 2B reader	`unsloth/Qwen3.5-2B-GGUF`	1.28	`aaf42c8b7c3cab2b`
Qwen 3.5 4B reader	`unsloth/Qwen3.5-4B-GGUF`	2.74	`00fe7986ff5f6b46`
Vector embedder	`nomic-ai/nomic-embed-text-v1.5-GGUF`	0.08	`d4e388894e09cf38`

Table B.1: On-device model artifacts. Digests were recomputed from the cached file bytes; full values and the repository revisions are kept with the run records. The E2B entry is the later of two revisions used during the campaign, discussed in Section B.10.

The automatic judge is `llama3.1:70b-instruct-q4_K_M`, served on the workstation and run at temperature 0 with a 256-token answer budget. Judging is performed on the workstation in every case and is outside the measured phone workload.

B.6 Runtime configuration

Workstation. Models are served through Ollama. The reader context window is 4,096 tokens and decoding is greedy at temperature 0. The long-window full-context reruns (Section 4.6) raise the window to 131,072 tokens, or 262,144 for Qwen3.5 4B on LongMemEval-S, and change nothing else.

Smartphone. Models are served locally through llama.cpp, one inference slot per server, with an 8,192-token slot context (16,384 for the long-window sittings), five threads pinned to the performance cluster through a CPU affinity mask, flash attention enabled, the model's own chat template applied, and the runtime's reasoning budget set to zero. Every phone sitting therefore runs without reasoning traces. The thinking conditions evaluated in the workstation experiments were not repeated on the device, so no on-device energy figure in this thesis describes a thinking configuration. The embedding server runs separately with mean pooling. Decoding is greedy, which is a deliberate deviation from the vendor-recommended sampling settings; Section 3.8 reports the check that compared it against the recommended operating point. Greedy decoding is what makes the replicate comparison possible: repeated sittings produced byte-identical predictions, so the variation between them is physical rather than sampling noise.

B.7 Software versions and phone server commands

Phone. The phone ran Android 15 with llama.cpp built from source in Termux. The binary reports version `0.1.2-dev` (build 0, commit not recorded) and was built with Clang 21.0.0 for Android aarch64. Readers were served with the command below (Gemma 4 E2B shown; the other readers differ only in the model reference), with `-c 16384` for the long-window sittings.

```
llama-server -hf unsloth/gemma-4-E2B-it-GGUF:Q4_K_M -c 8192 -np 1
    --cpu-mask f8 -t 5 -fa on --jinja --reasoning-budget 0 --cache-ram 0
```

Here `-np 1` gives one inference slot, `--cpu-mask f8` pins execution to cores 3 to 7 (the prime core and the four performance cores), `-t 5` sets five threads, `-fa on` enables flash attention, `--jinja` applies the model's own chat template, `--reasoning-budget 0` disables reasoning traces and `--cache-ram 0` disables the server's prompt cache between requests. The embedder ran as a separate server:

```
llama-server -hf nomic-ai/nomic-embed-text-v1.5-GGUF:Q4_K_M --embeddings
    --pooling mean
```

The `-hf` selector resolves the latest repository revision at download time; Table B.1 gives the file digests, and Section B.10 records the one revision change during the campaign.

Workstation. Ollama served the readers and the judge; its server version was not recorded in the run records. The harness's locked Python environment uses Python 3.12.10 with NumPy 2.4.3, SciPy 1.17.1, NetworkX 3.6.1 and the Ollama client library 0.6.2.

B.8 Instrumentation

Resource sampling runs alongside the harness, requested every 0.5 seconds (realized 1.2 to 3.2 seconds, median per run, because each battery query takes a second or more), and records the battery charge counter, voltage and current, memory from the kernel's own reporting, and the device-wide maximum temperature across the available sensors. Sampling starts after the model servers have loaded, so server start-up is excluded from the measured spans. A wake lock keeps the processor running with the screen off. Energy is obtained by voltage-weighted Coulomb counting from the charge counter rather than from a nominal-voltage assumption.

B.9 Analysis scripts

Every table, figure and statistical test in the thesis is produced from the stored per-run records by script, not transcribed by hand. Three scripts carry the reported numbers: one aggregates the accuracy table across runs, one derives the on-device energy metric and is the sole method of record for it, and one produces the statistical tests. Re-running them on the stored records regenerates the reported values.

B.10 Known limitations of this record

Two gaps are stated here rather than left for a reader to find.

1. **The primary phone retrieval campaign remains unprefixed.** The 20-question, ten-turn retrieval comparison was not rerun with prefixes. The prefixed auxiliary conditions are listed in Section 3.5; they do not isolate a prefix effect.

2. **The charge counter can stall at the start of a sitting.** Several sittings show a frozen counter for the first few minutes with no later catch-up, which under-counts energy. The affected sittings and the size of the effect are reported in Appendix A.

B.11 Glossary

Reader The model that receives the memory text and writes the answer.

E2B, E4B The two Gemma 4 readers, with about 2.3 and 4.5 billion effective parameters.

Workstation ("the box") The Linux host that runs the larger accuracy study through Ollama and hosts the judge.

Sitting One complete phone run of a workload, unplugged and measured from start to end; repeated sittings measure energy variation.

Truncated full context The newest dialogue turns that fit a 4,096-token budget, with older turns dropped.

Vector retrieval (Vector RAG) The k dialogue turns most similar to the question by embedding, re-sorted into chronological order; $k = 10$ by default.

SLM-extracted graph A graph of (subject, relation, object) triples extracted by the reader model itself. Figures and tables label its versions GraphRAG v1–v4; it is not Microsoft's GraphRAG (Section 6.1).

v1–v4 The four graph versions of Table 3.2; v3 shows triples only and v4 adds the source turns linked to them.

Source restoration The v4 step of adding the linked dialogue turns to the selected triples.

Extraction failure An extraction call whose output the pipeline could not parse; a valid empty list is counted separately.

SPE, SPE-1, SPE-2, B Selective Provenance Expansion, its heuristic and representation-specific scoring versions, and its retrieval-context token budget ($B = 240$ or 370).

Oracle A post hoc upper bound that chooses v3 or v4 per question using the judged labels; not an implementable policy.

Prefixed, unprefixed Retrieval with or without the embedding model's task prefixes (Section 3.5).

Prefill, decode Processing the input prompt, and generating the answer token by token.

KV cache The stored attention keys and values that let a model reuse earlier tokens during generation.

Correct answers per kilojoule (G_E) Phone-judged correct answers divided by the voltage-weighted energy of the same workload; G_Q is the same count per 5,000 mAh full charge.

Parity gate The predefined rule that decided whether workstation accuracy could stand in for phone accuracy.

Bibliography

Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic AI. *arXiv preprint arXiv:2506.02153*, 2025. doi:10.48550/arXiv.2506.02153. URL `https://arxiv.org/abs/2506.02153v2`. Position paper; version 2, 15 September 2025.

Massimo Bini, Ondrej Bohdal, Umberto Michieli, Zeynep Akata, Mete Ozay, and Taha Ceritli. MemLoRA: Distilling expert adapters for on-device memory systems. *arXiv preprint arXiv:2512.04763*, 2025. arXiv:2512.04763v1.

Jiayi Chen, Yingcong Li, and Guiling Wang. MemFlow: Intent-driven memory orchestration for small language model agents. *arXiv preprint arXiv:2605.03312*, 2026. doi:10.48550/arXiv.2605.03312. URL `https://arxiv.org/abs/2605.03312v1`. Version 1, 5 May 2026.

Nuo Chen, Hongguang Li, Juhua Huang, Baoyuan Wang, and Jia Li. Beyond retrieval: Embracing compressive memory in real-world long-term conversations. *arXiv preprint arXiv:2402.11975*, 2024. doi:10.48550/arXiv.2402.11975. URL `https://arxiv.org/abs/2402.11975v2`. Version 2, 1 July 2024. The arXiv record lists the alternative title "Compress to Impress: Unleashing the Potential of Compressive Memory in Real-World Long-Term Conversations".

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *ECAI 2025 – 28th European Conference on Artificial Intelligence*, Frontiers in Artificial Intelligence and Applications, pages 2993–3000. IOS Press, 2025. doi:10.3233/FAIA251160. Also available as arXiv:2504.19413. Venue verified via Crossref, 13 October 2026.

Hongchao Du, Shangyu Wu, Qiao Li, Riwei Pan, Jinheng Li, Youcheng Sun, and Chun Jason Xue. ClawMobile: Rethinking smartphone-native agentic systems. In *Proceedings of the Sixth European Workshop on Machine Learning and Systems (EuroMLSys '26)*, pages 370–376. Association for Computing Machinery, 2026. doi:10.1145/3805621.3807655. Also available as arXiv:2602.22942v2. Venue verified via Crossref, 13 October 2026.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. doi:10.48550/arXiv.2404.16130. URL `https://arxiv.org/abs/2404.16130v2`. Version 2, revised 19 February 2025.

Gemma Team, Google DeepMind. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026. Dated 19 June 2026.

Georgi Gerganov and ggml-org contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2026. Software.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. doi:10.48550/arXiv.2407.21783.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.findings-emnlp.568. URL `https://aclanthology.org/2025.findings-emnlp.568/`. Preprint first posted in 2024 as arXiv:2410.05779; version 3, 28 April 2025.

Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008. doi:10.25080/TCWV9851.

Haoyu Han, Li Ma, Yu Wang, Harry Shomer, Yongjia Lei, Zhisheng Qi, Kai Guo, Zhigang Hua, Bo Long, Hui Liu, Charu C. Aggarwal, and Jiliang Tang. RAG vs. GraphRAG: A systematic evaluation and key insights. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 8966–8977. Association for Computing Machinery, 2026. doi:10.1145/3770855.3817575. Also available as arXiv:2502.11371v3. Venue verified via Crossref, 13 October 2026.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware LLM reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24842–24855. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.findings-acl.1274. URL `https://aclanthology.org/2025.findings-acl.1274/`. Preprint first posted in 2024 as arXiv:2412.18547; version 5, 2 June 2025.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022. arXiv:2112.09118v4; TMLR 08/2022.

Jingbo Ji, Lingyi Li, Xilong Cheng, Yuhao Zhou, Wenji Zhang, Yuting Tan, and Yunxiao Qin. RippleMem: From isolated retrieval to associative recollection for long-term agent memory. *arXiv preprint arXiv:2608.13334*, 2026. doi:10.48550/arXiv.2608.13334. URL `https://arxiv.org/abs/2608.13334v1`. Version 1, 13 August 2026.

Zixia Jia, Jiaqi Li, Yipeng Kang, Yuxuan Wang, Tong Wu, Quansen Wang, Xiaobo Wang, Shuyi Zhang, Junzhe Shen, Qing Li, Siyuan Qi, Yitao Liang, Di He, Zilong Zheng, and Song-Chun Zhu. The AI hippocampus: How far are we from human memory? *Transactions on Machine Learning Research*, 2025. arXiv:2601.09113v1; TMLR 11/2025.

Zhihan Jiang, Meng Li, Shenghao Liu, Keran Li, Ruiben Zhou, Wei Wang, Xianjun Deng, Shuai Wang, and Haipeng Dai. SmartRAG: Native graph-based RAG for mobile device. *arXiv preprint arXiv:2607.14661*, 2026. arXiv:2607.14661v2.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, 2024. doi:10.48550/arXiv.2405.14831. URL `https://arxiv.org/abs/2405.14831v3`. arXiv version 3, revised 14 January 2025.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020.

Dimitrios Kelesis, Konstantinos Bougiatiotis, and Georgios Paliouras. Enhancing small language models reasoning through knowledge graph grounding. In *Proceedings of the 2nd Workshop on Causal Neuro-symbolic Artificial Intelligence (Causal NeSy)*. Institute of Informatics and Telecommunications, NCSR "Demokritos", 2026.

Songxin Lei, Kun Ouyang, Weilin Ruan, Yuqian Wu, Zhijiang Guo, Yushi Sun, and Fugee Tsung. MemoryCPT: An end-to-end agent memory framework for cost-performance tradeoff. *arXiv preprint arXiv:2608.04843*, 2026. arXiv:2608.04843, submitted 5 August 2026; no publication venue asserted.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. arXiv:2005.11401v4.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353. Association for Computational Linguistics, 2023. doi:10.18653/v1/2023.emnlp-main.391. URL `https://aclanthology.org/2023.emnlp-main.391/`. Also available as arXiv:2310.06201v1.

Yuxin Liao, Le Wu, Min Hou, Hao Liu, Han Wu, and Zishu Wang. LeanMem: Simple and efficient long-term memory for LLM agents. *arXiv preprint arXiv:2608.03463*, 2026. arXiv:2608.03463v1; no publication venue asserted.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:10.1162/tacl_a_00638. URL `https://aclanthology.org/2024.tacl-1.9/`. Preprint first posted in 2023 as arXiv:2307.03172.

Zibin Liu, Cheng Zhang, Xi Zhao, Yunfei Feng, Bingyu Bai, Dahu Feng, Erhu Feng, Yubin Xia, and Haibo Chen. Beyond training: Enabling self-evolution of agents with MOBIMEM. *arXiv preprint arXiv:2512.15784*, 2025. arXiv:2512.15784.

Zhengxuan Lu, Dongfang Li, Yukun Shi, Beilun Wang, Longyue Wang, and Baotian Hu. Structured episodic event memory. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6125–6141. Association for Computational Linguistics, 2026. URL `https://aclanthology.org/2026.acl-long.277/`.

Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Wei Liu, Jian Luan, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. Demystifying small language models for edge deployment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14747–14764. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.acl-long.718.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870. Association for Computational Linguistics, 2024. doi:10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi:10.1007/BF02295996.

Nomic AI. nomic-embed-text-v1.5: Model card. Hugging Face. URL https://huggingface.co/nomic-ai/nomic-embed-text-v1.5. Task instruction prefixes section. Accessed 6 September 2026.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=IPmzyQSiQE. arXiv:2402.01613v2.

Ollama. Ollama. https://github.com/ollama/ollama, 2026. Software.

Kai Tzu-iunn Ong, Namyoung Kim, Minju Gwak, Hyungjoo Chae, Taeyoon Kwon, Yohan Jo, Seung-won Hwang, Dongha Lee, and Jinyoung Yeo. Towards lifelong dialogue agents via timeline-based memory management. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8631–8661. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.naacl-long.435. URL https://aclanthology.org/2025.naacl-long.435/. Preprint first posted in 2024 as arXiv:2406.10996; version 3, 29 January 2025.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. doi:10.48550/arXiv.2310.08560. URL https://arxiv.org/abs/2310.08560v2. First posted in 2023; version 2, 12 February 2024.

Taehwan Park, Geonho Lee, and Min-Soo Kim. MobileRAG: A fast, memory-efficient, and energy-efficient method for on-device RAG. *arXiv preprint arXiv:2507.01079*, 2025. arXiv:2507.01079v1.

Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*, 2025. doi:10.1145/3696410.3714805. arXiv:2409.05591v3.

Qwen Team. Qwen3.5: Towards native multimodal agents. Qwen blog, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

Jon Saad-Falcon, Avanika Narayan, Hakki Orhun Akengin, J. Wes Griffin, Herumb Shandilya, Adrian Gamarra Lafuente, Medhya Goel, Rebecca Joseph, Shlok Natarajan, Etash Kumar Guha, Shang Zhu, Ben Athiwaratkun, John Hennessy, Azalia Mirhoseini, and Christopher Ré. Intelligence per watt: Measuring intelligence efficiency of local AI. *arXiv preprint arXiv:2511.07885*, 2025. doi:10.48550/arXiv.2511.07885. URL https://arxiv.org/abs/2511.07885v6. First posted in 2025; the authors' arXiv record (version 7) lists the paper as appearing at NeurIPS 2026.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*, 2024. doi:10.48550/arXiv.2401.18059. URL https://openreview.net/forum?id=GN921JHCRw. Also available as arXiv:2401.18059v1; DOI identifies the preprint.

Haoran Sun, Shaoning Zeng, and Bob Zhang. H-MEM: Hierarchical memory for high-efficiency long-term reasoning in LLM agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 341–350. Association for Computational Linguistics, 2026. doi:10.18653/v1/2026.eacl-long.15. Preprint first posted in 2025 as arXiv:2507.22925. Venue verified via Crossref, 13 October 2026.

Unsloth AI. Unsloth GGUF model quantizations. https://huggingface.co/unsloth, 2026. Hugging Face model repositories; exact files are listed in the reproducibility appendix.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html.

Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*, 2024. arXiv:2409.00088v2.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi:10.52202/085713-0593. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html. Also available as arXiv:2502.12110v11, 8 October 2025.

Xiucheng Xu, Bingbing Xu, Xueyun Tian, Zihe Huang, Rongxin Chen, Yunfan Li, and Huawei Shen. Chain-of-memory: Lightweight memory construction with dynamic evolution for LLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11618–11631. Association for Computational Linguistics, 2026. doi:10.18653/v1/2026.acl-long.534. URL https://aclanthology.org/2026.acl-long.534/.

Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, Yujing Zhang, Zhu Wang, Zijin Hong, Zheng Yuan, Zhishang Xiang, Shengyuan Chen, Huachi Zhou, Qinggang Zhang, Ninghao Liu, Jinsong Su, Xinrun Wang, Yi Chang, and Xiao Huang. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026. doi:10.48550/arXiv.2602.05665. URL https://arxiv.org/abs/2602.05665v1. Version 1, 5 February 2026.

Jianing Yin, Tan Tang, and Yingcai Wu. DeferMem: Query-time evidence distillation via reinforcement learning for long-term agent memory. *arXiv preprint arXiv:2605.22411*, 2026. doi:10.48550/arXiv.2605.22411. URL https://arxiv.org/abs/2605.22411v2. Version 2; title and author list follow the current arXiv record.

Jiawei Yu, Yixiang Fang, Xilin Liu, and Yuchi Ma. H-MEM: A novel memory mechanism for evolving and retrieving agent memory via a hybrid structure. *arXiv preprint*

arXiv:2605.15701, 2026a. arXiv:2605.15701v1; distinct from the identically named system of Sun and Zeng.

Jing Yu, Yibo Zhao, Jiaming Zhang, and Xiang Li. LazyMem: Retrieve broadly, construct selectively for efficient long-term agent memory. *arXiv preprint arXiv:2607.22690*, 2026b. arXiv preprint, under review.

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026a. doi:10.48550/arXiv.2602.02474. URL `https://arxiv.org/abs/2602.02474v2`. Version 2, 24 May 2026.

Haozhen Zhang, Haodong Yue, Tao Feng, Quanyu Long, Jianzhu Bao, Bowen Jin, Weizhi Zhang, Xiao Li, Jiaxuan You, Chengwei Qin, and Wenya Wang. Learning query-aware budget-tier routing for runtime agent memory. *arXiv preprint arXiv:2602.06025*, 2026b. arXiv:2602.06025v3; the authors' arXiv record states acceptance at ICML 2026.

Liyu Zhang, Huanqi Yang, and Xiaomin Ouyang. Benchmarking memory architectures for mobile agents in short-term and long-term applications. In *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services Workshops*, pages 175–181. Association for Computing Machinery, 2026c. doi:10.1145/3812836.3814752.

Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43:1–47, 2025. doi:10.1145/3748302. Preprint first posted in 2024 as arXiv:2404.13501. Venue verified via Crossref, 13 October 2026.

Zicheng Zhao, Xinyang Guo, Luyao Lv, Menghan Wang, Ming Li, and Shuaicheng Li. Accurate and efficient long-term memory for LLM agents. *arXiv preprint arXiv:2607.16211*, 2026. MOSAIC; arXiv:2607.16211, no publication venue asserted.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi:10.52202/075280-2020. URL `https://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html`. Datasets and Benchmarks Track; also available as arXiv:2306.05685v4.